

Systematische Betrachtung ausgewählter Workflow-Management-Systeme für HPC-Umgebungen

Anas Tahiri

25.08.2026

Version: CleanThesis 0.4.1 - JGU patches



Johannes Gutenberg-Universität Mainz

FB 08

Institut für Informatik

Bachelorarbeit

Systematische Betrachtung ausgewählter Workflow-Management-Systeme für HPC-Umgebungen

Anas Tahiri

Matrikelnummer: 2777866

1. Gutachterin

Prof. Dr. Sarah Neuwirth

Institut für Informatik / Zentrum für Datenverarbeitung
Johannes Gutenberg-Universität Mainz

2. Gutachter

Prof. Dr. Nicolai Kuntze

Angewandte Informatik mit Schwerpunkt IT-Sicherheit
Hochschule Mainz

Betreuer

Zhaobin Zhu und Niklas Bartelheimer

25.08.2026

Anas Tahiri

Systematische Betrachtung ausgewählter Workflow-Management-Systeme für HPC-Umgebungen

Bachelorarbeit, 25.08.2026

Gutachter: Prof. Dr. Sarah Neuwirth und Prof. Dr. Nicolai Kuntze

Betreuer: Zhaobin Zhu und Niklas Bartelheimer

Johannes Gutenberg-Universität Mainz

Institut für Informatik

FB 08

Staudingerweg 9

55128 Mainz

Zusammenfassung

Diese Arbeit vergleicht die fünf Workflow-Management-Systeme Nextflow, StreamFlow, Merlin, Pegasus und AiiDA anhand zweier Workflow-Muster, einer Pipeline und eines Scatter-Gather-Musters, die in allen Systemen mit derselben Rechenlogik umgesetzt werden. Betrachtet werden die Modellierung der Workflows, das Laufzeitverhalten anhand von Makespan, Overhead und Koordinationsgrößen sowie der Betrieb in einer HPC-Umgebung. Zunächst werden alle fünf Systeme unter kontrollierten Bedingungen auf einem einzelnen Rechner verglichen. Anschließend wird der Einsatz auf dem HPC-System MOGON unter Slurm betrachtet, auf dem vier der fünf Systeme betrieben werden konnten. Beide Workflow-Muster ließen sich in allen fünf Systemen umsetzen. Die wesentlichen Unterschiede zeigen sich in der Art der Workflow-Beschreibung, im Laufzeitverhalten und bei der Koordination der Schritte sowie im Betriebsmodell. Beim Laufzeitverhalten liegen StreamFlow, Nextflow und Merlin beim Overhead in einem ähnlichen Bereich, während AiiDA höhere Werte aufweist und Pegasus in der lokalen Untersuchung deutlich darüber liegt. Mit zunehmender Rechenlast verliert der Overhead gegenüber der eigentlichen Rechenzeit an Bedeutung. Auf MOGON zeigte sich zudem, dass durch die Clusterumgebung verursachte Zeitanteile die Unterschiede zwischen den Systemen überlagern können. Insgesamt unterscheiden sich die Systeme sowohl in der Modellierung und Ausführung der Workflows als auch in ihrem Betrieb in der HPC-Umgebung.

Inhaltsverzeichnis

1. Einleitung	1
1.1. Motivation	1
1.2. Zielsetzung und Forschungsfragen	2
1.3. Aufbau der Arbeit	2
2. Grundlagen und verwandte Arbeiten	5
2.1. Wissenschaftliche Workflows und Workflow-Management-Systeme	5
2.2. Workflow-Struktur und Ausführung	6
2.3. HPC-Systeme und Ressourcenverwaltung	7
2.4. Untersuchte Workflow-Management-Systeme	8
2.4.1. Nextflow	8
2.4.2. StreamFlow	9
2.4.3. Merlin	9
2.4.4. Pegasus	10
2.4.5. AiiDA	10
2.5. Verwandte Arbeiten und Abgrenzung	11
3. Methodik	13
3.1. Evaluationsdesign	13
3.2. Auswahl der Systeme	14
3.3. Qualitative Vergleichsaspekte	15
3.4. Quantitative Messgrößen	17
3.5. Benchmark-Workflows und Versuchsplan	18
3.6. Messverfahren	20
3.6.1. Referenzmessung	23
3.6.2. Äußere Messung	23
3.6.3. Innere Instrumentierung	24
3.6.4. Auswertung	26
3.6.5. Grenzen des Messverfahrens	27
4. Modellierung und lokale Evaluation	29
4.1. Versuchsumgebung und Systembereitstellung	29
4.2. Pipeline-Muster	31
4.2.1. Nextflow	31

4.2.2.	StreamFlow	33
4.2.3.	Merlin	34
4.2.4.	Pegasus	36
4.2.5.	AiiDA	37
4.2.6.	Vergleichende Beobachtungen	39
4.3.	Scatter-Gather-Muster	41
4.3.1.	Nextflow	41
4.3.2.	StreamFlow	42
4.3.3.	Merlin	44
4.3.4.	Pegasus	45
4.3.5.	AiiDA	47
4.3.6.	Vergleichende Beobachtungen	48
4.4.	Ergebnisse der lokalen Messungen	50
4.5.	Zwischenfazit	55
5.	Evaluation auf dem HPC-System MOGON	57
5.1.	Zielsetzung	57
5.2.	Versuchsumgebung	58
5.3.	Bereitstellung und Betrieb der Workflow-Management-Systeme	58
5.3.1.	Nextflow	59
5.3.2.	StreamFlow	60
5.3.3.	Merlin	62
5.3.4.	Pegasus	63
5.3.5.	AiiDA	64
5.4.	Ausführung im Slurm-Betrieb	65
5.5.	Aufbau der dedizierten Messung	68
5.6.	Ergebnisse der dedizierten Messung	70
5.7.	Einfluss der AiiDA-Konfiguration	74
5.8.	Zwischenfazit	75
6.	Diskussion	77
6.1.	Beantwortung der Forschungsfragen	77
6.2.	Grenzen der Arbeit	81
7.	Fazit und Ausblick	85
7.1.	Fazit	85
7.2.	Ausblick	86
	Literatur	87

Abbildungsverzeichnis	91
Tabellenverzeichnis	93
Programmausdrucksverzeichnis	95
A. Ergänzende Grafiken	97
A.1. Laufartefakte in Nextflow	97
A.2. Laufartefakte in StreamFlow	99
A.3. Laufartefakte in Pegasus	99
A.4. Laufartefakte in AiiDA	102

Einleitung

Wissenschaftliches Rechnen besteht häufig nicht aus einer einzelnen Berechnung, sondern aus mehreren aufeinander aufbauenden Rechenschritten. Ausgeführt werden solche Abläufe sowohl auf einzelnen Rechnern als auch auf High-Performance-Computing-Systemen (HPC-Systemen). Mit zunehmender Zahl der Schritte wird es aufwendiger, ihre Ausführung und die dabei entstehenden Daten zu verwalten.

1.1 Motivation

Wissenschaftliche Workflows bestehen aus mehreren miteinander verbundenen Rechenschritten, zwischen denen Daten weitergegeben werden und die in ihrer Ausführung voneinander abhängen können. Bei komplexeren Abläufen wird es dadurch zunehmend aufwendig, die einzelnen Schritte von Hand in der richtigen Reihenfolge auszuführen und die benötigten Daten zwischen ihnen weiterzugeben. Workflow-Management-Systeme automatisieren diese Abläufe [3, S. 28 f.][39, S. 2]. Es steht eine große Zahl solcher Systeme zur Verfügung. Die Wahl eines Systems wird dabei unter anderem durch die Anforderungen der jeweiligen Fachgemeinschaft, die Verbreitung einzelner Systeme oder historische Gründe beeinflusst [3, S. 28]. Eine einzelne, für alle wissenschaftlichen Prozesse und Ausführungsumgebungen geeignete Lösung gibt es nicht [39, S. 2]. Wer ein System auswählen will, steht damit vor der Frage, wie sich die Systeme in der Beschreibung von Workflows, im Laufzeitverhalten und im Betrieb unterscheiden. Für den Einsatz auf einem HPC-System kommt zusätzlich die Frage hinzu, wie sich die Systeme dort bereitstellen und mit dem Ressourcenverwalter betreiben lassen, der die verfügbaren Rechenressourcen auf die auszuführenden Aufgaben verteilt.

Hier setzt die vorliegende Arbeit an. Sie vergleicht mit Nextflow, StreamFlow, Merlin, Pegasus und AiiDA fünf Workflow-Management-Systeme anhand zweier Workflow-Muster, einer Pipeline und eines Scatter-Gather-Musters, die in allen Systemen mit derselben Rechenlogik umgesetzt werden. Die Systeme

werden zunächst auf einem einzelnen Rechner und anschließend auf dem HPC-System MOGON unter Slurm untersucht. Dabei werden sowohl qualitative Unterschiede zwischen den Systemen als auch ihr Laufzeitverhalten betrachtet.

1.2 Zielsetzung und Forschungsfragen

Ziel der Arbeit ist ein systematischer Vergleich ausgewählter Workflow-Management-Systeme hinsichtlich ihrer Modellierung, ihres Laufzeitverhaltens und ihres Einsatzes in einer HPC-Umgebung. Die Arbeit orientiert sich an vier Forschungsfragen. Die erste Frage betrifft die Modellierung: Wie unterscheiden sich die betrachteten Workflow-Management-Systeme bei der Umsetzung eines Pipeline- und eines Scatter-Gather-Workflows (F1)? Die zweite Frage richtet sich auf das Laufzeitverhalten: Welches Laufzeitverhalten zeigen die Systeme bei der Ausführung dieser Workflows, gemessen an Gesamtlaufzeit (Makespan), Overhead und Koordinationsgrößen (F2)? Die Koordinationsgrößen erfassen dabei zeitliche Abstände beim Übergang zwischen aufeinanderfolgenden Schritten und beim Start paralleler Verarbeitungsinstanzen. Die dritte Frage gilt dem Einsatz auf dem HPC-System: Wie lassen sich die Systeme auf einem Slurm-basierten HPC-System betreiben, und wie setzen sich die Laufzeiten unter Clusterbedingungen zusammen (F3)? Die vierte Frage zielt auf die Einordnung: Für welche Arten von Workflows und Anwendungsfällen erscheinen die einzelnen Systeme besonders geeignet (F4)?

1.3 Aufbau der Arbeit

Die Arbeit ist wie folgt aufgebaut. Kapitel 2 erläutert die Grundlagen wissenschaftlicher Workflows und HPC-Systeme, stellt die betrachteten Workflow-Management-Systeme vor und ordnet die Arbeit gegenüber verwandten Arbeiten ein. Kapitel 3 beschreibt das Evaluationsdesign, die verwendeten Workflow-Muster und das Vorgehen für den Vergleich der Systeme. Kapitel 4 beschreibt die Umsetzung der Workflow-Muster in den fünf Systemen, vergleicht sie anhand der qualitativen Kriterien und stellt die Ergebnisse der lokalen Messungen dar. Kapitel 5 betrachtet anschließend den Einsatz der

Systeme auf dem HPC-System MOGON und die dort durchgeführten Messungen. Kapitel 6 diskutiert die Ergebnisse entlang der Forschungsfragen und ordnet die Eignung der Systeme sowie die Grenzen der Arbeit ein. Kapitel 7 fasst die Ergebnisse zusammen und gibt einen Ausblick.

Grundlagen und verwandte Arbeiten

Dieses Kapitel führt die Begriffe ein, auf denen die Arbeit aufbaut. Es beschreibt zunächst wissenschaftliche Workflows und Workflow-Management-Systeme, anschließend die Strukturen, aus denen Workflows zusammengesetzt sind, sowie den Aufbau von HPC-Systemen und die Ressourcenverwaltung mit Slurm. Danach werden die fünf untersuchten Workflow-Management-Systeme vorgestellt und die Arbeit gegenüber verwandten Arbeiten eingeordnet.

2.1 Wissenschaftliche Workflows und Workflow-Management-Systeme

Wissenschaftliche Workflows beschreiben wissenschaftliche Berechnungs- oder Analyseprozesse, die aus mehreren miteinander verbundenen Rechenschritten bestehen. Ein solcher Workflow verfolgt ein bestimmtes Forschungs- oder Analyseziel und beschreibt, wie die einzelnen Schritte innerhalb des Ablaufs zusammenhängen. Dazu gehören insbesondere ihre Reihenfolge sowie die zwischen ihnen bestehenden Abhängigkeiten [39, S. 2]. Anschaulich lässt sich diese Struktur als Graph darstellen. Die Knoten stehen dabei für die einzelnen Rechenschritte, in der Literatur meist als Tasks bezeichnet, während die Kanten die Abhängigkeiten zwischen ihnen darstellen [3, S. 28 f.]. Eine verbreitete Form eines Rechenschritts ist ein eigenständiges ausführbares Programm, das Eingabedaten verarbeitet und daraus Ausgaben erzeugt [39, S. 4]. Die zwischen den Rechenschritten ausgetauschten Daten liegen dabei häufig als Dateien vor [3, S. 29]. In dieser Arbeit wird durchgehend der Begriff Schritt verwendet.

Die Ausführung eines Workflows wird durch ein Workflow-Management-System gesteuert. Badia et al. definieren ein solches System als Softwareumgebung, die die Ausführung einer Menge voneinander abhängiger Re-

chenaufgaben orchestriert, welche untereinander Daten austauschen, um ein bestimmtes Experiment durchzuführen [3, S. 28]. Zu seinen Aufgaben gehören je nach System die Planung und Koordination der Ausführung, die Übertragung von Daten sowie die Überwachung des Ausführungszustands [3, S. 29][39, S. 2].

2.2 Workflow-Struktur und Ausführung

Die Abhängigkeiten zwischen den Schritten bestimmen, wann ein Schritt ausgeführt werden kann. Badia et al. unterscheiden dabei Datenabhängigkeiten von Kontrollabhängigkeiten [3, S. 28]. Eine Datenabhängigkeit besteht, wenn ein Schritt Daten benötigt, die von einem anderen Schritt erzeugt werden. Eine Kontrollabhängigkeit besteht dagegen, wenn der Abschluss eines Schritts Voraussetzung dafür ist, dass ein anderer Schritt ausgeführt werden kann [42, S. 259]. Das Workflow-Management-System orchestriert die Ausführung so, dass die bestehenden Abhängigkeiten eingehalten werden [39, S. 4].

Aus den Abhängigkeiten ergibt sich, welche Schritte nacheinander und welche gleichzeitig ausgeführt werden können. Bei einer sequenziellen Ausführung werden aufeinanderfolgende Schritte einer nach dem anderen ausgeführt [29, S. 8]. Ein Ausführungszweig kann sich aber auch in mehrere Zweige aufteilen, die gleichzeitig ausgeführt werden [29, S. 9]. Mehrere Zweige können anschließend wieder in einem gemeinsamen Folgeschritt zusammengeführt werden, der erst beginnt, wenn alle eingehenden Zweige abgeschlossen sind [29, S. 10].

Wissenschaftliche Workflows können aus solchen einfachen Strukturen zu komplexeren Abläufen zusammengesetzt sein [4, S. 2]. Bharathi et al. beschreiben als einfachste Struktur einen einzelnen Verarbeitungsschritt, der Eingabedaten verarbeitet und daraus eine Ausgabe erzeugt [4, S. 3]. Werden mehrere solcher Schritte hintereinandergeschaltet, entsteht eine Pipeline, in der jeder Schritt auf der Ausgabe des vorherigen arbeitet und seine Ausgabe an den nächsten weitergibt [4, S. 3]. Daneben beschreiben sie Schritte, die einen Datensatz in kleinere Teilstücke zerlegen, damit nachfolgende Schritte diese parallel verarbeiten können, sowie Schritte, die die Ergebnisse mehrerer Schritte zu einem gemeinsamen Ergebnis zusammenführen [4, S. 3]. Suter et al. nennen Datenverarbeitungs-Pipelines und Fan-out/Fan-in-Muster entsprechend als einfache Formen wissenschaftlicher Workflows [39, S. 4].

Die Verbindung aus Zerlegen, paralleler Verarbeitung der Teilstücke und Zusammenführen wird in dieser Arbeit als Scatter-Gather-Muster bezeichnet. Die parallel ausgeführten Einheiten desselben Schritts werden im Folgenden als Verarbeitungsinstanzen bezeichnet.

2.3 HPC-Systeme und Ressourcenverwaltung

High-Performance Computing (HPC) dient der Bearbeitung rechenintensiver Probleme, deren Berechnung auf einem einzelnen Rechner zu aufwendig oder zu zeitintensiv wäre. Dafür kommen unter anderem Supercomputer zum Einsatz, die aus hunderten oder tausenden über ein Hochgeschwindigkeitsnetzwerk verbundenen Rechnern bestehen können [33]. Der Zugang zu einem HPC-System erfolgt über Login-Knoten, auf denen Nutzende Daten bereitstellen, Rechenaufträge vorbereiten und einreichen sowie deren Zustand verfolgen können [33]. Die eigentlichen Berechnungen werden dagegen auf den Rechenknoten ausgeführt [33]. Da mehrere Aufträge um die verfügbaren Rechenressourcen konkurrieren können, muss deren Zuteilung koordiniert werden. Ein Ressourcenverwalter teilt dazu den Zugriff auf Rechenressourcen zu und verwaltet konkurrierende Anforderungen [34]. Workflow-Management-Systeme können die Ressourcenzuteilung an einen solchen Ressourcenverwalter delegieren. In HPC-Systemen übernimmt diese Aufgabe typischerweise ein Batch-Scheduler, der eingereichte Rechenaufträge zur Ausführung einplant [39, S. 5]. In dieser Arbeit wird hierfür Slurm eingesetzt, ein System zur Ressourcenverwaltung und Job-Steuerung auf Linux-Clustern [34].

Rechenaufträge werden bei Slurm als Jobs eingereicht. Für Batch-Jobs erfolgt dies mit dem Kommando `sbatch`, das ein Jobskript an Slurm übermittelt. Das Skript kann über `#SBATCH`-Direktiven unter anderem die benötigten CPUs, den Hauptspeicher, ein Zeitlimit und die gewünschte Partition festlegen [35]. Nach erfolgreicher Einreichung weist Slurm dem Job eine Job-ID zu. Stehen die angeforderten Ressourcen nicht unmittelbar zur Verfügung, kann der Job zunächst in der Warteschlange ausstehender Jobs verbleiben, bis die benötigten Ressourcen verfügbar sind [35]. Die Rechenknoten sind in Slurm in Partitionen gegliedert, also in Gruppen von Knoten, innerhalb derer den Jobs Ressourcen zugeteilt werden [34]. Teilt Slurm einem Job die angeforderten Ressourcen zu, erhält er eine Allokation, also den Zugriff auf diese Ressourcen für eine begrenzte Dauer [34]. Der Zugriff kann exklusiv oder geteilt

mit anderen Jobs erfolgen. Bei einer exklusiven Allokation teilt der Job die zugeteilten Knoten nicht mit anderen laufenden Jobs [35]. Nach erteilter Allokation führt Slurm das Jobskript auf dem ersten der zugeteilten Knoten aus [35]. Abbildung 2.1 fasst diesen Ablauf zusammen.

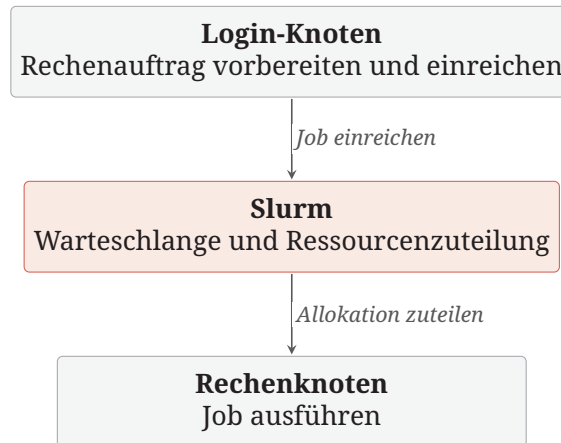


Abb. 2.1.: Ablauf von der Einreichung eines Rechenauftrags bis zu seiner Ausführung auf einem Slurm-verwalteten HPC-System.

Vor der eigentlichen Ausführung eines Jobs können auf den zugeteilten Rechenknoten systemseitige Vorbereitungen stattfinden. Slurm stellt dafür unter anderem Prolog-Programme bereit, die auf den Rechenknoten ausgeführt werden, bevor dort die Aufgaben des Jobs beginnen [32]. Diese vorbereitenden Arbeiten sind von der eigentlichen Nutzlast des Jobs zu unterscheiden. Slurm überträgt ausführbare Dateien und Daten nicht automatisch auf die einem Job zugeteilten Knoten. Die benötigten Dateien müssen daher entweder auf dem lokalen Speicher des jeweiligen Knotens oder in einem gemeinsam erreichbaren Dateisystem vorliegen [34]. Werden aufeinanderfolgende Schritte eines Workflows auf unterschiedlichen Knoten ausgeführt, müssen die zwischen ihnen weitergegebenen Dateien für die jeweils nachfolgenden Knoten erreichbar sein, beispielsweise über ein gemeinsames Dateisystem.

2.4 Untersuchte Workflow-Management-Systeme

2.4.1 Nextflow

Nextflow ist ein Workflow-Management-System aus der Bioinformatik. Es wurde entwickelt, um rechnergestützte Analysen über unterschiedliche Re-

chenumgebungen hinweg reproduzierbar zu machen, und setzt dafür unter anderem auf Container [11, S. 316]. Workflows werden in einer systemeigenen Sprache beschrieben, die auf dem Datenfluss-Modell beruht. Ein Workflow setzt sich aus Prozessen zusammen, die über Channels verbunden werden, indem die Ausgaben eines Prozesses als Eingaben eines anderen dienen. Ein Prozess startet, sobald seine Eingaben über die Channels eintreffen [11, S. 316 f.].

2.4.2 StreamFlow

StreamFlow ist ein Workflow-Management-System, das die Ausführung eines Workflows über mehrere, auch unterschiedliche Ausführungsumgebungen hinweg ermöglicht, etwa über Cloud- und HPC-Ressourcen gemeinsam [8, S. 1723]. Dazu erweitert StreamFlow ein klassisches Workflow-Modell um die Beschreibung verschiedener Ausführungsumgebungen und die Zuordnung von Workflow-Schritten zu diesen Umgebungen [8, S. 1723]. Als Workflow-Beschreibung verwendet StreamFlow die Common Workflow Language (CWL) [8, S. 1723].

2.4.3 Merlin

Merlin ist ein Workflow-Management-System aus dem HPC-Umfeld. Es wurde entwickelt, um sehr große Ensembles von Simulationsläufen zu erzeugen, etwa als Trainingsdaten für maschinelles Lernen [28, S. 255 f.]. Workflows werden in einer Spezifikation beschrieben, in der eine Studie aus Schritten mit Kommandos, Abhängigkeiten und Parametern festgelegt wird [28, S. 258]. Für die Ausführung verwendet Merlin Celery, ein verteiltes System zur Aufgabenverwaltung [28, S. 258][@19]. Merlin übersetzt die Schritte des Workflows dazu in Aufgaben und stellt diese über einen Nachrichtenvermittler (Broker) in einer Warteschlange bereit [28, S. 258][@19]. Worker sind Prozesse, die Aufgaben aus dieser Warteschlange beziehen und ausführen [28, S. 258][@19]. Dadurch sind die Worker von der Erzeugung der Aufgaben entkoppelt und können unabhängig davon auf den Rechenressourcen gestartet werden [28, S. 261].

2.4.4 Pegasus

Pegasus richtet sich auf großskalige, mehrstufige Simulations- und Analyse-Abläufe, deren Rechen- und Datenumfang ein System erfordert, das Datentransfer und Aufgabenausführung auf verteilten Rechenressourcen koordiniert und automatisiert [10, S. 17]. Das System beruht auf der Trennung zwischen der Beschreibung eines Workflows und seiner konkreten Ausführung. Nutzende erstellen einen abstrakten Workflow, den Pegasus in einen ausführbaren Workflow überführt [10, S. 18]. Der abstrakte Workflow wird als gerichteter azyklischer Graph in einem XML-Format namens DAX beschrieben [10, S. 19]. Die Überführung in einen ausführbaren Workflow übernimmt ein Planer, der Mapper. Dabei bestimmt er unter anderem, auf welchen Ressourcen die Jobs ausgeführt und von welchen Speicherorten benötigte Daten bezogen werden [10, S. 20]. Für die anschließende Ausführung verwendet Pegasus standardmäßig HTCondor DAGMan als Workflow-Engine. DAGMan berücksichtigt die Abhängigkeiten des ausführbaren Workflows und gibt einen Job zur Ausführung frei, sobald dessen Vorgänger abgeschlossen sind [10, S. 20]. Die einzelnen Jobs werden dabei über HTCondor verwaltet und können je nach Ausführungsumgebung lokal oder an entfernte Ressourcenverwalter übergeben werden [10, S. 20].

2.4.5 AiiDA

AiiDA stellt die Nachvollziehbarkeit der erzeugten Daten in den Mittelpunkt. Provenance bezeichnet dabei die Herkunft eines Datenelements und den Vorgang, durch den es entstanden ist [5, S. 316]. Jede vom System ausgeführte Berechnung und jeder Workflow werden automatisch in einem Provenance-Graphen aufgezeichnet, in dem Prozesse und Daten als Knoten und ihre Beziehungen als Kanten festgehalten werden, um die Reproduzierbarkeit der Ergebnisse zu ermöglichen [15, S. 2]. Für die Ablage stellt AiiDA zwei Speicher bereit: Die Eigenschaften der Knoten liegen als Attribute in einer relationalen Datenbank, Dateien werden in einem Dateirepository abgelegt [15, S. 8]. Workflows werden in Python beschrieben. Dazu stellt AiiDA die Klasse `WorkChain` bereit, in der die Schritte als logischer Ablauf in einer Prozessspezifikation festgelegt werden [15, S. 4 f.]. Berechnungen, die externe Programme ausführen, etwa über einen Job-Scheduler, übernimmt die Klasse `CalcJob` [15, S. 4]. Für die Ausführung verwendet AiiDA mehrere Runner, die die eingereichten Prozesse abarbeiten. Sie werden von einem Daemon überwacht, einem

dauerhaft im Hintergrund laufenden Prozess [15, S. 3]. Die dabei anfallenden Aufgaben werden über eine Aufgabenwarteschlange verteilt, die mit dem Nachrichtenvermittler RabbitMQ umgesetzt ist [15, S. 3].

2.5 Verwandte Arbeiten und Abgrenzung

Workflow-Management-Systeme wurden bereits in verschiedenen Arbeiten anhand unterschiedlicher Kriterien verglichen. Diercks et al. leiten Anforderungen an Workflow-Management-Systeme aus Anwendungsszenarien des Computational Science and Engineering ab und vergleichen sechs Systeme anhand dieser Anforderungen. Im Mittelpunkt stehen dabei insbesondere die Beschreibung, Reproduzierbarkeit und Wiederverwendung wissenschaftlicher Workflows [12, S. 1]. Singh et al. vergleichen fünf wissenschaftliche Workflow-Management-Systeme anhand qualitativer Merkmale wie Orchestrierung, Datenverwaltung und Provenance. Dabei betrachten sie die Systeme insbesondere im Hinblick auf daten- und rechenintensive wissenschaftliche Workflows [31, S. 4553]. Larssonneur et al. vergleichen fünf Workflow-Management-Systeme, darunter Nextflow und Pegasus, anhand eines Bioinformatik-Workflows. Die Systeme werden dabei anhand von zehn Effizienzmetriken wie Laufzeit, CPU- und Speicherbedarf bewertet [16, S. 2773 f.].

Die vorliegende Arbeit unterscheidet sich von diesen Arbeiten durch die Auswahl der Systeme und den experimentellen Aufbau. Mit StreamFlow und Merlin werden zwei Systeme betrachtet, die in den genannten Vergleichen nicht vertreten sind. Für alle fünf Systeme werden dieselben Workflow-Muster mit identischer Nutzlast sowohl lokal als auch auf einem HPC-System mit Slurm betrachtet.

Methodik

Dieses Kapitel beschreibt das Vorgehen der Untersuchung. Es legt das Evaluationsdesign fest, begründet die Auswahl der fünf Workflow-Management-Systeme, führt die qualitativen Vergleichsaspekte und die quantitativen Messgrößen ein und beschreibt die Benchmark-Workflows, den Versuchsplan sowie das Mess- und Auswertungsverfahren.

3.1 Evaluationsdesign

Die Untersuchung vergleicht fünf Workflow-Management-Systeme entlang zweier Workflow-Muster und verbindet dabei zwei Blickrichtungen. Qualitativ wird zunächst betrachtet, wie sich die Muster in den Systemen beschreiben lassen und wie sich die Umsetzungen entlang fester Vergleichsaspekte unterscheiden. Anschließend wird quantitativ gemessen, wie lange die Systeme für die Ausführung der Muster benötigen, welchen Zeitanteil die Systeme der reinen Rechenlogik hinzufügen, wie sie die Übergaben zwischen den Schritten abwickeln und wie die parallelen Instanzen gestartet werden.

Die Untersuchung ist zweistufig aufgebaut. Auf der lokalen Ebene werden beide Muster in allen fünf Systemen auf einem einzelnen Rechner umgesetzt und vermessen. Diese Ebene bildet die Vergleichsbasis, da dort alle fünf Systeme vertreten sind und unter denselben Bedingungen ausgeführt werden. Auf der zweiten Ebene wird untersucht, wie sich die Workflow-Management-Systeme auf einem Slurm-basierten HPC-System bereitstellen und betreiben lassen, ergänzt um Messungen auf einer dediziert zugewiesenen Ressource dieses Systems. Die lokale Ebene behandelt Kapitel 4, die HPC-Ebene Kapitel 5.

In allen Systemen kommen dieselben Python-Skripte als Rechenschritte zum Einsatz. Unterschiede in den gemessenen Laufzeiten gehen damit nicht auf unterschiedliche fachliche Rechenoperationen zurück.

3.2 Auswahl der Systeme

Untersucht werden Nextflow, StreamFlow, Merlin, Pegasus und AiiDA. Alle fünf sind für wissenschaftliche Workflows entwickelt, unterscheiden sich aber in ihrem Entstehungskontext. Nextflow stammt aus dem Umfeld der Bioinformatik und zielt auf die reproduzierbare Ausführung von Analysen großer biologischer Datensätze über unterschiedliche Rechenumgebungen hinweg [11, S. 316]. StreamFlow zielt darauf, denselben Workflow in verschiedenen, auch hybriden Ausführungsumgebungen auszuführen [8, S. 1723]. Merlin ist für große Ensembles wissenschaftlicher Simulationen ausgelegt, deren Ergebnisse für maschinelles Lernen aufbereitet werden [28, S. 255]. Pegasus richtet sich auf großskalige, mehrstufige Simulations- und Analyse-Abläufe, deren Rechen- und Datenumfang ein System erfordert, das Datentransfer und Aufgabenausführung auf verteilten Rechenressourcen zuverlässig koordiniert und automatisiert [10, S. 17]. AiiDA verbindet die Ausführung mit einer durchgängigen Aufzeichnung der Datenherkunft in einer Datenbank [15, S. 1]. Eine Kurzvorstellung der Systeme gibt Kapitel 2.

Die Auswahl zielte darauf, unterschiedliche Ansätze wissenschaftlicher Workflow-Management-Systeme abzudecken, Grundlage waren die Veröffentlichungen und Dokumentationen der Systeme. Berücksichtigt wurden drei Merkmale. Das erste ist der Anwendungskontext, in dem die Systeme entstanden sind. Dieser wurde oben je System benannt und umfasst unter anderem Bioinformatik, große Simulationsensembles und einen besonderen Schwerpunkt auf Provenance. Auch Suter et al. unterscheiden Systeme danach, ob sie einer Fachgemeinschaft entstammen oder anwendungsneutral ausgelegt sind [39, S. 4]. Das zweite Merkmal ist die Art der Workflow-Beschreibung, die Systeme unterscheiden sich darin, in welcher Form ein Workflow niedergeschrieben wird und wie darin die Abhängigkeiten zwischen den Schritten und die parallele Verarbeitung ausgedrückt werden. Das dritte Merkmal ist die Architektur der Ausführung, einige Systeme führen Workflows in einer eigenen Laufzeit aus, andere übergeben die Ausführung an eigenständige Komponenten. Wie sich diese Merkmale in den einzelnen Systemen zeigen, zeigt die Modellierung in Kapitel 4.

Hinzu kamen praktische Erwägungen. Nextflow steht auf dem betrachteten HPC-System bereits als Modul zur Verfügung, StreamFlow wurde von der betreuenden Arbeitsgruppe vorgeschlagen. Für alle Systeme wurde vorausgesetzt, dass sie frei verfügbar sind, aktiv weiterentwickelt werden [41], sich

auf einem einzelnen Rechner installieren lassen und eine öffentliche Dokumentation besitzen, da beide Muster zunächst lokal umgesetzt und vermessen werden.

Das Feld wissenschaftlicher Workflow-Management-Systeme umfasst mehrere hundert Vertreter [39, @41]. Eine vollständige Erhebung ist im Rahmen dieser Arbeit nicht möglich, da jedes System einzeln bereitgestellt, in beiden Mustern umgesetzt und vermessen wird. Die Auswahl ist deshalb eine begründete Stichprobe unterschiedlicher Ansätze.

3.3 Qualitative Vergleichsaspekte

Der qualitative Vergleich betrachtet die Umsetzungen entlang von sieben Aspekten: dem Ausdruck des Musters, dem Modellierungs- und Konfigurationsaufwand, der Sichtbarkeit von Datenfluss und Abhängigkeiten, der Ausführung und Steuerung, der Trennung von Workflow-Logik und Ausführungsumgebung, der Nachvollziehbarkeit der Ausführung und der Verständlichkeit aus Einsteigerperspektive. Zusammen decken sie den Weg eines Workflows von der Beschreibung über die Ausführung bis zur Nachvollziehbarkeit ab. Die folgenden Absätze begründen die Wahl der einzelnen Aspekte.

Ausdruck des Musters: Dieser Aspekt betrachtet, mit welchen Mitteln eines Systems die Struktur der beiden Workflow-Muster beschrieben wird, insbesondere die Abfolge der Schritte und die parallele Verarbeitung der Teilstücke. Er bildet die Grundlage des Vergleichs, da die Workflow-Muster in allen fünf Systemen dieselben sind, ihre Umsetzung jedoch unterschiedlich erfolgt. Untersucht wird der Aspekt anhand der Workflow-Beschreibungen beider Muster in allen fünf Systemen.

Modellierungs- und Konfigurationsaufwand: Dieser Aspekt betrachtet, welcher Beschreibungs- und Konfigurationsaufwand nötig ist, um ein Workflow-Muster in einem System umzusetzen und anzupassen. Da sich der Aufwand nicht unmittelbar messen lässt, wird er über zwei nachvollziehbare Größen erfasst: die Zahl der manuell erstellten Artefakte zur Beschreibung und Konfiguration des Workflows sowie die Frage, welche Stellen bei einer Änderung der Rechenlast oder der Teilstückzahl angepasst werden müssen. Beide Größen sind unmittelbar an den Workflow-Beschreibungen und Konfigurationsdateien der Umsetzungen ablesbar.

Sichtbarkeit von Datenfluss und Abhängigkeiten: Dieser Aspekt betrachtet, ob sich aus der Workflow-Beschreibung ablesen lässt, welche Dateien zwischen den Schritten weitergegeben werden und wodurch die Abhängigkeiten zwischen ihnen entstehen. Je unmittelbarer diese Informationen aus der Beschreibung hervorgehen, desto leichter lässt sich der Ablauf eines Workflows nachvollziehen. Der Aspekt wird anhand der Workflow-Beschreibungen beider Muster in allen fünf Systemen untersucht.

Ausführung und Steuerung: Dieser Aspekt betrachtet, welche Komponente einen beschriebenen Workflow ausführt und wodurch der Beginn eines einzelnen Schritts ausgelöst wird. Damit bildet er die Verbindung zwischen der Workflow-Beschreibung und dem beobachtbaren Ablauf eines Workflows, da sich daraus ergibt, wann und auf welchem Weg einzelne Schritte ausgeführt werden. Der Aspekt wird anhand der Umsetzung und Ausführung der beiden Benchmark-Workflows betrachtet.

Trennung von Workflow-Logik und Ausführungsumgebung: Dieser Aspekt betrachtet, ob die fachliche Beschreibung eines Workflows unabhängig von der Umgebung formuliert ist, in der er ausgeführt wird, und über welchen Mechanismus beides verbunden wird. Eine Trennung von Workflow-Logik und Ausführungsumgebung erleichtert es, denselben Workflow in unterschiedlichen Ausführungsumgebungen einzusetzen und ist für diese Arbeit besonders relevant, da beide Workflow-Muster zunächst lokal und anschließend auf einem HPC-System betrachtet werden. Mehrere der untersuchten Systeme nennen diese Trennung selbst als Entwurfsziel [8, S. 1723][10, S. 18–20]. Untersucht wird der Aspekt anhand der Mechanismen, mit denen die Systeme Workflow-Beschreibung und Ausführungsumgebung voneinander trennen, sowie daran, welche Teile der Umsetzung bei einem Wechsel der Ausführungsumgebung angepasst werden müssten.

Nachvollziehbarkeit der Ausführung: Dieser Aspekt betrachtet, welche Möglichkeiten ein System bietet, den Fortschritt eines Laufs zu verfolgen, und welche Artefakte nach dem Lauf zur Verfügung stehen. Die Aufzeichnung des Ausführungsverlaufs erleichtert es, Ergebnisse nachzuvollziehen und Fehlerquellen im Ablauf einzugrenzen und ist insbesondere in komplexen Workflows und HPC-Umgebungen von Bedeutung [9, 30]. Untersucht wird der Aspekt anhand der Ausgaben während der Ausführung, der verbleibenden Verzeichnisse und Protokolle sowie weiterer Artefakte und Darstellungen, die sich aus einem Lauf erzeugen lassen.

Verständlichkeit aus Einsteigerperspektive: Dieser Aspekt betrachtet, welche systemeigenen Konzepte verstanden sein müssen, bevor sich ein Workflow in einem System umsetzen und ausführen lässt. Die Einstiegshürde ist für die Wahl eines Systems von praktischer Bedeutung. Untersucht wird der Aspekt anhand der Konzepte, die für die erstmalige Umsetzung der beiden Workflow-Muster in den einzelnen Systemen verstanden werden mussten. Die Einordnung beruht auf dieser erstmaligen Umsetzung und ersetzt keine Nutzerstudie.

3.4 Quantitative Messgrößen

Der quantitative Vergleich stützt sich auf vier Messgrößen: die Gesamtlaufzeit (Makespan), den Overhead gegenüber einem Referenzlauf sowie die beiden Koordinationsgrößen Übergangslatenz und Startspreizung. Die folgenden Absätze definieren die vier Größen.

Gesamtlaufzeit: Als Gesamtlaufzeit wird die Zeitspanne vom Aufruf des Workflow-Management-Systems bis zu dessen Beendigung bezeichnet. Sie umfasst damit alle Vorgänge eines Laufs, die Ausführung der Rechenschritte ebenso wie die Vorbereitung und die Übergaben zwischen den Schritten, und entspricht der Zeit, die Nutzende bei der Ausführung eines Workflows beobachten.

Overhead: Der Overhead bezeichnet die zusätzliche Zeit, die durch den Einsatz eines Workflow-Management-Systems gegenüber der Ausführung der reinen Rechenlogik entsteht. Er wird als Differenz zwischen der Gesamtlaufzeit eines Laufs und der Laufzeit eines Referenzlaufs bestimmt, der dieselben Rechenschritte in derselben Reihenfolge ohne Workflow-Management-System ausführt. Berichtet wird der Median der je Wiederholung gebildeten Differenzen zwischen System- und Referenzlauf.

Übergangslatenz: Die Übergangslatenz bezeichnet die Zeitspanne zwischen dem Ende eines Rechenschritts und dem Beginn des nachfolgenden. Sie erfasst damit den Zeitbedarf, der beim Übergang zwischen zwei Schritten entsteht, und wird für jeden Übergang eines Musters einzeln bestimmt.

Startspreizung: Die Startspreizung bezeichnet den Abstand zwischen dem Beginn der ersten und dem Beginn der letzten parallelen Verarbeitungsinstanz eines Scatter-Gather-Laufs. Sie erfasst, wie eng beieinander ein System die

Instanzen der parallelen Phase startet, und ist bei einem einzelnen Teilstück nicht definiert.

Die vier Größen erfassen den zeitlichen Einfluss eines Workflow-Management-Systems auf zwei Ebenen. Gesamtlaufzeit und Overhead beschreiben den Zeitbedarf eines vollständigen Laufs, Übergangslatenz und Startspreizung betrachten zwei Koordinationsvorgänge innerhalb eines Laufs gesondert. Makespan und Overhead knüpfen an bestehende Arbeiten zur Analyse des Laufzeitverhaltens wissenschaftlicher Workflows an. Chen und Deelman analysieren den Zeitbedarf eines Workflow-Laufs anhand des Makespan und verschiedener Overhead-Anteile des Workflow-Management-Systems und der Ausführungsumgebung [6]. Die beiden Koordinationsgrößen werden für diese Arbeit eingeführt. Sie bestimmen den zeitlichen Abstand zwischen zwei aufeinanderfolgenden Rechenschritten beziehungsweise die Spreizung der Startzeitpunkte paralleler Instanzen. Beide Größen lassen sich in allen fünf Systemen auf dieselbe Weise erheben, ohne auf systemeigene Protokolle zurückzugreifen.

3.5 Benchmark-Workflows und Versuchsplan

Für die Untersuchung wurden zwei Benchmark-Workflows entworfen, die die beiden in Kapitel 2 eingeführten Workflow-Muster umsetzen. Der Pipeline-Workflow bildet eine vollständig sequenzielle Verarbeitungskette ab, der Scatter-Gather-Workflow erweitert diese Kette um eine parallele Verarbeitungsphase mit anschließender Zusammenführung. Die Wahl fiel auf diese beiden Muster, da sie in der Literatur als grundlegende Strukturen wissenschaftlicher Workflows beschrieben werden. Suter et al. nennen Datenverarbeitungs-Pipelines und Fan-out/Fan-in-Ausführungsmuster als einfache Formen wissenschaftlicher Workflows [39, S. 4].

Der Pipeline-Workflow besteht aus vier Schritten. `generate_input` erzeugt einen reproduzierbaren Eingabedatensatz. `preprocess` bereitet die Daten für die Verarbeitung vor, indem Werte unterhalb einer Schwelle entfernt werden. `compute` führt als einziger Schritt die rechenintensive Verarbeitung aus. `postprocess` fasst das Ergebnis zu einer abschließenden Ausgabedatei zusammen.

Der Scatter-Gather-Workflow übernimmt die Vor- und Nachbereitungsschritte des Pipeline-Workflows und erweitert die Verarbeitung um eine parallele

Phase. Nach der Vorverarbeitung zerlegt `split` den Datensatz in eine einstellbare Zahl von Teilstücken. Für jedes Teilstück wird eine eigene Instanz von `compute` ausgeführt. `aggregate` führt die Teilergebnisse zu einem gemeinsamen Ergebnis zusammen, bevor `postprocess` den Workflow abschließt.

Abbildung 3.1 zeigt den Aufbau beider Workflows.

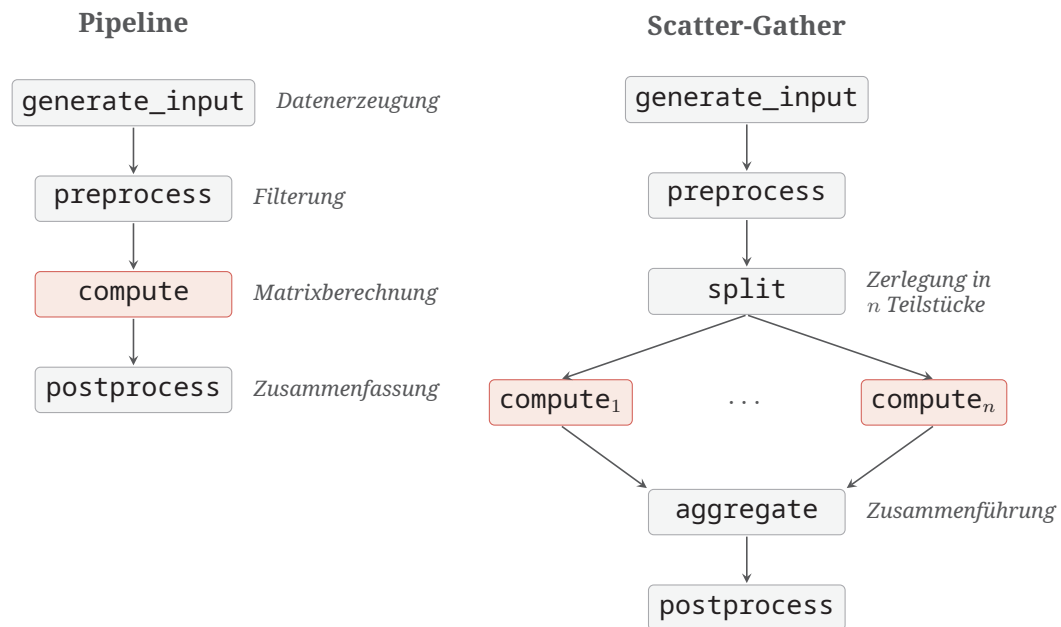


Abb. 3.1.: Aufbau der beiden Benchmark-Workflows. Hervorgehoben ist der `compute`-Schritt als einziger Träger der Rechenlast, beim Scatter-Gather-Muster in n parallelen Instanzen.

Die Struktur der beiden Workflows orientiert sich an Grundstrukturen wissenschaftlicher Workflows, die Bharathi et al. beschreiben, darunter sequenzielle Verarbeitungsketten, die Zerlegung von Datensätzen in Teilstücke und deren anschließende Zusammenführung [4, S. 3]. Die konkrete Ausgestaltung der einzelnen Schritte stellt eine vereinfachte Benchmark-Implementierung dieser Grundstrukturen dar.

Alle Schritte sind Python-Skripte, die ihre Ein- und Ausgabepfade als Argumente entgegennehmen und ihre Ergebnisse als Dateien ablegen. `generate_input` erzeugt einhundert ganzzahlige Werte mit einem festen Startwert des Zufallszahlengenerators, die Eingabe ist dadurch in jedem Lauf identisch. Die eigentliche Rechenlast liegt ausschließlich im `compute`-Schritt: Für jeden Eingabewert wird das Produkt zweier Matrizen der Größe 96×96 berechnet, die deterministisch aus dem Wert abgeleitet werden, und zu einer

Prüfsumme zusammengefasst. Die übrigen Schritte beschränken sich auf das Erzeugen, Filtern, Zerlegen und Zusammenführen der Textdateien und tragen keine nennenswerte Rechenlast.

Für die Erzeugung von Workflow-Benchmarks existieren auch Werkzeuge wie WfBench, das realitätsnahe Rechenlasten und Workflow-Strukturen erzeugt [7, S. 101]. Für diese Arbeit wurden stattdessen bewusst einfache Benchmark-Workflows verwendet, um die Rechenlogik und die Versuchsbedingungen über alle fünf Systeme hinweg möglichst kontrolliert und einheitlich zu halten.

Die Untersuchung verwendet drei Rechenlasten, die sich ausschließlich in der Zahl der Wiederholungen der Matrixberechnung je Eingabewert unterscheiden: einmal bei der kurzen, dreizehnmal bei der mittleren und sechzigmal bei der langen Rechenlast. Die Ausführung der reinen Rechenlogik benötigt auf der lokalen Ebene etwa fünf Sekunden, eine Minute beziehungsweise viereinhalb Minuten. Die drei Rechenlasten ermöglichen damit die Untersuchung sowohl kurzer Workflows, bei denen der Koordinationsaufwand der Systeme einen vergleichsweise hohen Anteil an der Gesamtlaufzeit hat, als auch längerer Workflows, bei denen die Rechenzeit dominiert. Beim Scatter-Gather-Workflow wird die Zahl der Teilstücke zwischen eins, zwei und vier variiert, die parallele Phase umfasst damit bis zu vier gleichzeitig laufende Verarbeitungsinstanzen.

Aus den Mustern, Rechenlasten und Teilstückzahlen ergibt sich der Versuchsplan: zwölf Konfigurationen je System, drei für das Pipeline-Muster und neun für das Scatter-Gather-Muster. Jede Konfiguration wurde bei der kurzen und der mittleren Rechenlast fünfmal wiederholt, bei der langen Rechenlast aufgrund des deutlich höheren Zeitbedarfs je Lauf dreimal. Daraus ergeben sich insgesamt 52 Läufe je System und 52 Läufe ohne System. Damit stellt die Zahl der Wiederholungen einen Kompromiss zwischen Messaufwand und praktischer Durchführbarkeit dar.

3.6 Messverfahren

Die Messung erfolgt auf zwei Ebenen. Eine äußere Messung um den Lauf liefert die Gesamtlaufzeit und, im Vergleich mit einem Referenzlauf, den Overhead. Eine innere Instrumentierung in den Benchmark-Skripten liefert die Zeitpunkte, aus denen Übergangslatenz und Startspreizung berechnet

werden. Abbildung 3.2 gibt einen Überblick über das Mess- und Auswertungsverfahren einer Konfiguration, die folgenden Abschnitte beschreiben die einzelnen Bestandteile.

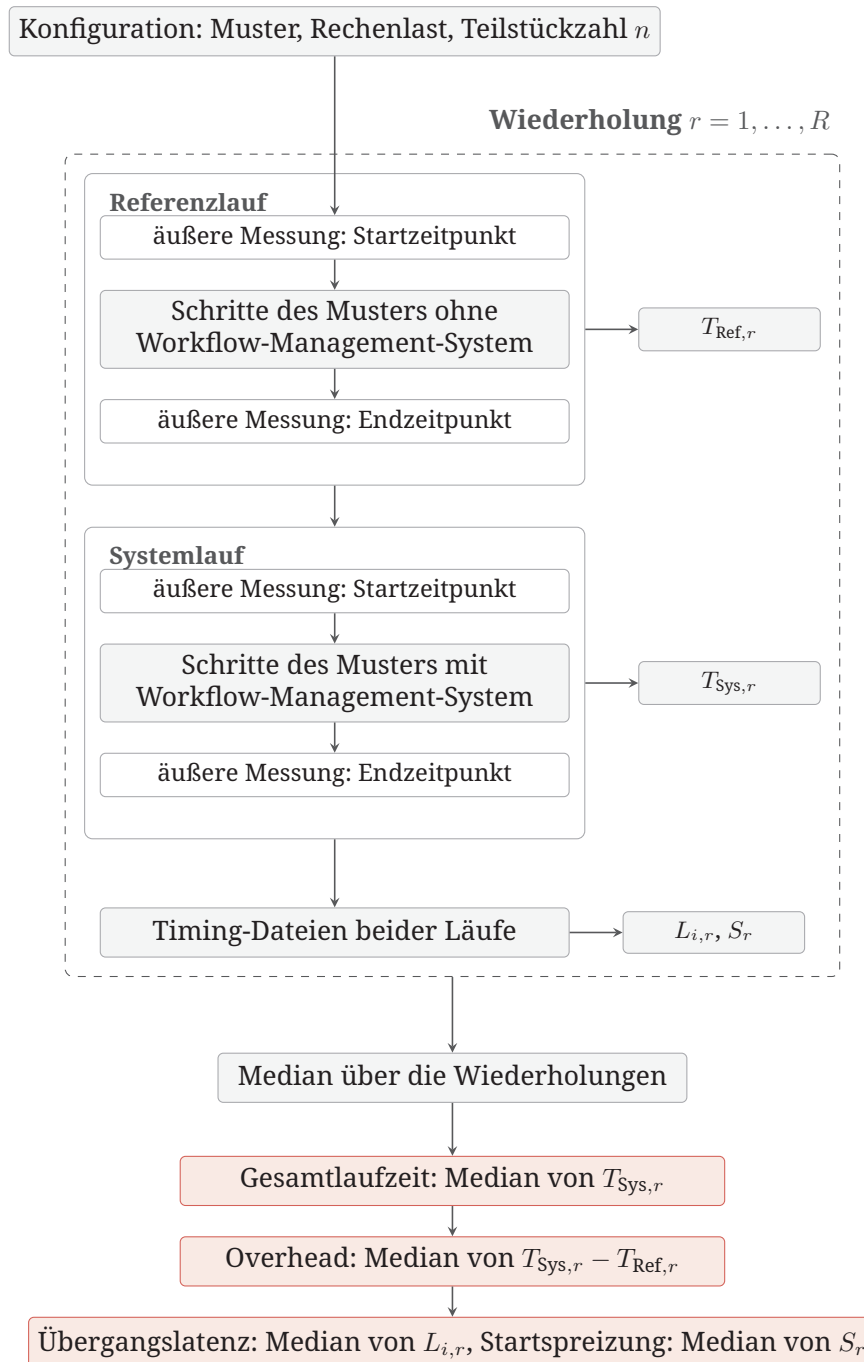


Abb. 3.2.: Ablauf des Mess- und Auswertungsverfahrens für eine Konfiguration. Jede Wiederholung umfasst einen Referenzlauf und den zugehörigen Systemlauf, beide werden nach demselben Verfahren von außen gemessen und erzeugen Timing-Dateien. Aus den Wiederholungen werden anschließend die berichteten Kennwerte gebildet.

3.6.1 Referenzmessung

Der Overhead eines Systems lässt sich nicht unmittelbar messen, sondern nur als Differenz zu einer Ausführung derselben Rechenlogik ohne Workflow-Management-System. Zu jeder Wiederholung einer Konfiguration wurde deshalb neben dem Systemlauf ein Referenzlauf erhoben, der als Bezugsgröße dient.

Der Referenzlauf verwendet dieselben Benchmark-Skripte und dieselben fachlichen Ein- und Ausgaben wie der zugehörige Systemlauf und führt sie in der Reihenfolge des jeweiligen Musters aus. Referenz- und Systemläufe werden für jedes System durch ein Steuerskript gestartet und gemessen. Für den Referenzlauf startet dieses Steuerskript die Benchmark-Skripte unmittelbar als eigene Prozesse. Beim Pipeline-Muster laufen die vier Schritte nacheinander ab. Beim Scatter-Gather-Muster folgen auf die Vorverarbeitung und die Zerlegung so viele gleichzeitig gestartete Verarbeitungsprozesse, wie Teilstücke vorgesehen sind, anschließend das Zusammenführen und die Nachbereitung. Da die Eingabe mit einem festen Seed erzeugt wird, verarbeiten Referenz- und Systemlauf dieselben Werte. Der Referenzlauf bildet damit dieselbe Rechenlogik sowie dieselbe Abfolge und Parallelität der Rechenschritte ab wie der Systemlauf, jedoch ohne die Steuerung durch ein Workflow-Management-System.

Innerhalb einer Wiederholung wurde stets zuerst der Referenzlauf und anschließend der zugehörige Systemlauf ausgeführt. Beide liefen in eigenen Arbeitsverzeichnissen und wurden nach demselben Verfahren von außen gemessen. Durch die unmittelbare zeitliche Paarung bleiben Unterschiede der Ausführungsbedingungen gering, der wesentliche Unterschied zwischen beiden Läufen liegt in der Steuerung durch das Workflow-Management-System.

3.6.2 Äußere Messung

Die äußere Messung erfasst die Gesamtlaufzeit eines Laufs. Das Steuerskript nimmt unmittelbar vor dem Start eines Laufs und nach dessen vollständigem Abschluss jeweils einen Zeitstempel. Die Zeitspanne dazwischen bildet die Gesamtlaufzeit T_{Sys} eines Systemlaufs beziehungsweise T_{Ref} des zugehörigen Referenzlaufs.

Der Beginn der Messung liegt bei Nextflow, StreamFlow, Merlin und AiiDA unmittelbar vor dem Startbefehl. Bei Pegasus beginnt die Messung bereits vor der Planung, sodass auch die Planungszeit in der Gesamtlaufzeit enthalten ist. Wie das Ende eines Laufs festgestellt wird, hängt vom Ausführungsmodell ab. Bei Nextflow und StreamFlow kehrt der Aufruf erst zurück, wenn der Workflow abgeschlossen ist, das Ende der Messung fällt mit der Rückkehr des Aufrufs zusammen. Merlin, AiiDA und Pegasus übergeben die Ausführung an eine eigenständige Komponente, der Startbefehl kehrt vorher zurück. Das Steuerskript fragt in diesen Fällen den Zustand des Laufs zyklisch ab und beendet die Messung, sobald alle Schritte abgeschlossen sind. Tabelle 3.1 fasst je System zusammen, womit ein Lauf gestartet und woran sein Ende erkannt wird.

Tab. 3.1.: Start der Läufe und Erkennung des Laufendes in der äußeren Messung

System	Start der Messung	Erkennung des Laufendes
Nextflow	vor <code>nextflow run</code>	Rückkehr des blockierenden Aufrufs
StreamFlow	vor <code>streamflow run</code>	Rückkehr des blockierenden Aufrufs
Merlin	vor <code>merlin run</code>	MERLIN_STATUS.json mit Status FINISHED und MERLIN_FINISHED im Workspace des terminalen Schritts <code>postprocess</code> , Abfrage alle 20 ms
AiiDA	vor <code>submit()</code> der WorkChain	terminaler Zustand der obersten WorkChain, Abfrage alle 50 ms
Pegasus	vor <code>pegasus-plan</code>	erfolgreicher Workflow-Zustand über <code>pegasus-status -l</code> , Abfrage alle 500 ms

Bei Pegasus umfasst die Messung damit den vorgelagerten Planungsschritt, da dieser Teil der Ausführung eines Workflows ist. Dauerhaft laufende Komponenten wie Worker, Daemon oder HTCondor sind beim Start der Messung bereits aktiv, ihr Start zählt nicht zur Gesamtlaufzeit. Die äußere Messung umfasst damit in allen Systemen denselben Abschnitt: Sie beginnt mit der Übergabe des vorbereiteten Workflows an das jeweilige System und endet, sobald dieses den vollständigen Workflow als abgeschlossen meldet.

3.6.3 Innere Instrumentierung

Die beiden Koordinationsgrößen erfordern Zeitpunkte einzelner Schritte innerhalb eines Laufs. Diese Zeitpunkte werden nicht den Protokollen der Systeme entnommen, sondern in den Benchmark-Skripten selbst erhoben. Da alle fünf Systeme dieselben Skripte ausführen, entsteht dadurch in jedem

System dieselbe Datenquelle, unabhängig davon, welche Aufzeichnungen ein System von sich aus führt und in welcher Form und Auflösung es sie ablegt.

Jeder Schritt nimmt zu Beginn seiner Ausführung und nach Abschluss der eigentlichen Verarbeitung jeweils einen Zeitstempel und legt anschließend eine Timing-Datei in seinem Arbeitsverzeichnis ab, die den Namen des Schritts sowie beide Zeitpunkte enthält. Aus den Zeitpunkten aufeinanderfolgender Schritte ergibt sich die Übergangslatenz, aus den Startzeitpunkten der parallelen Instanzen eines Scatter-Gather-Laufs die Startspreizung. Die Instanzen tragen dabei die Nummer ihres Teilstücks im Namen und lassen sich dadurch einzeln auswerten. Listing 3.1 zeigt den Aufbau eines instrumentierten Schritts.

```
1 task_start_ns = time.time_ns()
2
3 # fachliche Verarbeitung des Schritts
4
5 task_end_ns = time.time_ns()
6 write_timing("preprocess", task_start_ns, task_end_ns)
7
8
9 def write_timing(task_name, start_ns, end_ns):
10     Path(f"timing_{task_name}.txt").write_text(
11         f"task_name={task_name}\n"
12         f"task_start_ns={start_ns}\n"
13         f"task_end_ns={end_ns}\n"
14     )
```

P. 3.1: Aufbau eines instrumentierten Benchmark-Schritts

Als Zeitquelle dient die Systemuhr in Nanosekunden (`time.time_ns()`). Da die Übergangslatenzen aus Zeitpunkten verschiedener Prozesse berechnet werden, müssen alle Schritte dieselbe Zeitbasis verwenden. Die gewählte Zeitquelle ermöglicht dadurch den direkten Vergleich der Zeitstempel aller Benchmark-Schritte.

Da Referenzlauf und Systemlauf dieselben Benchmark-Skripte verwenden, entstehen in beiden Fällen Timing-Dateien mit identischem Aufbau. Die Instrumentierung ist dadurch unabhängig vom verwendeten Workflow-Management-System und liefert für alle fünf Systeme dieselbe Grundlage zur Berechnung der Koordinationsgrößen.

3.6.4 Auswertung

Nach Abschluss aller Wiederholungen einer Konfiguration werden die äußeren Laufzeiten und die Timing-Dateien zusammengeführt. Aus den äußeren Laufzeiten ergeben sich Gesamtlaufzeit und Overhead, aus den Timing-Dateien die beiden Koordinationsgrößen. Anschließend werden die Messwerte über die Wiederholungen zusammengefasst.

Sei R die Zahl der Wiederholungen einer Konfiguration und $r \in \{1, \dots, R\}$ der Index einer Wiederholung, $T_{\text{Sys},r}$ und $T_{\text{Ref},r}$ die in Wiederholung r von außen gemessenen Laufzeiten des Systemlaufs und des zugehörigen Referenzlaufs. Die berichtete Gesamtlaufzeit T und der Overhead O sind dann

$$T = \text{Median}_r(T_{\text{Sys},r}), \quad O = \text{Median}_r(T_{\text{Sys},r} - T_{\text{Ref},r}).$$

Der Overhead wird damit entsprechend seiner Definition in Abschnitt 3.4 als Median der je Wiederholung gebildeten Differenzen bestimmt.

Die Koordinationsgrößen ergeben sich aus den Timing-Dateien. Für einen Übergang i vom Schritt a zum Schritt b bezeichnet $L_{i,r}$ den Abstand zwischen dem Ende von a und dem Beginn von b , für die parallele Phase eines Scatter-Gather-Laufs mit n Teilstücken bezeichnet S_r den Abstand zwischen dem frühesten und dem spätesten Start der Instanzen:

$$L_{i,r} = t_{b,r}^{\text{start}} - t_{a,r}^{\text{ende}}, \quad S_r = \max_{1 \leq k \leq n} t_{k,r}^{\text{start}} - \min_{1 \leq k \leq n} t_{k,r}^{\text{start}}.$$

Berichtet werden auch hier die Mediane über die Wiederholungen, $L_i = \text{Median}_r(L_{i,r})$ und $S = \text{Median}_r(S_r)$.

Der Median wurde dem arithmetischen Mittel vorgezogen, da bei einer kleinen Zahl von Wiederholungen vereinzelte ungewöhnlich lange Laufzeiten das Ergebnis sonst bestimmen würden.

Die Steuerskripte legen die Messwerte der einzelnen Wiederholungen als Rohdaten ab und erzeugen aus den Timing-Dateien die Koordinationswerte. Für die endgültige Auswertung werden die äußeren Laufzeitmessungen von einem separaten Auswertungsskript eingelesen und nach dem oben beschriebenen Verfahren zusammengefasst. Die bereits berechneten Koordinationswerte werden unverändert übernommen.

Auf der HPC-Ebene bleiben Instrumentierung und Auswertung unverändert. Die Ausführungsbedingungen unterscheiden sich dort jedoch, weshalb die

Ergebnisse beider Ebenen getrennt betrachtet werden. Kapitel 5 geht darauf ein.

3.6.5 Grenzen des Messverfahrens

Das Messverfahren ist darauf ausgerichtet, die Workflow-Management-Systeme unter vergleichbaren Bedingungen und mit derselben Rechenlogik zu vermessen. Einige Eigenschaften des Versuchsaufbaus sind bei der Einordnung der Ergebnisse zu berücksichtigen.

Die Zahl der Wiederholungen beträgt für die kurze und die mittlere Rechenlast fünf, für die lange Rechenlast aufgrund des deutlich höheren Zeitbedarfs drei. Die berichteten Mediane beruhen damit je nach Rechenlast auf unterschiedlich vielen Beobachtungen.

Referenzlauf und Systemlauf werden innerhalb einer Wiederholung unmittelbar nacheinander ausgeführt und bilden dadurch möglichst vergleichbare Bedingungen. Dennoch unterliegen beide Läufe den üblichen Schwankungen der Ausführungsumgebung, die sich nicht vollständig vermeiden lassen.

Das Messverfahren erfasst die von außen beobachtbaren Laufzeiten eines Systems. Die Messgrößen beschreiben damit den zeitlichen Effekt der Ausführung, erlauben jedoch keine unmittelbare Zuordnung der gemessenen Zeiten zu einzelnen internen Mechanismen oder Komponenten eines Workflow-Management-Systems. Die Auswirkungen dieser Einschränkungen auf die Interpretation der Ergebnisse werden in Abschnitt 6.2 diskutiert.

Modellierung und lokale Evaluation

Dieses Kapitel beschreibt die Umsetzung der beiden Workflow-Muster in den fünf Systemen und die Ergebnisse der lokalen Messreihe. Zunächst wird die Untersuchungsumgebung dargestellt, anschließend werden die Umsetzungen beider Muster entlang der Vergleichsaspekte gegenübergestellt. Den Abschluss bilden die gemessenen Laufzeiten.

4.1 Versuchsumgebung und Systembereitstellung

Alle Modellierungen und die lokale Messreihe wurden auf einem MacBook Pro (2021) mit Apple-M1-Pro-Prozessor (arm64) und 16 GB Arbeitsspeicher unter macOS 12.0.1 ausgeführt. Die Angaben zur Hardware werden genannt, weil die absoluten Laufzeiten an die verwendete Umgebung gebunden sind. Für die Systeme wurden getrennte Umgebungen angelegt (conda oder venv), um wechselseitige Abhängigkeitskonflikte auszuschließen. Eine HPC-Ausführung wird damit nicht abgebildet, die lokale Ebene dient der Modellierung und der Messung unter kontrollierten Bedingungen. Wie sich die Systeme auf einem Slurm-basierten HPC-System betreiben lassen, untersucht Kapitel 5, auf die Grenzen der lokalen Betrachtung geht Abschnitt 6.2 ein.

Die folgenden Absätze beschreiben die Bereitstellung je System in derselben Reihenfolge: eingesetzte Version und Installationsweg, zusätzlich benötigte Komponenten und die dafür nötigen Konfigurationsschritte sowie aufgetretene Fehler.

Nextflow: Nextflow wurde in Version 23.10.4 über Homebrew installiert. Als Voraussetzung benötigt Nextflow eine Java-Laufzeit ab Version 17 [24], die die verwendete Homebrew-Formel als Abhängigkeit mitinstalliert [14]. Weitere Komponenten und Konfigurationsschritte waren für den lokalen Betrieb nicht erforderlich. Bei der Einrichtung traten keine Fehler auf.

StreamFlow: StreamFlow wurde in der Vorabversion 0.2.0rc2 eingesetzt. Die Installation erfolgte mit pip direkt aus dem Quellcode-Repository des Projekts [38] und wurde auf den Commit 7704a8e festgelegt. StreamFlow wird als Python-Paket installiert und setzt Python ab Version 3.10 voraus [36]. Zusätzliche Komponenten waren nicht erforderlich. Konfigurationsschritte über die Installation hinaus und Fehler traten nicht auf.

Merlin: Merlin wurde in Version 1.13.0 mit pip installiert. Für die Ausführung benötigt Merlin einen Nachrichtenvermittler, in dessen Warteschlange Celery die Schritte als Aufgaben einstellt, die Worker entgegennehmen und abarbeiten [17]. In der lokalen Einrichtung diente Redis als Nachrichtenvermittler und wurde über einen Docker-Container bereitgestellt. Die Verbindungsdaten zum Nachrichtenvermittler stehen in der Datei `app.yaml` im Nutzerverzeichnis. Redis-Container und Celery-Worker müssen bei der Ausführung aktiv sein, bleiben aber über mehrere Läufe hinweg bestehen und wurden daher einmal je Sitzung gestartet. Bei der Einrichtung trat ein Fehler auf: Aktuelle Versionen von `setuptools` liefern das von Merlin vorausgesetzte Modul `pkg_resources` nicht mehr mit, sodass das Werkzeug nach der Installation zunächst nicht startete. Erst nach einem Downgrade von `setuptools` auf eine Version kleiner als 81 lief Merlin an.

Pegasus: Pegasus wurde in Version 5.1.2 installiert und benötigt HTCondor [26], das in Version 23.1.0 über Homebrew bezogen wurde. Die HTCondor-Prozesse wurden einmal über `condor_master` gestartet und blieben in der verwendeten Einrichtung dauerhaft aktiv. Vor jedem Aufruf waren zusätzlich zwei Umgebungsvariablen zu setzen, die nur für die aktuelle Terminal-Sitzung gelten: `CONDOR_CONFIG` für die HTCondor-Konfiguration und `PYTHONPATH` für die Pegasus-Python-Bibliothek. Ohne die zweite Variable brach das Erzeugungsskript mit einem `ModuleNotFoundError` ab.

AiiDA: AiiDA wurde in Version 2.8.0 installiert. Für den in der Dokumentation empfohlenen Betrieb setzt AiiDA zwei Dienste voraus, PostgreSQL als Datenbank-Backend für die Ablage aller Daten- und Prozessknoten und RabbitMQ für die Kommunikation mit dem Daemon [1]. Beide Dienste wurden lokal installiert. Über die Installation hinaus waren eine Datenbank und ein Profil anzulegen, vor jedem Lauf muss zusätzlich der Daemon aktiv sein, der die Workflows im Hintergrund ausführt.

Tabelle 4.1 fasst die für die lokale Bereitstellung nötigen Komponenten und Schritte zusammen. Die Gegenüberstellung bleibt beschreibend, eine Aufwandsrangfolge wird nicht gebildet, weil sich die Schritte in Art und Umfang unterscheiden.

Tab. 4.1.: Lokale Bereitstellung je System: zusätzliche Komponenten, einmalige Konfiguration und vor der Ausführung erforderliche Schritte

System	Version	Zusätzliche Komponenten	Einmalige Konfiguration	Vor der Ausführung
Nextflow	25.10.4	keine	keine	keine
StreamFlow	0.2.0rc2	keine	keine	keine
Merlin	1.13.0	Redis	setuptools-Downgrade, app.yaml	Redis-Container und Worker starten
Pegasus	5.1.2	HTCondor 23.1.0	keine	HTCondor-Prozesse aktiv, CONDOR_CONFIG und PYTHONPATH gesetzt (je Sitzung)
AiiDA	2.8.0	PostgreSQL, RabbitMQ	Datenbank, Profil	Daemon starten

4.2 Pipeline-Muster

Das in Abschnitt 3.5 beschriebene Pipeline-Muster wurde in allen fünf Workflow-Management-Systemen mit denselben Benchmark-Skripten umgesetzt. Im Folgenden wird gegenübergestellt, wie die fünf Systeme dieses Muster modellieren und ausführen. Die vollständigen Workflow-Beschreibungen sind im Repository zu dieser Arbeit abgelegt [40].

4.2.1 Nextflow

Nextflow beschreibt den Workflow in einer einzigen Datei `benchmark.nf` in der systemeigenen Sprache DSL2.

Ausdruck des Musters: Eine separate Abhängigkeitsangabe gibt es nicht. Die Reihenfolge entsteht dadurch, dass die Ausgabe eines Prozesses als Eingabe des nachfolgenden Prozesses dient (Listing 4.1), entsprechend dem Datenfluss-Modell, auf dem Nextflow beruht [11, S. 317].

```

1 workflow {
2   raw_ch      = GENERATE_INPUT()
3   prepared_ch = PREPROCESS(raw_ch)
4   result_ch   = COMPUTE(prepared_ch)
5   summary_ch  = POSTPROCESS(result_ch)
6 }

```

P. 4.1: workflow-Block aus benchmark.nf (Nextflow, Pipeline)

Modellierungs- und Konfigurationsaufwand: Die Workflow-Beschreibung liegt vollständig in einer Datei, in der Prozessdefinitionen und Komposition zusammenstehen. Eine Änderung der Rechenlast betrifft allein den Aufruf des Berechnungsskripts, die Fassungen für die drei Rechenlasten unterscheiden sich in genau einer Zeile.

Sichtbarkeit von Datenfluss und Abhängigkeiten: Nicht nur die Reihenfolge, auch der Weg der Dateien ist in der Beschreibung ablesbar. Die beteiligten Dateien erscheinen in den path-Deklarationen der Ein- und Ausgaben und eine so deklarierte Ausgabe wird als Verweis im Arbeitsverzeichnis des nachfolgenden Prozesses bereitgestellt, sodass die Skripte sie unter dem angegebenen Namen öffnen. Die Abhängigkeiten entstehen durch die Übergabe dieser Ausgaben über die Channels an die nachfolgenden Prozesse.

Ausführung und Steuerung: Die Ausführung übernimmt die Nextflow-Laufzeit. Ein Prozess wird gestartet, sobald sein Eingabe-Channel Daten enthält, der Startzeitpunkt jedes Prozesses ergibt sich damit erst zur Laufzeit aus dem Eintreffen der Daten.

Trennung von Workflow-Logik und Ausführungsumgebung: Workflow-Beschreibung und Ausführungsparameter liegen getrennt vor. Angaben zur Ausführungsumgebung, die Nextflow über sogenannte Executor konfiguriert, sowie zu den benötigten Ressourcen stehen in einer eigenen Konfigurationsdatei, sodass ein Wechsel der Ausführungsumgebung die Workflow-Datei unberührt lässt.

Nachvollziehbarkeit der Ausführung: Während des Laufs zeigt die Ausgabe im Terminal für jeden Prozess die Kennung des Arbeitsverzeichnisses sowie die Zahl der abgeschlossenen Aufgaben an. Je Prozess bleibt ein Arbeitsverzeichnis mit den erzeugten Dateien zurück. Zusätzlich lassen sich eine Trace-Datei, ein Zeitstrahl und der Workflow-Graph erzeugen [23]. Der Workflow-Graph ist in Abbildung A.1 dargestellt, Zeitstrahlen von Pipeline-Läufen in den Abbildungen A.2 und A.3.

Verständlichkeit aus Einsteigerperspektive: Die Abfolge der vier Schritte ist im workflow-Block kompakt sichtbar. Für das Verständnis der Beschreibung ist darüber hinaus insbesondere das Channel-Konzept relevant, über das die Ausgaben eines Prozesses an den nachfolgenden Prozess weitergegeben werden.

4.2.2 StreamFlow

StreamFlow verteilt die Umsetzung auf drei Dateien: `workflow.cwl` enthält die Workflow-Beschreibung, `streamflow.yml` die Zuordnung zur Ausführungsumgebung und `config.yml` die Eingabewerte und Pfade der Skripte.

Ausdruck des Musters: Eine separate Abhängigkeitsangabe gibt es nicht. Jeder Schritt benennt unter `in` seine Eingaben und eine dieser Eingaben verweist auf die Ausgabe des vorherigen Schritts. Die Reihenfolge entsteht damit aus den Verweisen zwischen den Schritten (Listing 4.2).

```
1 preprocess:
2   in:
3     script:      preprocess_script
4     timing_helper: timing_helper
5     raw_file:    generate_input/raw_file
6   out: [prepared_file, timing_file]
```

P. 4.2: Verweis auf die Ausgabe des Vorgängers in `workflow.cwl` (StreamFlow, Pipeline)

Modellierungs- und Konfigurationsaufwand: Für die Workflow-Beschreibung und -Konfiguration werden drei Dateien benötigt: `workflow.cwl`, `streamflow.yml` und `config.yml`. Eine Änderung der Rechenlast betrifft allein den Pfad des Berechnungsskripts in der fünfzeiligen `config.yml`, deren Fassungen sich in genau einer Zeile unterscheiden.

Sichtbarkeit von Datenfluss und Abhängigkeiten: Die weitergegebenen Dateien und die daraus entstehenden Abhängigkeiten sind in den Eingaben der Schritte sichtbar. Jede Eingabe verweist auf die Workflow-Eingabe oder die Ausgabe eines vorherigen Schritts, von der sie stammt. So erhält `preprocess` die Datei `raw_file` über den Verweis `generate_input/raw_file`. An diesen Verweisen lässt sich damit sowohl ablesen, welche Dateien zwischen den Schritten weitergegeben werden, als auch, welche Schritte voneinander abhängen.

Ausführung und Steuerung: Die Ausführung übernimmt die StreamFlow-Laufzeit. Ein Schritt wird ausgeführt, sobald die Eingaben vorliegen, auf die er unter `in` verweist. Für die lokale Untersuchung wurden alle Schritte einer lokalen Ausführungsumgebung zugewiesen.

Trennung von Workflow-Logik und Ausführungsumgebung: Die Zuordnung zur Ausführungsumgebung steht vollständig in `streamflow.yml` und ist von der fachlichen Beschreibung getrennt. Diese Aufteilung entspricht dem Entwurfsziel von StreamFlow, die Workflow-Beschreibung von der Ausführungsumgebung zu lösen [8, S. 1723]. Ein Wechsel der Ausführungsumgebung erfordert deshalb keine Änderung an `workflow.cwl`, sondern nur an der Zuordnung in `streamflow.yml`.

Nachvollziehbarkeit der Ausführung: Der Lauf lässt sich unmittelbar an der Ausgabe im Terminal verfolgen. Sie nennt je Schritt das Arbeitsverzeichnis und den vollständigen Aufruf mit allen Argumentpfaden sowie jede einzelne Dateiübertragung mit Quelle und Ziel. Am Ende gibt StreamFlow die als Workflow-Ausgaben deklarierten Dateien mit Pfad, Größe und Prüfsumme aus. Aus den gespeicherten Provenance-Daten lassen sich zudem ein Zeitstrahl des Laufs und ein Archiv im RO-Crate-Format erzeugen [37]. Der Zeitstrahl eines Pipeline-Laufs ist in Abbildung A.6 dargestellt.

Verständlichkeit aus Einsteigerperspektive: Für das Verständnis der Beschreibung sind die Konzepte der Common Workflow Language relevant, insbesondere die Typangaben der Ein- und Ausgaben und die Werkzeugbeschreibung, die je Schritt den Aufruf des Programms festlegt.

4.2.3 Merlin

Merlin beschreibt den Workflow in einer YAML-Spezifikation, deren Abschnitt `study` die vier Schritte enthält. Eine zweite Datei enthält die Verbindungsdaten des Nachrichtenvermittlers.

Ausdruck des Musters: Die Reihenfolge wird ausdrücklich benannt. Jeder Schritt führt unter `depends` seine Vorgänger auf (Listing 4.3).

```
1 - name: compute_1
2   run:
3     depends: [preprocess]
```

```
4 cmd: python3 $(SPECROOT)/scripts/compute_medium.py $(preprocess.  
workspace)/prepared_input.txt result.txt
```

P. 4.3: Abhängigkeit und Datenweitergabe eines Schritts (Merlin, Pipeline)

Modellierungs- und Konfigurationsaufwand: Für die Workflow-Beschreibung und -Konfiguration werden zwei Dateien benötigt: die YAML-Spezifikation mit der Beschreibung des Workflows und die Datei `app.yaml` mit den Verbindungsdaten des Nachrichtenvermittlers. Eine Änderung der Rechenlast betrifft allein den Aufruf des Berechnungsskripts in der Spezifikation.

Sichtbarkeit von Datenfluss und Abhängigkeiten: Die Abhängigkeiten werden über `depends` ausdrücklich angegeben. Welche Datei zwischen zwei Schritten weitergegeben wird, ist dagegen im jeweiligen Befehl sichtbar. So verweist `compute` mit `$(preprocess.workspace)/prepared_input.txt` auf die von `preprocess` erzeugte Datei. Abhängigkeit und Datenweitergabe stehen damit an zwei Stellen desselben Schritts.

Ausführung und Steuerung: Die Schritte werden als Aufgaben in die Warteschlange des Vermittlers eingestellt, aus der die Celery-Worker sie entgegennehmen und abarbeiten [28, S. 258][@17]. Ein Schritt wird dabei erst zur Ausführung freigegeben, wenn seine unter `depends` angegebenen Vorgänger abgeschlossen sind.

Trennung von Workflow-Logik und Ausführungsumgebung: Die Trennung beruht auf dem Erzeuger-Verbraucher-Modell. Die Spezifikation beschreibt, welche Schritte auszuführen sind, während die Worker die Aufgaben aus der Warteschlange abholen, unabhängig davon, wo sie laufen. Die Workflow-Beschreibung ist damit von der ausführenden Ressource entkoppelt [28, S. 261]. Die Spezifikation benennt zwar, welche Schritte ein Worker übernimmt, nicht aber, auf welcher Ressource dieser Worker läuft.

Nachvollziehbarkeit der Ausführung: Jeder Schritt läuft in einem eigenen Arbeitsverzeichnis, sodass sich Ergebnisse und Protokolle eindeutig einem Schritt zuordnen lassen. Während der Ausführung zeigen die Ausgaben der Celery-Worker fortlaufend an, welche Aufgaben angenommen und abgeschlossen werden. Über die Kommandozeile lassen sich zudem der Zustand der Warteschlangen und Worker sowie der Fortschritt eines Laufs abfragen [@18]. Eine Funktion zur Ausgabe des Workflow-Graphen wurde in der verwendeten Version 1.13.0 nicht gefunden.

Verständlichkeit aus Einsteigerperspektive: Die Vorgänger jedes Schritts sind über `depends` ausdrücklich bezeichnet. Für das Verständnis der Beschreibung sind darüber hinaus die Workspace-Verweise und das Zusammenspiel von Nachrichtenvermittler und Workern relevant.

4.2.4 Pegasus

Der Workflow wird in `build_workflow.py` über die Python-Schnittstelle von Pegasus aufgebaut.

Ausdruck des Musters: Die Reihenfolge ergibt sich aus den Dateien. Jeder Job benennt über `add_inputs` und `add_outputs`, welche Dateien er liest und schreibt und Pegasus leitet daraus die Kanten des Graphen ab (Listing 4.4).

```
1 compute_job = Job("compute")
2 compute_job.add_args("prepared_input.txt", "result.txt")
3 compute_job.add_inputs(prepared_file)
4 compute_job.add_outputs(result_file, stage_out=False, register_replica=False)
```

P. 4.4: Ein- und Ausgaben eines Jobs (Pegasus, Pipeline)

Modellierungs- und Konfigurationsaufwand: Für die Workflow-Beschreibung und -Konfiguration wurden vier Dateien manuell erstellt: `build_workflow.py`, `sites.yml`, `pegasus.properties` und `plan.sh`. Das Erzeugungsskript `build_workflow.py` beschreibt den Workflow und erzeugt die Workflow-Datei sowie den Transformationskatalog. `sites.yml` beschreibt die Ausführungsumgebung, `pegasus.properties` enthält die Pegasus-Konfiguration und `plan.sh` dient der Planung des Workflows. Eine Änderung der Rechenlast betrifft allein den Pfad des Berechnungsskripts im Erzeugungsskript. Die erzeugten Workflow-Dateien der drei Rechenlasten sind identisch.

Sichtbarkeit von Datenfluss und Abhängigkeiten: Die weitergegebenen Dateien werden als eigene File-Objekte angelegt und bei den Jobs als Ein- und Ausgaben angegeben. So erzeugt `preprocess` die Datei `prepared_input.txt`, die `compute` anschließend als Eingabe verwendet. Die Abhängigkeiten zwischen den Jobs entstehen damit aus diesen Ein- und Ausgaben.

Ausführung und Steuerung: Pegasus trennt Planung und Ausführung. Der Planer übersetzt die abstrakte Beschreibung in einen konkreten Graphen, der anschließend an HTCondor zur Ausführung übergeben wird [10, S. 20]. Welche

Jobs mit welchen Abhängigkeiten auszuführen sind, steht damit bereits vor Beginn der Ausführung fest.

Trennung von Workflow-Logik und Ausführungsumgebung: Die Trennung beruht auf der abstrakten Workflow-Beschreibung. Der Workflow wird ressourcenunabhängig formuliert, während der Transformations- und der Site-Katalog die Verbindung zu den Programmen und den verfügbaren Rechenumgebungen herstellen und der Planer daraus den ausführbaren Workflow erzeugt [10, S. 18–20].

Nachvollziehbarkeit der Ausführung: Die Jobs eines Laufs werden in einem gemeinsamen Arbeitsverzeichnis ausgeführt, ein eigenes Verzeichnis je Job entsteht nicht. Während des Laufs lässt sich der Fortschritt über die Statusabfrage verfolgen, die die laufenden Jobs einzeln benennt und den Anteil der abgeschlossenen Jobs ausweist. Bei jedem Lauf entstanden im Submit-Verzeichnis neben den Protokollen automatisch drei Abbildungen des Workflow-Graphen, eine abstrakte Darstellung, eine Darstellung mit den beteiligten Dateien und eine des konkreten, geplanten Graphen. Die drei Darstellungen des Pipeline-Workflows sind in den Abbildungen A.8, A.9 und A.10 dargestellt.

Verständlichkeit aus Einsteigerperspektive: Die Beschreibung erfolgt in Python. Für das Verständnis der Umsetzung sind darüber hinaus die Kataloge, der vorgelagerte Planungsschritt und die Rolle von HTCondor relevant.

4.2.5 AiiDA

In AiiDA wird der Workflow als Python-Klasse beschrieben. Eine WorkChain legt den Ablauf fest, ein allgemein gehaltener CalcJob führt je Schritt eines der Benchmark-Skripte aus, ein Submit-Skript übergibt den Lauf an den Daemon.

Ausdruck des Musters: Die Reihenfolge wird ausdrücklich als Liste von Methoden in `spec.outline()` festgelegt (Listing 4.5). Die abschließende Methode `results` reicht keinen weiteren CalcJob ein, sie veröffentlicht die Ausgaben der vier fachlichen Schritte als Ausgaben der WorkChain.

```
1 spec.outline(  
2     cls.submit_generate,  
3     cls.submit_preprocess,  
4     cls.submit_compute,  
5     cls.submit_postprocess,  
6     cls.results,
```


P. 4.5: Reihenfolge in `spec.outline()` (AiiDA, Pipeline)

Modellierungs- und Konfigurationsaufwand: Die Umsetzung umfasst WorkChain, CalcJob und Submit-Skript, wobei der CalcJob allgemein formuliert ist und in beiden Mustern unverändert verwendet wird. Eine eigene Fassung je Rechenlast entfällt, die Rechenlast wird dem Submit-Skript als Argument übergeben und die WorkChain wählt daraus das Berechnungsskript aus.

Sichtbarkeit von Datenfluss und Abhängigkeiten: Die weitergegebenen Dateien werden in der WorkChain ausdrücklich den aufeinanderfolgenden Schritten zugeordnet. So wird die Ausgabe eines CalcJobs als Eingabe des nachfolgenden CalcJobs verwendet. Welche Dateien zwischen den Schritten weitergegeben werden, ist damit in der WorkChain sichtbar. Die Abhängigkeiten ergeben sich aus der in `spec.outline()` festgelegten Reihenfolge der Schritte.

Ausführung und Steuerung: Das Submit-Skript übergibt die WorkChain an den Daemon, der die Schritte der Outline abarbeitet und für jeden Schritt einen CalcJob einreicht. Der jeweils nachfolgende Schritt wird erst ausgeführt, wenn der vorherige CalcJob erfolgreich abgeschlossen wurde.

Trennung von Workflow-Logik und Ausführungsumgebung: Die Ausführungsumgebung wird als Eingabe behandelt. Rechner und Code sind in der Datenbank als eigene Objekte abgelegt und werden vom Submit-Skript geladen. Ein Wechsel der Ausführungsumgebung erfordert damit keine Änderung der WorkChain, sondern die Verwendung entsprechend eingerichteter Rechner- und Code-Objekte.

Nachvollziehbarkeit der Ausführung: Während des Laufs lässt sich der Fortschritt über die Statusabfrage verfolgen, die die laufenden Prozesse einzeln benennt und ihren Zustand ausweist. Unabhängig davon hält AiiDA jeden Schritt bereits während des Laufs in einem dauerhaften Provenance-Graphen fest. Darin sind sowohl die Prozesse als auch die Daten eigene Knoten und die Kanten geben an, welche Eingaben ein Prozess verwendet hat und welche Ausgaben aus ihm hervorgegangen sind [15, S. 7 f.]. Dabei werden zwei Zusammenhänge unterschieden. Zum einen wird festgehalten, wie Daten aus vorherigen Daten und Berechnungsschritten entstanden sind. Zum anderen wird dokumentiert, welcher Workflow die einzelnen Berechnungen ausgelöst hat. In der vorliegenden Umsetzung erscheint deshalb die WorkChain als eigener Knoten, der mit den vier von ihr eingereichten CalcJobs verbunden ist.

Der gespeicherte Ablauf lässt sich im Nachhinein abfragen und als Graph ausgeben [2]. Der Provenance-Graph des Pipeline-Laufs ist in Abbildung A.14 dargestellt.

Verständlichkeit aus Einsteigerperspektive: Die Beschreibung erfolgt in Python. Für das Verständnis der Umsetzung sind darüber hinaus die Datenobjekte von AiiDA, das Zusammenspiel von WorkChain und CalcJob, der Kontext für die Weitergabe von Ergebnissen sowie die Einrichtung von Rechner und Code relevant.

4.2.6 Vergleichende Beobachtungen

Tabelle 4.2 stellt die Beobachtungen aus den fünf Systemabschnitten entlang der Vergleichsaspekte gegenüber. Als Artefakte gezählt werden dabei die manuell erstellten Dateien, die die Workflow-Beschreibung und ihre Konfiguration ausmachen, automatisch erzeugte Dateien und die unveränderten Benchmark-Skripte bleiben unberücksichtigt. Bei AiiDA treten zu den drei Dateien Rechner und Code hinzu, die einmalig als Objekte in der Datenbank eingerichtet werden.

Tab. 4.2.: Umsetzung des Pipeline-Musters im Überblick

	Nextflow	StreamFlow	Merlin	Pegasus	AiiDA
Ausdruck des Musters	Übergabe der Prozessausgaben	Verweis auf Vorgängerausgabe	Angabe unter depends	deklarierte Ein- und Ausgaben	Liste in spec.outline()
Modellierung und Konfiguration	eine Datei	drei Dateien	zwei Dateien	vier Dateien	drei Dateien, dazu Rechner und Code
Datenfluss und Abhängigkeiten	path-Angaben und Channels	Verweise auf Schritt-ausgaben	Pfad im Befehl	File-Objekte der Jobs	Zuordnung in der WorkChain
Ausführung und Steuerung	eigene Laufzeit	eigene Laufzeit	Celery-Worker	HTCondor nach Planung	Daemon
Trennung Logik und Umgebung	eigene Konfigurationsdatei	eigene Deployment-Datei	Warteschlange trennt Schritt und Ressource	abstrakte Beschreibung, Kataloge, Planer	Rechner und Code in der Datenbank
Nachvollziehbarkeit	Fortschrittsanzeige, Trace-Datei, Zeitstrahl, Graph	Terminalausgabe, Zeitstrahl, RO-Crate	Worker-Ausgaben, Statusabfragen	Statusabfragen, drei Graphabbildungen	Statusabfragen, Provenance-Graph

Beim Ausdruck des Musters zeigen sich zwei Ansätze. Nextflow, StreamFlow und Pegasus leiten die Reihenfolge der Schritte aus den Datenbeziehungen ab, während Merlin und AiiDA die Abfolge ausdrücklich beschreiben.

Beim Modellierungs- und Konfigurationsaufwand reicht die Umsetzung von einer Datei bei Nextflow über mehrere Beschreibungs- und Konfigurationsdateien bei StreamFlow, Merlin und Pegasus bis zu drei Python-Bausteinen bei AiiDA. Eine Änderung der Rechenlast betrifft bei Nextflow, StreamFlow, Merlin und Pegasus jeweils nur eine Stelle, jedoch in unterschiedlichen Dateien. AiiDA behandelt die Rechenlast stattdessen als Aufrufargument.

Auch Datenfluss und Abhängigkeiten werden unterschiedlich sichtbar. Nextflow und StreamFlow deklarieren die weitergegebenen Dateien an den Schritten, Pegasus führt sie als eigene Objekte, bei Merlin stehen Abhängigkeit und Dateipfad getrennt und AiiDA ordnet die Ausgaben eines CalcJobs dem nachfolgenden CalcJob in der WorkChain zu.

Bei der Ausführung steuern Nextflow und StreamFlow die Schritte über ihre eigene Laufzeit. Merlin übergibt Aufgaben an Celery-Worker, Pegasus den zuvor geplanten Workflow an HTCondor und AiiDA die WorkChain an den Daemon. Bei Pegasus steht der auszuführende Graph damit bereits vor der Ausführung fest, während bei Nextflow der Start eines Prozesses durch eintreffende Daten ausgelöst wird.

Bei der Trennung von Logik und Umgebung greifen unterschiedliche Mechanismen. Nextflow und StreamFlow lagern die Umgebungsangaben in eigene Dateien aus, Merlin entkoppelt Beschreibung und Ressource über die Aufgabenwarteschlange, Pegasus formuliert den Workflow abstrakt und stellt die Verbindung zur Umgebung über Kataloge und den Planer her und AiiDA hält Rechner und Code als Objekte in der Datenbank.

Bei der Nachvollziehbarkeit bieten alle fünf Systeme Möglichkeiten, den Fortschritt während eines Laufs zu verfolgen, unterscheiden sich aber in den verbleibenden Artefakten. Nextflow stellt unter anderem Trace-Datei, Zeitstrahl und Workflow-Graph bereit, StreamFlow einen Zeitstrahl und Provenance-Daten, Merlin vor allem Arbeitsverzeichnisse und Worker-Protokolle und Pegasus Protokolle sowie Darstellungen des abstrakten und geplanten Graphen. AiiDA unterscheidet sich durch den dauerhaft gespeicherten Provenance-Graphen, der Prozesse, Daten und deren Entstehungszusammenhänge festhält.

Aus Einsteigerperspektive unterscheiden sich die Umsetzungen vor allem darin, wie viele systemeigene Konzepte vor dem ersten Lauf verstanden sein müssen. Bei Nextflow ist insbesondere das Channel-Konzept relevant, bei StreamFlow die Common Workflow Language, bei Merlin das Zusammenspiel von Nachrichtenvermittler und Workern und bei Pegasus Kataloge, Planung und HTCondor. AiiDA erfordert das Verständnis von Datenobjekten, WorkChain und CalcJob sowie der Einrichtung von Rechner und Code. Diese Einordnung beruht auf der erstmaligen Umsetzung und ersetzt keine Nutzerstudie.

4.3 Scatter-Gather-Muster

Das in Abschnitt 3.5 beschriebene Scatter-Gather-Muster wurde ebenfalls in allen fünf Systemen mit denselben Benchmark-Skripten umgesetzt. Untersucht wurden Läufe mit einem, zwei und vier Teilstücken. Im Folgenden wird gegenübergestellt, wie die fünf Systeme dieses Muster modellieren und ausführen. Die vollständigen Workflow-Beschreibungen sind im Repositorium zu dieser Arbeit abgelegt [40].

4.3.1 Nextflow

Die Beschreibung folgt dem Aufbau des Pipeline-Workflows und liegt wieder vollständig in einer Datei, die Zahl der Teilstücke steht als Parameter am Dateianfang.

Ausdruck des Musters: Die parallele Phase wird über die Operatoren `flatten` und `collect` auf den Channels ausgedrückt (Listing 4.6). `SPLIT` erzeugt die Teilstücke als gemeinsame Ausgabe, `flatten` gibt sie als einzelne Elemente weiter, sodass `COMPUTE` für jedes Element gestartet wird. `collect` sammelt anschließend die Teilergebnisse zu einer gemeinsamen Eingabe für `AGGREGATE`.

```
1 chunks_ch = SPLIT(prepared_ch)
2   .flatten()
3 results_ch = COMPUTE(chunks_ch)
4   .collect()
5 aggregated_ch = AGGREGATE(results_ch)
```

P. 4.6: Fan-out und Fan-in im workflow-Block (Nextflow, Scatter-Gather)

Modellierungs- und Konfigurationsaufwand: Die Zahl der Teilstücke ist ein Parameter, die Fassungen für einen, zwei und vier Teilstücke unterscheiden sich in genau einer Zeile. Der Wechsel der Rechenlast betrifft wie beim Pipeline-Muster allein den Aufruf des Berechnungsskripts, ebenfalls eine Zeile.

Sichtbarkeit von Datenfluss und Abhängigkeiten: Der Weg der Dateien durch die parallele Phase ist in der Beschreibung ablesbar. SPLIT deklariert seine Ausgabe als Dateimuster, an dem sichtbar ist, dass mehrere Dateien entstehen. Nach flatten erhält jede Instanz von COMPUTE genau ein Teilstück als Eingabe, während AGGREGATE nach collect die Ergebnisdateien aller Instanzen erhält. Die Abhängigkeiten entstehen wie beim Pipeline-Muster durch die Übergabe dieser Ausgaben über die Channels an die nachfolgenden Prozesse.

Ausführung und Steuerung: Für jedes Element des Eingabe-Channels startet die Nextflow-Laufzeit eine eigene Instanz von COMPUTE, die Instanzen laufen nebeneinander. AGGREGATE startet erst, wenn collect die Ergebnisse aller Instanzen zusammengeführt hat.

Trennung von Workflow-Logik und Ausführungsumgebung: Wie beim Pipeline-Muster steht die Ausführungsumgebung in der eigenen Konfigurationsdatei, die Workflow-Datei bleibt bei einem Wechsel unverändert.

Nachvollziehbarkeit der Ausführung: Die Fortschrittsanzeige zählt die parallelen Instanzen von COMPUTE als Aufgaben desselben Prozesses. Workflow-Graph und Zeitstrahl stehen wie beim Pipeline-Muster zur Verfügung, der Graph des Scatter-Gather-Workflows und der Zeitstrahl eines Laufs mit vier Teilstücken sind in den Abbildungen A.4 und A.5 dargestellt.

Verständlichkeit aus Einsteigerperspektive: Zusätzlich zum Channel-Konzept sind die beiden Operatoren flatten und collect zu verstehen, die mehrere ausgegebene Elemente einzeln weitergeben beziehungsweise wieder zu einer gemeinsamen Eingabe sammeln.

4.3.2 StreamFlow

Die Umsetzung des Scatter-Gather-Musters verwendet wie beim Pipeline-Muster `workflow.cwl`, `streamflow.yml` und `config.yml`.

Ausdruck des Musters: Die Aufteilung wird über das `scatter`-Attribut des `compute`-Schritts ausgedrückt (Listing 4.7). Der Schritt erhält das von `split`

erzeugte Feld der Teilstücke und wird für jedes Element einzeln ausgeführt. Für die Zusammenführung verweist `aggregate` mit `result_files: compute/result_file` auf die Ausgaben aller Instanzen und erhält sie als eine Liste.

```
1   compute:
2     in:
3       chunk_file: split/chunk_files
4       output_name: output_names
5     scatter: [chunk_file, output_name]
6     scatterMethod: dotproduct
```

P. 4.7: Fan-out über `scatter` am `compute`-Schritt (StreamFlow, Scatter-Gather)

Modellierungs- und Konfigurationsaufwand: Wie beim Pipeline-Muster werden drei Dateien für Beschreibung und Konfiguration verwendet. Die `workflow.cwl` bleibt über alle Varianten unverändert. Eine Änderung der Teilstückzahl betrifft allein die `config.yml`. Dort ändern sich der Eingabewert und die Liste der Ergebnisnamen, die einen Eintrag je Teilstück enthält. Der Wechsel der Rechenlast betrifft ebenfalls nur die `config.yml`, dort ändert sich der Pfad des Berechnungsskripts.

Sichtbarkeit von Datenfluss und Abhängigkeiten: Wie beim Pipeline-Muster sind die weitergegebenen Dateien und ihre Herkunft in den Eingaben der Schritte sichtbar. Zusätzlich zeigen die Typangaben, ob eine einzelne Datei oder ein Feld von Dateien weitergegeben wird. `split` erzeugt ein Feld von Teilstücken, aus dem jede Ausführung von `compute` ein Teilstück erhält. `aggregate` verweist anschließend auf die Ausgaben von `compute` und erhält das Feld aller Ergebnisdateien. Die Abhängigkeiten zwischen den Schritten ergeben sich damit wie beim Pipeline-Muster aus diesen Verweisen auf die jeweiligen Ausgaben.

Ausführung und Steuerung: Die Laufzeit startet für jedes Element eine eigene Instanz des `compute`-Schritts, die Instanzen laufen parallel in der zugewiesenen Umgebung. `aggregate` startet erst, wenn die Ergebnisse aller Instanzen vorliegen.

Trennung von Workflow-Logik und Ausführungsumgebung: Wie beim Pipeline-Muster steht die Zuordnung zur Ausführungsumgebung in `streamflow.yml`.

Nachvollziehbarkeit der Ausführung: Der Lauf lässt sich wie beim Pipeline-Muster an der Terminalausgabe verfolgen, Zeitstrahl und RO-Crate-Archiv stehen ebenfalls zur Verfügung. Der Zeitstrahl eines Scatter-Gather-Laufs ist in Abbildung A.7 dargestellt.

Verständlichkeit aus Einsteigerperspektive: Zusätzlich zu den Konzepten des Pipeline-Musters ist das `scatter`-Attribut zu verstehen sowie die Unterscheidung, ob eine Ein- oder Ausgabe eine einzelne Datei oder eine Sammlung von Dateien bezeichnet.

4.3.3 Merlin

Merlin beschreibt auch das Scatter-Gather-Muster in einer YAML-Spezifikation.

Ausdruck des Musters: Die parallele Verarbeitung wird über den Parameter `CHUNK` ausgedrückt, dessen Werteliste einen Eintrag je Teilstück enthält. Der einmal beschriebene `compute`-Schritt wird dadurch für jeden Parameterwert instanziiert. Jede Instanz hängt von `split` ab. Für den anschließenden Fan-in verweist `aggregate` unter `depends` mit `compute_*` auf alle hervorgegangenen `Compute`-Instanzen (Listing 4.8).

```
1 - name: compute
2   run:
3     depends: [split]
4     cmd: python3 $(SPECROOT)/scripts/compute_medium.py $(split.
        workspace)/chunk_$(CHUNK).txt result_$(CHUNK).txt
5
6 - name: aggregate
7   run:
8     depends: [compute_*]
```

P. 4.8: Parametrisierter Verarbeitungsschritt und Fan-in (Merlin, Scatter-Gather)

Modellierungs- und Konfigurationsaufwand: Die Zahl der Teilstücke steht an zwei Stellen der Spezifikation, in der Werteliste des Parameters und im Argument des Zerlegeschritts. Der Wechsel der Rechenlast betrifft wie beim Pipeline-Muster allein den Aufruf des Berechnungsskripts.

Sichtbarkeit von Datenfluss und Abhängigkeiten: Die Abhängigkeiten werden wie beim Pipeline-Muster über `depends` ausdrücklich angegeben. Welche Dateien weitergegeben werden, ist in den Befehlen sichtbar. Jede `compute`-Instanz liest über den jeweiligen Wert von `CHUNK` eine Datei aus dem Arbeitsverzeichnis von `split` und erzeugt die entsprechende Ergebnisdatei. `aggregate` verweist anschließend auf die Ergebnisdateien aller `Compute`-Instanzen. Abhängigkeiten und Datenweitergabe stehen damit auch im Scatter-Gather-Muster an getrennten Stellen der Beschreibung.

Ausführung und Steuerung: Die aus der Parametrisierung hervorgegangenen Compute-Instanzen werden wie beim Pipeline-Muster als Aufgaben in die Warteschlange eingestellt und von den Celery-Workern ausgeführt. Eine Compute-Instanz wird nach Abschluss von `split` zur Ausführung freigegeben. `aggregate` wird erst freigegeben, wenn alle unter `depends: [compute_*]` erfassten Compute-Instanzen abgeschlossen sind. In der lokalen Untersuchung konnte der Worker laut Spezifikation bis zu vier Aufgaben gleichzeitig bearbeiten.

Trennung von Workflow-Logik und Ausführungsumgebung: Die Trennung entspricht dem Pipeline-Muster. Die Compute-Instanzen werden als Aufgaben über die Warteschlange an die Worker übergeben, ohne dass ihre ausführende Ressource in den einzelnen Workflow-Schritten festgelegt wird. Die Anzahl gleichzeitig bearbeitbarer Aufgaben wird für den Worker im Abschnitt `resources` konfiguriert.

Nachvollziehbarkeit der Ausführung: Die einzelnen Compute-Instanzen erscheinen in den Ausgaben des Workers mit eigenem Namen, Arbeitsverzeichnis und Abschlussmeldung und hinterlassen jeweils ein eigenes Arbeitsverzeichnis. Statusabfragen über die Kommandozeile stehen wie beim Pipeline-Muster zur Verfügung, eine Graphausgabe weiterhin nicht.

Verständlichkeit aus Einsteigerperspektive: Zusätzlich zu den Konzepten des Pipeline-Musters ist das Parameter-Konzept zu verstehen. Zu beachten ist außerdem, dass die Werteliste und die im Zerlegeschritt angegebene Zahl der Teilstücke zueinander passen müssen.

4.3.4 Pegasus

Beim Scatter-Gather-Muster wird der Workflow ebenfalls über das Erzeugungsskript `build_workflow.py` aufgebaut.

Ausdruck des Musters: Die parallele Phase entsteht im Erzeugungsskript. Eine Schleife legt je Teilstück einen eigenen `compute`-Job an, der sein Teilstück als Eingabe und die zugehörige Ergebnisdatei als Ausgabe deklariert (Listing 4.9), die Kanten leitet Pegasus wie beim Pipeline-Muster aus den Dateien ab. Für die Zusammenführung deklariert der `aggregate`-Job die Ergebnisdateien aller Teilstücke als Eingaben. In der erzeugten Workflow-Datei stehen die Verarbeitungsjobs einzeln, der Umfang der parallelen Phase liegt damit bereits in der abstrakten Beschreibung fest.


```

1  for index, (chunk_file, result_file) in enumerate(zip(chunk_files,
2      result_files), start=1):
3      compute_job = Job("compute")
4      compute_job.add_args(f"chunk_{index}.txt", f"result_{index}.txt")
5      compute_job.add_inputs(chunk_file)
6      compute_job.add_outputs(result_file, stage_out=False,
          register_replica=False)
7      compute_jobs.append(compute_job)

```

P. 4.9: Erzeugung der compute-Jobs je Teilstück (Pegasus, Scatter-Gather)

Modellierungs- und Konfigurationsaufwand: Die vier für das Pipeline-Muster benötigten Dateien bleiben bestehen. Das Erzeugungsskript ist für alle Teilstückzahlen einer Rechenlast dasselbe, die Teilstückzahl wird ihm als Aufrufargument übergeben. Die erzeugte Workflow-Datei wächst mit der Teilstückzahl, da je Teilstück ein Verarbeitungsjob und die zugehörigen Dateiobjekte hinzukommen. Eine Änderung der Rechenlast betrifft wie beim Pipeline-Muster allein den Pfad des Berechnungsskripts im Erzeugungsskript.

Sichtbarkeit von Datenfluss und Abhängigkeiten: Wie beim Pipeline-Muster werden die weitergegebenen Dateien als eigene File-Objekte angelegt und bei den Jobs als Ein- und Ausgaben angegeben. Die parallele Phase ist daran ablesbar, dass `split` alle Teilstücke als Ausgaben deklariert, jeder `compute`-Job genau eines dieser Teilstücke als Eingabe verwendet und `aggregate` die Ergebnisdateien aller `compute`-Jobs als Eingaben erhält. Die Abhängigkeiten zwischen den Jobs entstehen damit aus diesen Ein- und Ausgaben.

Ausführung und Steuerung: Planung und Ausführung bleiben getrennt. Bei vier Teilstücken enthält die abstrakte Beschreibung neun fachliche Jobs. Der Planer übersetzt sie in den ausführbaren Graphen, den HTCondor abarbeitet. Dabei ergänzt er Jobs für das Anlegen von Verzeichnissen, das Sichern der Ergebnisse und das Aufräumen.

Trennung von Workflow-Logik und Ausführungsumgebung: Die Trennung über die abstrakte Beschreibung und die Kataloge bleibt wie beim Pipeline-Muster bestehen, ebenso die Einschränkung, dass das Erzeugungsskript die Zielsite benennt.

Nachvollziehbarkeit der Ausführung: Die drei Abbildungen des Workflow-Graphen entstehen wie beim Pipeline-Muster bei jedem Lauf. In der abstrakten Darstellung sind die Verarbeitungsjobs einzeln sichtbar, im geplanten

Graphen kommen die vom Planer ergänzten Jobs hinzu. Die drei Darstellungen des Scatter-Gather-Workflows sind in den Abbildungen A.11, A.12 und A.13 dargestellt.

Verständlichkeit aus Einsteigerperspektive: Zusätzlich zu den Konzepten des Pipeline-Musters ist zu verstehen, dass die parallele Phase nicht über ein Sprachmittel des Systems ausgedrückt, sondern im Erzeugungsskript ausgeschrieben wird. In der erzeugten Beschreibung ist sie nur daran erkennbar, dass die Verarbeitungsjobs keine Dateien voneinander lesen.

4.3.5 AiiDA

Der Aufbau aus WorkChain, CalcJob und Submit-Skript bleibt bestehen. Die Outline wächst um das Zerlegen und das Zusammenführen, die parallele Verarbeitung bleibt darin ein einzelner Eintrag. Teilstückzahl und Rechenlast sind Eingaben der WorkChain.

Ausdruck des Musters: Die parallele Phase entsteht in einem einzigen Outline-Schritt. Eine Schleife reicht je Teilstück einen eigenen CalcJob ein, der sein Teilstück verarbeitet (Listing 4.10) und ToContext lässt die WorkChain auf den Abschluss aller eingereichten CalcJobs warten, bevor der nächste Outline-Schritt beginnt. Dort sammelt submit_aggregate die Ergebnisordner aller Instanzen ein und reicht einen einzelnen CalcJob für das Zusammenführen ein.

```
1     def submit_compute(self):
2         futures = {}
3         for index in range(1, self.inputs.num_chunks.value + 1):
4             future = self.submit_task(
5                 label=f"compute_{index}",
6                 arguments=[f"chunk_{index}.txt", f"result_{index}.txt"
7                     ],
8             )
9             futures[f"compute_{index}"] = future
10        return ToContext(**futures)
```

P. 4.10: Fan-out in submit_compute (AiiDA, Scatter-Gather)

Modellierungs- und Konfigurationsaufwand: Eine eigene Fassung je Teilstückzahl oder Rechenlast entfällt. Beide werden dem Submit-Skript als Argumente übergeben und gehen als Eingaben in die WorkChain ein. Die Rechenlast wählt wie beim Pipeline-Muster das Berechnungsskript aus.

Sichtbarkeit von Datenfluss und Abhängigkeiten: Die Weitergabe ist wie beim Pipeline-Muster an drei Stellen beschrieben, als Quellordner, als Zuordnung und als Abrufliste, nun jedoch je Instanz. Jede Verarbeitungsinstanz erhält den Ordner des Zerlegeschritts als Quelle mit der Zuordnung genau ihres Teilstücks. Beim Zusammenführen erhält der aggregate-CalcJob die Ordner aller Instanzen als Quellen mit einer Zuordnung je Ergebnisdatei. Die Abhängigkeit des anschließenden Zusammenführens von allen Verarbeitungsinstanzen ergibt sich aus dem von ToContext hergestellten Wartepunkt innerhalb der Outline.

Ausführung und Steuerung: Die WorkChain wird weiterhin über den Daemon verwaltet. Die Ausführung übernehmen dabei mehrere parallel laufende Runner, die Aufgaben unabhängig voneinander verarbeiten [15, S. 3]. Da die Verarbeitungsinstanzen gemeinsam eingereicht werden, können ihre CalcJobs gleichzeitig verarbeitet werden. Durch ToContext setzt die WorkChain erst fort, wenn alle eingereichten CalcJobs beendet sind.

Trennung von Workflow-Logik und Ausführungsumgebung: Die Trennung entspricht dem Pipeline-Muster: Rechner und Code sind in der AiiDA-Datenbank abgelegt und werden der WorkChain als Eingaben übergeben.

Nachvollziehbarkeit der Ausführung: Der Provenance-Graph enthält die Verarbeitungsinstanzen als eigene Prozessknoten, die sämtlich mit der WorkChain verbunden sind. Die Kanten zeigen, aus welchem Ordnerknoten jede Instanz ihr Teilstück bezogen hat und welche Ergebnisse in das Zusammenführen eingegangen sind. Der Provenance-Graph eines Scatter-Gather-Laufs ist in Abbildung A.15 dargestellt.

Verständlichkeit aus Einsteigerperspektive: Zusätzlich zu den Konzepten des Pipeline-Musters ist zu verstehen, dass ein Outline-Schritt mehrere CalcJobs einreichen kann und ToContext die WorkChain anschließend auf deren Abschluss warten lässt, bevor der nächste Outline-Schritt beginnt.

4.3.6 Vergleichende Beobachtungen

Tabelle 4.3 stellt die Beobachtungen aus den fünf Systemabschnitten entlang der Vergleichsaspekte gegenüber. Da die beteiligten Dateien und Bausteine gegenüber dem Pipeline-Muster unverändert bleiben, tritt bei der Modellierung an die Stelle der Artefaktzahl die Frage, welche Stellen eine Änderung der Teilstückzahl betrifft. Auch die Trennung von Logik und Umgebung sowie die

verfügbaren Mittel der Nachvollziehbarkeit entsprechen dem Pipeline-Muster, aufgeführt ist deshalb nur, wie sich die parallele Phase darin abbildet.

Tab. 4.3.: Umsetzung des Scatter-Gather-Musters im Überblick

	Nextflow	StreamFlow	Merlin	Pegasus	AiiDA
Ausdruck des Musters	flatten und collect	scatter-Attribut	Parameter CHUNK, compute_*	Schleife im Erzeugungsskript	Schleife im Outline-Schritt, ToContext
Änderung der Teilstückzahl	ein Parameter	Eingabewert und Namensliste in config.yml	zwei Stellen der Spezifikation	Aufrufargument, Workflow-Datei wird neu erzeugt	Aufrufargument, WorkChain bleibt unverändert
Datenfluss und Abhängigkeiten	path-Angaben und Channels	Verweise auf Schrittausgaben	depends und Pfade in den Befehlen	File-Objekte als Ein- und Ausgaben	Quellordner und Zuordnungen je Instanz, ToContext
Ausführung und Steuerung	Nextflow-Laufzeit, eine Instanz je Teilstück	StreamFlow-Laufzeit, eine Instanz je Teilstück	Celery-Worker, eine Aufgabe je Instanz	HTCondor, ein Job je Teilstück	Daemon/Runner, ein CalcJob je Teilstück
Trennung Logik und Umgebung	Konfigurationsdatei	Deployment-Datei	Warteschlange	abstrakte Beschreibung und Kataloge	Rechner und Code
Nachvollziehbarkeit	Instanzen als Aufgaben desselben Prozesses	Instanzen einzeln in der Terminalausgabe	Instanzen einzeln in den Worker-Ausgaben	Instanzen einzeln in der Statusabfrage	Instanzen einzeln in Statusabfrage und Graph

Beim Ausdruck des Musters verwenden die Systeme unterschiedliche Mechanismen für die parallele Phase. Nextflow nutzt `flatten` und `collect`, StreamFlow das `scatter`-Attribut und Merlin die Parametrisierung über `CHUNK`. Pegasus und AiiDA erzeugen die Verarbeitungsinstanzen programmatisch über eine Python-Schleife, bei Pegasus im Erzeugungsskript und bei AiiDA innerhalb eines Outline-Schritts.

Bei einer Änderung der Teilstückzahl unterscheiden sich die erforderlichen Anpassungen. In Nextflow wird ein einzelner Parameter geändert, in StreamFlow der Eingabewert und die Liste der Ergebnisnamen und in Merlin zwei Stellen der Spezifikation. Pegasus und AiiDA verwenden jeweils ein Aufrufargument. Bei Pegasus wird daraus jedoch eine neue, mit der Teilstückzahl wachsende Workflow-Datei erzeugt, während die WorkChain von AiiDA unverändert bleibt.

Bei Datenfluss und Abhängigkeiten unterscheiden sich die Systeme darin, wie die Weitergabe der Teilstücke in der Beschreibung sichtbar wird. Nextflow und StreamFlow deklarieren die weitergegebenen Dateien an den Schritten,

Pegasus führt sie als eigene Dateiobjekte, in Merlin stehen die Dateipfade in den Befehlen und AiiDA verwendet Quellordner mit Dateizuordnungen. Die Abhängigkeiten ergeben sich bei Nextflow, StreamFlow und Pegasus aus der Datenweitergabe. Merlin gibt sie mit `depends` ausdrücklich an, während in AiiDA die Outline und der durch ToContext hergestellte Wartepunkt die Abfolge bestimmen.

Bei der Ausführung bleiben die ausführenden Komponenten gegenüber dem Pipeline-Muster unverändert. Neu ist die parallele Phase, in der je Teilstück eine eigene Verarbeitungsinstanz ausgeführt wird. Das Zusammenführen wird in allen fünf Systemen erst ausgeführt, nachdem alle Verarbeitungsinstanzen abgeschlossen sind.

Die Trennung von Workflow-Logik und Ausführungsumgebung bleibt in allen fünf Systemen unverändert bestehen, die parallele Phase berührt sie nicht.

Bei der Nachvollziehbarkeit kommt hinzu, wie die einzelnen Verarbeitungsinstanzen sichtbar werden. Nextflow zählt sie in der Fortschrittsanzeige als Aufgaben desselben Prozesses, während sie in StreamFlow, Merlin und Pegasus einzeln in den jeweiligen Ausgaben beziehungsweise Statusinformationen erscheinen. AiiDA hält die Instanzen zusätzlich als eigene Prozessknoten im dauerhaft gespeicherten Provenance-Graphen fest.

Aus Einsteigerperspektive kommen gegenüber dem Pipeline-Muster zusätzliche Konzepte für die parallele Phase hinzu: bei Nextflow `flatten` und `collect`, bei StreamFlow `scatter`, bei Merlin die Parametrisierung und bei AiiDA das Einreichen mehrerer CalcJobs mit ToContext. Bei Pegasus wird die parallele Phase dagegen im Python-Erzeugungsskript aufgebaut.

Wie sich die beschriebenen Unterschiede im Laufzeitverhalten niederschlagen, untersucht Abschnitt 4.4.

4.4 Ergebnisse der lokalen Messungen

Die folgenden Ergebnisse werden entsprechend dem in Kapitel 3 beschriebenen Mess- und Auswertungsverfahren als Mediane über die Wiederholungen berichtet. Betrachtet werden die Gesamtlaufzeit, der Overhead sowie die beiden Koordinationsgrößen Übergangslatenz und Startspreizung.

Abbildung 4.1 zeigt die Gesamtlaufzeiten des Pipeline-Musters je Rechenlast. Vier Systeme liegen dabei näher beieinander, bei der kurzen Rechenlast zwischen 5,8 und 9,2 Sekunden, bei der mittleren zwischen 53,4 und 63,1 Sekunden und bei der langen zwischen 238 und 280 Sekunden. Pegasus liegt mit 184, 228 und 442 Sekunden durchgängig deutlich darüber. Sein Abstand zum eigenen Referenzlauf beträgt bei allen drei Rechenlasten zwischen 171 und 180 Sekunden und ist damit nahezu konstant. Er geht damit auf Anteile zurück, die unabhängig von der Rechenlast anfallen, darunter der vorgelagerte Planungsschritt und die Übergänge zwischen den Jobs. Auffällig ist zudem AiiDA bei der kurzen Rechenlast, dessen Laufzeit von 9,2 Sekunden über den drei übrigen Systemen dieser Gruppe liegt, während sich dieser Unterschied bei der mittleren und langen Rechenlast nicht in gleicher Weise zeigt.

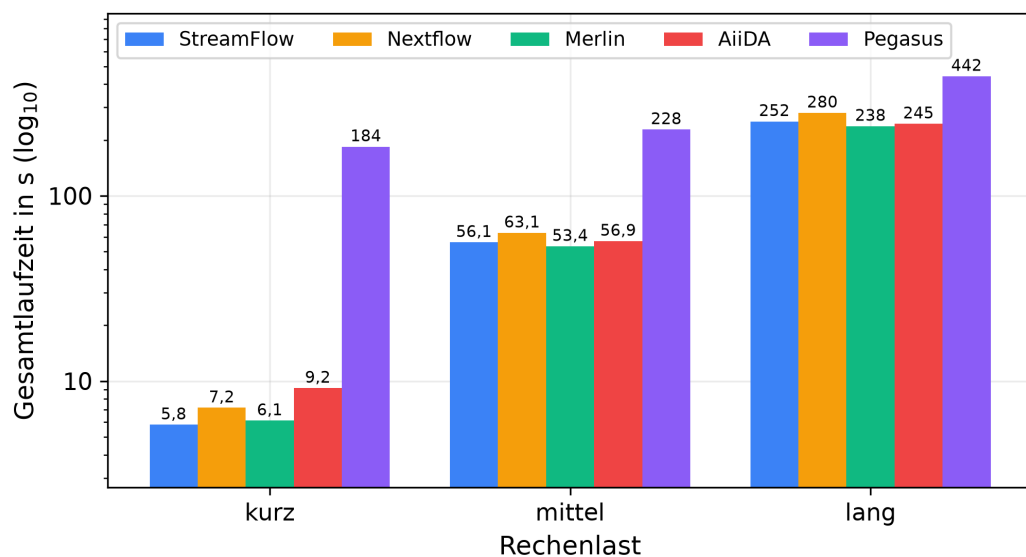


Abb. 4.1.: Median der Gesamtlaufzeit des Pipeline-Musters je Rechenlast auf der lokalen Ebene, logarithmische Skala

Abbildung 4.2 zeigt die Gesamtlaufzeiten des Scatter-Gather-Musters über die Zahl der Teilstücke. Bei der mittleren und der langen Rechenlast sinken die Laufzeiten der vier Systeme im engen Band mit jeder Verdopplung der Teilstückzahl deutlich, bei der langen Rechenlast etwa von 248 auf 66 Sekunden in StreamFlow und von 255 auf 73 Sekunden in AiiDA. Bei der kurzen Rechenlast fällt der Rückgang geringer aus, AiiDA liegt dort bei vier Teilstücken mit 10,7 Sekunden sogar leicht über dem Wert bei zwei Teilstücken. Pegasus folgt dem Verlauf ebenfalls, jedoch flacher, seine Laufzeiten sinken bei der langen Rechenlast von 480 auf 289 Sekunden, bei der kurzen bleiben sie mit

rund 207 Sekunden über alle Teilstückzahlen nahezu unverändert. Der Anteil, der nicht auf die Rechenlogik zurückgeht, verändert sich dabei in allen Systemen nur wenig, in StreamFlow, Nextflow und Merlin liegt er bei der langen Rechenlast zwischen 1,5 und 3,5 Sekunden, in AiiDA zwischen 8,9 und 10,3 Sekunden und in Pegasus zwischen 216 und 221 Sekunden.

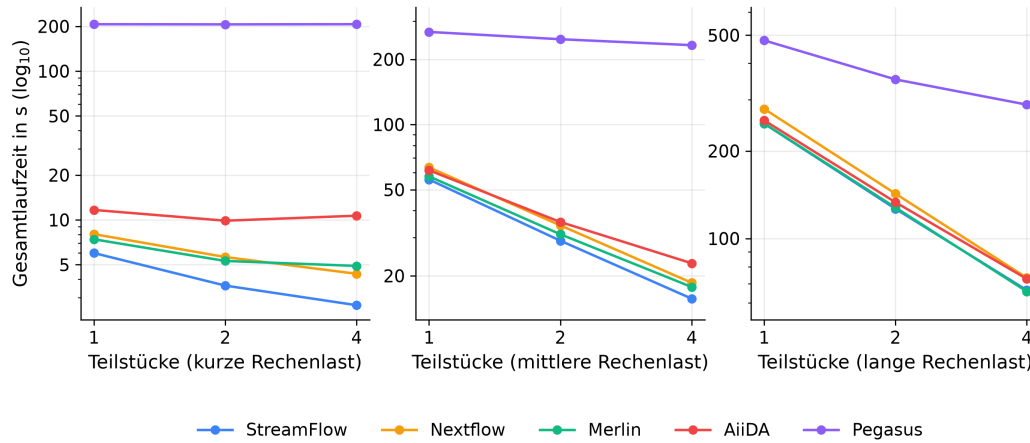


Abb. 4.2.: Median der Gesamtlaufzeit des Scatter-Gather-Musters über die Zahl der Teilstücke je Rechenlast auf der lokalen Ebene, logarithmische Skala

Abbildung 4.3 stellt den Overhead aller zwölf Konfigurationen gegenüber. Die Systeme bilden drei Gruppen, deren Reihenfolge in jeder einzelnen Konfiguration gleich bleibt. StreamFlow, Nextflow und Merlin liegen zwischen 1,4 und 4,2 Sekunden. AiiDA liegt zwischen 5,1 und 10,3 Sekunden und Pegasus zwischen 171 und 221 Sekunden. Ein proportionaler Zusammenhang zwischen Rechenlast und Overhead ist in keiner der Gruppen erkennbar. Ein Zusammenhang mit dem Umfang des Workflows zeigt sich dagegen bei einzelnen Systemen. Beim Scatter-Gather-Muster mit seinen sechs Schritten liegt der Overhead von Pegasus durchgängig und der von AiiDA bei der kurzen und der mittleren Rechenlast über dem des Pipeline-Musters mit vier Schritten. Bei AiiDA steht dies im Einklang mit der Abwicklung jedes Schritts als eigener CalcJob über Daemon und Datenbank, bei Pegasus mit den zusätzlichen Übergängen zwischen den Schritten.

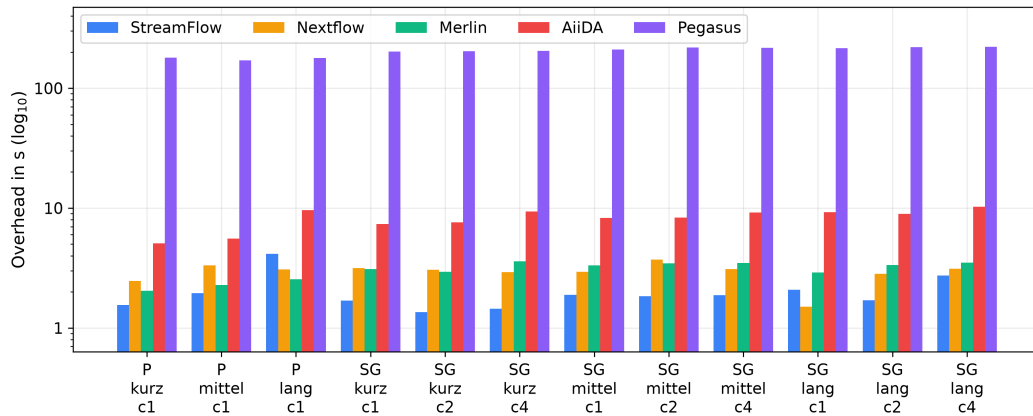


Abb. 4.3.: Overhead gegenüber dem Referenzlauf für alle zwölf Konfigurationen auf der lokalen Ebene, logarithmische Skala

Abbildung 4.4 zeigt die Übergangslatenzen zwischen aufeinanderfolgenden Schritten. Die Systeme unterscheiden sich darin über mehrere Größenordnungen, in StreamFlow liegt der Median je Übergang bei etwa 0,06 Sekunden, in Nextflow bei 0,09, in Merlin bei 0,38, in AiiDA bei 1,16 und in Pegasus bei 20,1 Sekunden. Innerhalb eines Systems sind die Werte über die einzelnen Übergänge und über beide Muster hinweg weitgehend gleich. Die Übergangslatenz zeigt sich damit im lokalen Aufbau als eine weitgehend systemspezifische Größe. Die Unterschiede stehen im Zusammenhang mit der jeweiligen Ausführungsarchitektur. In Merlin erfolgt die Koordination über die Warteschlange, in AiiDA über Daemon und Datenbank und in Pegasus über die Übergabe der einzelnen Jobs an HTCondor. In Nextflow und StreamFlow werden die Schritte direkter durch die jeweilige Laufzeit koordiniert. Bei diesen beiden Systemen fallen zugleich die kleinsten gemessenen Abstände an.

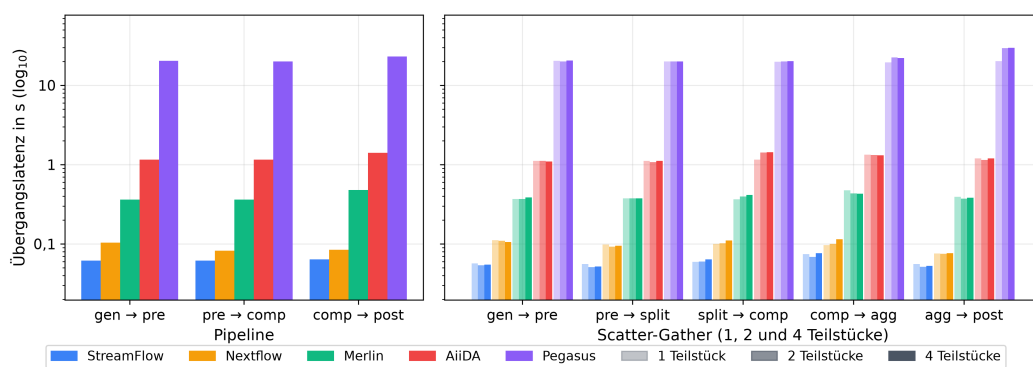


Abb. 4.4.: Median der Übergangslatenzen zwischen aufeinanderfolgenden Schritten auf der lokalen Ebene, logarithmische Skala

Abbildung 4.5 zeigt die Startspreizung der parallelen Verarbeitungsinstanzen, also den Abstand zwischen dem ersten und dem letzten Instanzstart. Die Werte reichen dabei von wenigen Millisekunden bis in den Bereich einer knappen Sekunde. Bei zwei Teilstücken reicht der Median von 1 Millisekunde in Nextflow über 9 in StreamFlow und 19 in Pegasus bis zu 58 in AiiDA und 228 in Merlin, bei vier Teilstücken von 7 über 40 und 62 bis zu 698 und 734 Millisekunden. In allen fünf Systemen wächst die Spreizung mit der Zahl der Teilstücke, am deutlichsten in AiiDA, wo sie sich von zwei auf vier Teilstücke etwa verzwölffacht. Die Staffelung entspricht dem Weg, über den die Instanzen untereinander gestartet werden. In Nextflow, StreamFlow und Pegasus entstehen sie ohne Zwischenstelle, in Nextflow und StreamFlow startet die Laufzeit sie unmittelbar nacheinander, in Pegasus liegen die Jobs bereits geplant vor und werden gemeinsam in die Warteschlange gegeben. In AiiDA und Merlin wird dagegen jede Instanz einzeln in eine Warteschlange eingereiht, aus der sie in AiiDA die Daemon-Runner und in Merlin ein Worker entgegennehmen, und der Abstand wächst dort mit jeder weiteren Instanz.

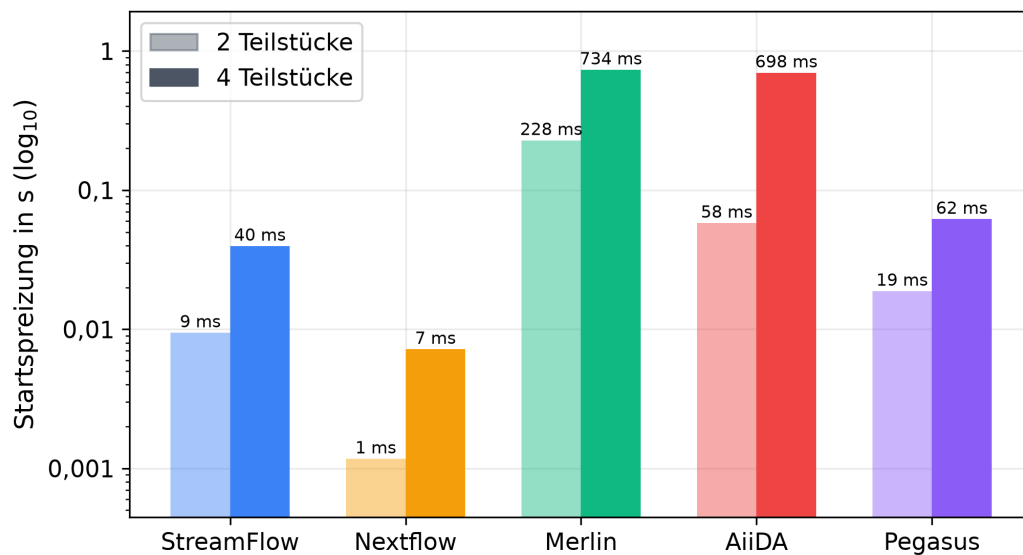


Abb. 4.5.: Median der Startspreizung der parallelen Verarbeitungsinstanzen auf der lokalen Ebene, logarithmische Skala

Über die einzelnen Kenngrößen hinweg fügen sich die Befunde zu einem stimmigen Bild. Bei Pegasus entspricht der höhere Overhead des Scatter-Gather-Musters näherungsweise den zusätzlichen Übergangslatenzen, die Differenz der Overhead-Mediane beider Muster beträgt 37,9 Sekunden, während das Muster zwei Übergänge mehr enthält als die Pipeline und zweimal 20,1 Sekunden gemessener Latenz 40,2 Sekunden ergeben. Zugleich zeigt Pe-

gasus, dass die beiden Koordinationsgrößen verschiedene Vorgänge erfassen, seine Übergänge sind mit Abstand die teuersten, seine Startspreizung liegt mit 19 und 62 Millisekunden dagegen im unteren Feld. AiiDA und Merlin, die jede Instanz einzeln in eine Warteschlange einreihen, liegen in beiden Koordinationsgrößen über Nextflow und StreamFlow. Ein unmittelbarer Schluss von den Koordinationsgrößen auf die Höhe des Overheads ist damit nicht möglich. Merlin weist höhere Übergangslatenzen auf als Nextflow und StreamFlow, liegt beim Overhead aber in derselben Gruppe und in einzelnen Konfigurationen darunter. Die Koordinationsgrößen zeigen, an welchen Stellen ein System zusätzliche Zeit beansprucht, sie bestimmen den Overhead jedoch nicht allein.

4.5 Zwischenfazit

Beide Workflow-Muster ließen sich in allen fünf Systemen mit denselben Benchmark-Skripten umsetzen. Die qualitative Gegenüberstellung zeigt dabei Unterschiede darin, wie diese Muster in den einzelnen Systemen umgesetzt werden. Bei der Modellierung unterscheiden sich die Systeme unter anderem darin, ob die Reihenfolge aus den Datenbeziehungen entsteht oder ausdrücklich benannt wird und wann die auszuführenden Aufgaben feststehen. Eine geänderte Rechenlast betrifft in allen Systemen nur eine Stelle, eine geänderte Teilstückzahl reicht dagegen von einem einzelnen Parameter bis zum erneuten Erzeugen der Workflow-Beschreibung. Auch die Ausführung erfolgt unterschiedlich: Nextflow und StreamFlow steuern die Schritte über ihre eigene Laufzeit, während bei Merlin, Pegasus und AiiDA weitere Komponenten an der Ausführung beteiligt sind.

Die Messungen zeigen drei Overhead-Gruppen, deren Reihenfolge in allen zwölf Konfigurationen gleich bleibt, StreamFlow, Nextflow und Merlin im unteren Bereich, AiiDA darüber und Pegasus mit deutlichem Abstand darüber. Ein proportionaler Zusammenhang zwischen Rechenlast und Overhead ist dabei nicht erkennbar. Unterschiede zwischen den beiden Workflow-Mustern zeigen sich insbesondere bei AiiDA und Pegasus. Die Übergangslatenz reicht über mehrere Größenordnungen von StreamFlow bis Pegasus und bleibt innerhalb eines Systems weitgehend stabil, während die Startspreizung mit der Zahl der parallelen Instanzen wächst. Die Koordinationsgrößen machen damit Unterschiede in der Übergabe aufeinanderfolgender Schritte und im

Start paralleler Instanzen sichtbar, ohne die Höhe des gesamten Overheads allein zu bestimmen.

Diese Befunde beschreiben das Verhalten der Systeme unter den kontrollierten Bedingungen eines einzelnen Rechners. Kapitel 5 untersucht, wie sich die Systeme auf einem Slurm-basierten HPC-System bereitstellen und betreiben lassen.

Evaluation auf dem HPC-System MOGON

5.1 Zielsetzung

Die lokale Evaluation in Kapitel 4 betrachtet die fünf Workflow-Management-Systeme unter kontrollierten Bedingungen auf einem einzelnen Rechner. Damit ist die Frage, wie sich dieselben Systeme in einer HPC-Umgebung einsetzen lassen, noch nicht beantwortet. Die Untersuchung auf MOGON unterscheidet sich in zwei Punkten von der lokalen Ausführung. Die Software muss unter den Rechten eines gewöhnlichen Nutzerkontos bereitgestellt werden, ohne Administrationsrechte. Die Ausführung der Rechenschritte erfolgt nicht unmittelbar, sondern über einen Ressourcenverwalter, der die Rechenknoten zwischen allen Nutzenden aufteilt.

Dieses Kapitel untersucht deshalb zunächst, mit welchem Aufwand sich die Systeme auf dem HPC-System MOGON bereitstellen lassen und wie sie ihre Rechenschritte an den Ressourcenverwalter Slurm übergeben. Darauf aufbauend wird betrachtet, welche Zeitanteile eines Laufs im regulären Clusterbetrieb entstehen und woher sie stammen. Aus dieser Betrachtung ergibt sich der Aufbau der anschließenden Messung, deren Ergebnisse den letzten Teil des Kapitels bilden.

Workflows, Benchmark-Skripte, Instrumentierung und Messgrößen sind dieselben wie in Kapitel 3 beschrieben, ebenso die Referenzmessung ohne Workflow-Management-System und die fünf beziehungsweise drei Wiederholungen je Konfiguration, über die die Messwerte zusammengefasst werden. Da sich die Ausführungsbedingungen beider Ebenen unterscheiden, werden die Ergebnisse getrennt betrachtet und nicht unmittelbar gegeneinander gestellt.

5.2 Versuchsumgebung

Die Untersuchung wurde auf dem HPC-System MOGON NHR Süd-West der Johannes Gutenberg-Universität Mainz durchgeführt, das aus 590 Rechenknoten mit insgesamt rund 75.000 Rechenkernen besteht [22]. Der Zugang erfolgt über einen Login-Knoten, auf dem Software eingerichtet und Rechenaufträge eingereicht werden. Die eigentliche Ausführung findet auf Rechenknoten statt, die der Ressourcenverwalter Slurm in der Version 24.11.6 zuteilt.

Alle Läufe wurden in der Partition `ki-smallcpu` ausgeführt, die 29 gleichartige Rechenknoten umfasst. Ein Knoten dieser Partition besitzt zwei Prozessoren des Typs AMD EPYC 7713 mit je 64 Kernen [22], insgesamt 128 Rechenkerne und 248 GB Hauptspeicher. Als Betriebssystem kommt AlmaLinux 8.10 zum Einsatz.

Für die Datenhaltung stehen zwei Ablageorte zur Verfügung. Das gemeinsame Dateisystem `/fshpc` ist von allen Knoten aus erreichbar. Zusätzlich verfügt jeder Rechenknoten über lokalen Speicher unter `/localscratch`. Dieser steht während eines Rechenauftrags zur Verfügung, wird nach dessen Ende wieder gelöscht und ist nur auf dem jeweiligen Knoten sichtbar [20]. Die Benchmark-Workflows und ihre Arbeitsverzeichnisse lagen im gemeinsamen Dateisystem, da Workflow-Schritte auf unterschiedlichen Rechenknoten ausgeführt werden können und ihre Zwischenergebnisse deshalb über einen von allen Knoten erreichbaren Speicher austauschen müssen.

Software wird auf dem Cluster über ein Modulsystem bereitgestellt, über das sich vorinstallierte Pakete in die eigene Umgebung laden lassen [20]. Welche der untersuchten Systeme darüber verfügbar waren und wie die übrigen eingerichtet wurden, beschreibt Abschnitt 5.3.

5.3 Bereitstellung und Betrieb der Workflow-Management-Systeme

Die folgenden Abschnitte beschreiben je System, wie es auf MOGON bereitgestellt wurde, welche Bestandteile der lokalen Umsetzung übernommen werden konnten, welche Anpassungen für die Ausführung über Slurm erforderlich waren und wie ein Lauf betrieben wird.

5.3.1 Nextflow

Nextflow steht auf MOGON als Modul zur Verfügung und wird mit `module load tools/Nextflow/25.10.4` in die eigene Umgebung geladen. Damit ist die Bereitstellung abgeschlossen. Das Modul bringt die benötigte Java-Laufzeitumgebung bereits mit. Wie in der lokalen Untersuchungsumgebung (Abschnitt 4.1) benötigt Nextflow auch auf MOGON keine zusätzlichen Dienste oder dauerhaft laufenden Hintergrundprozesse.

Die Workflow-Beschreibungen beider Muster wurden unverändert aus der lokalen Umsetzung übernommen. Zusätzlich wurde eine Konfigurationsdatei `nextflow.config` verwendet. Während Nextflow die Schritte in der lokalen Ausführungsumgebung unmittelbar als Prozesse auf dem ausführenden Rechner startet, legt die Konfigurationsdatei auf dem Cluster fest, dass die Ausführung über Slurm erfolgt und mit welchen Ressourcen die einzelnen Workflow-Schritte angefordert werden.

Listing 5.1 zeigt die verwendete Konfiguration. Die Zeile `executor = 'slurm'` legt fest, dass Nextflow die Workflow-Schritte über Slurm ausführt, alle weiteren Angaben betreffen die einzelnen Slurm-Jobs. Die Angaben zu Partition, Konto, Rechenkernen, Hauptspeicher und Zeitlimit werden für die einzelnen Slurm-Jobs verwendet und gelten damit je Schritt und nicht für den Workflow als Ganzes. Der Eintrag `queueSize` begrenzt darüber hinaus, wie viele Jobs Nextflow gleichzeitig bei Slurm einreicht.

```
1 process {
2     executor = 'slurm'
3     queue = 'ki-smallcpu'
4     clusterOptions = '-A ki-mawahpc'
5     cpus = 1
6     memory = '1 GB'
7     time = '00:10:00'
8 }
9 executor {
10     queueSize = 4
11 }
```

P. 5.1: Konfiguration von Nextflow für die Ausführung auf MOGON

Gestartet wird ein Lauf mit einem einzelnen Aufruf von `nextflow run` auf dem Login-Knoten. Der dabei entstehende Vordergrundprozess übernimmt die Steuerung des Workflows, reicht die Schritte bei Slurm ein und überwacht deren Ausführung. Mit dem Ende dieses Prozesses ist der Lauf abgeschlossen,

es bleibt kein Dienst zurück. Bei Einrichtung und Betrieb traten keine Fehler auf.

5.3.2 StreamFlow

StreamFlow steht auf MOGON nicht als Modul zur Verfügung und wurde deshalb mit pip in einer eigenen Conda-Umgebung installiert. Zum Einsatz kommt dieselbe Vorabversion 0.2.0rc2. Wie in der lokalen Untersuchungsumgebung (Abschnitt 4.1) benötigt StreamFlow auch auf MOGON keine zusätzlichen Dienste.

Die Workflow-Beschreibungen beider Muster wurden unverändert aus der lokalen Umsetzung übernommen. Zusätzlich wurde die Datei `streamflow.yml` angepasst. Während sie in der lokalen Ausführungsumgebung ein Deployment vom Typ `local` verwendet, das die Workflow-Schritte unmittelbar als Prozesse auf dem ausführenden Rechner startet, legt sie auf dem Cluster fest, dass die Ausführung über Slurm erfolgt und mit welchen Ressourcen die einzelnen Workflow-Schritte angefordert werden.

Listing 5.2 zeigt einen Auszug aus der Umgebungsbeschreibung. Der Eintrag `bindings` ordnet die Schritte des Workflows dem Deployment `slurm-mogon` zu. Dessen Angaben zu Partition, Konto, Rechenkernen, Hauptspeicher und Zeitlimit gelten je Schritt und nicht für den Workflow als Ganzes. Der Eintrag `maxConcurrentJobs` begrenzt darüber hinaus, wie viele Jobs StreamFlow gleichzeitig bei Slurm einreicht. Über `wraps` ist festgelegt, dass das Slurm-Deployment auf dem lokalen Deployment `mogon-local` aufbaut, über das die Slurm-Kommandos vom Login-Knoten aus ausgeführt werden. Der Eintrag `workdir` legt fest, in welchem Verzeichnis StreamFlow die Arbeitsverzeichnisse der einzelnen Workflow-Schritte anlegt. Voreingestellt ist `/tmp`, das auf jedem Rechenknoten getrennt existiert. Für die Ausführung auf MOGON wurde stattdessen das gemeinsame Dateisystem `/fshpc` verwendet, da Workflow-Schritte auf unterschiedlichen Rechenknoten ausgeführt werden können und die erzeugten Dateien allen nachfolgenden Schritten zur Verfügung stehen müssen.

```
1 bindings:
2   - step: /
3     target:
4       deployment: slurm-mogon
5       service: default
6
```

```

7 deployments:
8   mogon-local:
9     type: local
10    workdir: /fshpc/antahiri/slurm_modus/streamflow/pipeline/workdir
11    config: {}
12
13   slurm-mogon:
14     type: slurm
15     wraps: mogon-local
16     workdir: /fshpc/antahiri/slurm_modus/streamflow/pipeline/workdir
17     config:
18       maxConcurrentJobs: 4
19       services:
20         default:
21           account: ki-mawahpc
22           partition: ki-smallcpu
23           nodes: "1"
24           ntasks: 1
25           cpusPerTask: 1
26           mem: "1G"
27           time: "00:10:00"

```

P. 5.2: Auszug aus der Umgebungsbeschreibung von StreamFlow für die Ausführung auf MOGON

Eine weitere Anpassung betraf StreamFlow selbst. Um den Zustand eines eingereichten Jobs zu verfolgen, ruft StreamFlow die Slurm-Kommandos `scontrol` und `squeue` mit der Kennung des Jobs auf. MOGON umfasst mehrere Cluster [21], und beim Einreichen eines Jobs wird neben der Nummer auch der ausführende Cluster zurückgemeldet. StreamFlow führte beide Angaben zu einer Kennung der Form `454783;mogonki` zusammen und gab diese unverändert an die Abfragen weiter, die daraufhin kein Ergebnis lieferten. Die Anpassung in der Datei `queue_manager.py` entfernt den angehängten Clusternamen, bevor die Kennung verwendet wird. Dabei ist zu berücksichtigen, dass es sich bei Version 0.2.0rc2 um eine Vorabversion handelt.

Gestartet wird ein Lauf mit einem einzelnen Aufruf von `streamflow run` auf dem Login-Knoten. Der dabei entstehende Vordergrundprozess übernimmt die Steuerung des Workflows, reicht die Schritte bei Slurm ein und überwacht deren Ausführung. Mit dem Ende dieses Prozesses ist der Lauf abgeschlossen, es bleibt kein Dienst zurück.

5.3.3 Merlin

Merlin steht auf MOGON nicht als Modul zur Verfügung und wurde deshalb in einer eigenen Conda-Umgebung installiert. Zum Einsatz kommt dieselbe Version 1.13.0 wie in der lokalen Untersuchungsumgebung (Abschnitt 4.1). Wie bereits dort beschrieben, benötigt Merlin zusätzlich einen Redis-Server. Da auf dem Cluster kein Docker zur Verfügung steht, wurde Redis über einen Apptainer-Container bereitgestellt, der sich mit den Rechten eines gewöhnlichen Nutzerkontos ausführen lässt. Die Verbindungsdaten zum Redis-Server stehen in der Datei `app.yaml` im Nutzerverzeichnis. Redis läuft auf dem Login-Knoten und ist von den Rechenknoten aus erreichbar, sodass die Worker die Aufgaben unabhängig von ihrem Ausführungsort beziehen können.

Die Workflow-Beschreibungen beider Muster wurden unverändert aus der lokalen Umsetzung übernommen. Hinzu kam für den Betrieb auf dem Cluster das Slurm-Jobskript `worker_job.sbatch`, mit dem die Worker bei Slurm eingereicht werden.

Listing 5.3 zeigt das verwendete Slurm-Jobskript. Der obere Teil fordert die Ressourcen für den Worker-Job bei Slurm an, darunter Partition, Konto, Rechenkerne, Hauptspeicher und Zeitlimit. Anders als bei Nextflow und StreamFlow gelten diese Angaben nicht für einzelne Workflow-Schritte, sondern für die gesamte Allokation, in der die Worker ausgeführt werden. Der untere Teil bereitet die Umgebung vor und startet den Celery-Worker.

```
1  #!/bin/bash
2  #SBATCH --account=ki-mawahpc
3  #SBATCH --job-name=merlin-worker-pipeline
4  #SBATCH --partition=ki-smallcpu
5  #SBATCH --nodes=1
6  #SBATCH --ntasks=1
7  #SBATCH --cpus-per-task=1
8  #SBATCH --mem=2G
9  #SBATCH --time=00:15:00
10 #SBATCH --output=worker_%j.out
11
12 module purge
13 module load lang/Anaconda3/2024.06-1
14 conda activate merlin2
15 cd ~/slurm_modus/merlin/pipeline
16
17 export LANG=C LC_ALL=C
18
19 celery -A merlin worker \
```



```

20  --concurrency 1 \
21  --prefetch-multiplier 1 \
22  -O fair \
23  -l INFO \
24  -n beobachtung_worker.%h \
25  -Q "[merlin]_merlin"

```

P. 5.3: Jobskript zum Start der Merlin-Worker im regulären Slurm-Betrieb auf MOGON

Ein Lauf wird auf dem Cluster mit zwei Befehlen gestartet. Der Aufruf von `merlin run` bereitet die Ausführung des Workflows vor und übergibt die auszuführenden Aufgaben an den Nachrichtenvermittler und kehrt danach zurück. Getrennt davon wird das Jobskript mit `sbatch` eingereicht, woraufhin die Worker die Aufgaben abarbeiten. Der Worker-Job endet nicht von selbst, sondern muss nach dem Lauf beendet werden. Auch der Redis-Server bleibt danach aktiv.

Bei der Einrichtung trat derselbe Fehler auf wie in der lokalen Umgebung. Das von Merlin vorausgesetzte Modul `pkg_resources` fehlt in aktuellen Versionen von `setuptools`, sodass ein Downgrade nötig war.

5.3.4 Pegasus

Pegasus setzt für die Ausführung der geplanten Workflows HTCondor voraus, an dessen Warteschlange die einzelnen Aufgaben übergeben werden. HTCondor stand auf MOGON nicht als Systemsoftware zur Verfügung. Für HPC-Systeme beschreibt die Dokumentation neben einer Installation durch die Administration, die allen Nutzenden zur Verfügung steht, auch eine nutzerseitige Installation, bei der HTCondor auf dem Login-Knoten betrieben wird [25].

Der dafür vorgesehene Bezugsweg war auf MOGON nicht nutzbar, da die Downloadadressen über den Proxy des Rechenzentrums nicht erreichbar waren. Eine Installation über Conda stellte lediglich Clientwerkzeuge und Python-Komponenten bereit, nicht jedoch die für den Betrieb erforderlichen HTCondor-Dienste. Damit stand für die geplante Ausführung keine funktionsfähige HTCondor-Installation zur Verfügung.

Pegasus konnte daher in die nachfolgenden Untersuchungen nicht einbezogen werden.

5.3.5 AiiDA

AiiDA steht auf MOGON nicht als Modul zur Verfügung und wurde deshalb in einer eigenen Conda-Umgebung installiert. Zum Einsatz kommt dieselbe Version 2.8.0 wie in der lokalen Untersuchungsumgebung (Abschnitt 4.1). Wie bereits dort beschrieben, verwendet die empfohlene Konfiguration von AiiDA PostgreSQL als Datenbank und RabbitMQ für die Kommunikation mit dem Daemon [1]. Beide Dienste lassen sich mit den Rechten eines gewöhnlichen Nutzerkontos betreiben, RabbitMQ wurde über dieselbe Conda-Umgebung installiert. Welche Datenbank ein Profil verwendet, wird bei der Einrichtung festgelegt. Auf die Auswirkungen der verschiedenen Konfigurationen geht Abschnitt 5.7 ein.

Die WorkChains beider Muster wurden aus der lokalen Umsetzung übernommen und um die Angaben ergänzt, mit denen die einzelnen Aufgaben bei Slurm eingereicht werden, darunter Partition, Konto und Zeitlimit. Profil, Rechner- und Code-Eintrag wurden wie in der lokalen Umgebung angelegt, mit zwei Änderungen im Rechner-Eintrag: Als Scheduler ist `core.slurm` statt `core.direct` eingetragen, wodurch AiiDA die Aufgaben als Slurm-Jobs einreicht, und das Arbeitsverzeichnis liegt auf dem gemeinsamen Dateisystem.

Listing 5.4 zeigt den Rechner-Eintrag, der für die Ausführung über Slurm angelegt wurde. Die Angaben zu Partition, Konto und Zeitlimit stehen nicht in diesem Eintrag, sondern in den WorkChains, bei den jeweiligen CalcJobs. Der Transport `core.local` legt fest, dass die Kommandos auf dem Login-Knoten ausgeführt werden, der Scheduler `core.slurm` bestimmt, dass AiiDA für jede Aufgabe ein Jobskript erzeugt und dieses bei Slurm einreicht. Das Arbeitsverzeichnis liegt auf dem gemeinsamen Dateisystem.

```
1  Label                                mogon-slurm
2  Transport type                       core.local
3  Scheduler type                      core.slurm
4  Work directory                      /fshpc/antahiri/slurm_modus/aiida/work
5  Default #procs/machine              1
6
7  # Ressourcenangaben je Aufgabe in der WorkChain
8  "queue_name": "ki-smallcpu",
9  "account": "ki-mawahpc",
10 "max_wallclock_seconds": 600,
11 "max_memory_kb": 1048576,
12 "resources": {
13     "num_machines": 1,
14     "num_mpiprocs_per_machine": 1,
```

P. 5.4: Rechner-Eintrag und Ressourcenangaben für die Ausführung über Slurm

Ein Lauf wird über ein kurzes Python-Skript eingereicht, das die Eingaben als Knoten anlegt und die WorkChain an den Daemon übergibt. Damit dessen Worker die WorkChain-Klassen laden können, muss das Verzeichnis, das diese Klassen enthält, im Suchpfad von Python liegen, bevor der Daemon gestartet wird, da er seine Umgebung nur beim Start übernimmt. Nach dem Einreichen kehrt das Skript zurück und der Daemon führt den Workflow im Hintergrund aus. Daemon und Nachrichtenvermittler laufen dabei unabhängig von der Sitzung weiter.

5.4 Ausführung im Slurm-Betrieb

Alle vier verbleibenden Systeme führen ihre Workflow-Schritte über Slurm aus, unterscheiden sich jedoch darin, wie sie den Ressourcenverwalter in die Ausführung einbinden. Bei Nextflow und StreamFlow reicht der Vordergrundprozess auf dem Login-Knoten für jeden Workflow-Schritt einen eigenen Slurm-Job ein, beobachtet deren Zustand durch wiederholte Abfragen und reicht abhängige Schritte erst ein, wenn ihre Vorgänger abgeschlossen sind. Ein Lauf mit neun Schritten erzeugt so neun Slurm-Jobs, die jeweils Warteschlange und Prolog durchlaufen, bevor die eigentliche Ausführung beginnt.

Bei AiiDA übernimmt der Daemon diese Aufgabe. Er erzeugt für jede Aufgabe ein Jobskript, reicht dieses bei Slurm ein und fragt den Zustand der Jobs zyklisch ab. Der Unterschied zu Nextflow und StreamFlow liegt nicht im Umgang mit Slurm, sondern darin, dass die Steuerung nicht an einen Vordergrundprozess gebunden ist. Das einreichende Skript kehrt unmittelbar zurück, während der Daemon den Workflow unabhängig von der Sitzung fortführt.

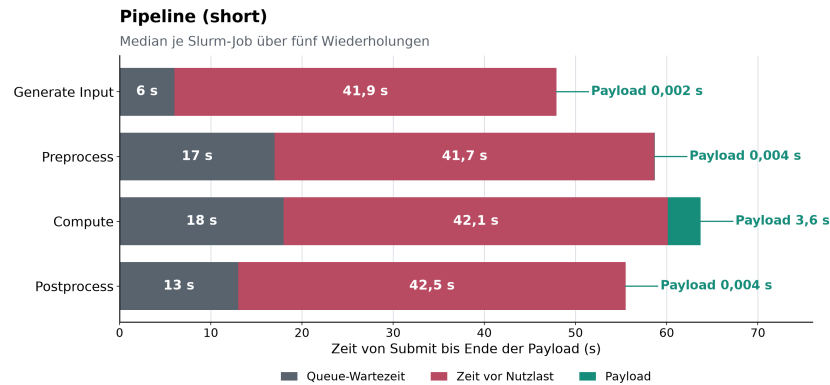
Merlin bindet Slurm auf andere Weise ein. Statt je Schritt einen Job einzureichen, wird ein einziger Job gestartet, dessen Allokation für die gesamte Dauer des Laufs bestehen bleibt. In dieser Allokation laufen die Worker, die sich die Aufgaben aus der Warteschlange holen und abarbeiten. Warteschlange und

Prolog fallen dadurch einmal je Lauf an statt einmal je Schritt und aufeinanderfolgende Schritte starten ohne erneuten Weg über den Ressourcenverwalter. Dieses Modell nutzt das Prinzip eines Pilot-Jobs: Ein Platzhalter-Job belegt die Ressourcen, und mehrere Aufgaben werden innerhalb seiner Laufzeit ausgeführt, wodurch die je Job anfallenden Scheduling-Anteile entfallen [6, S. 18].

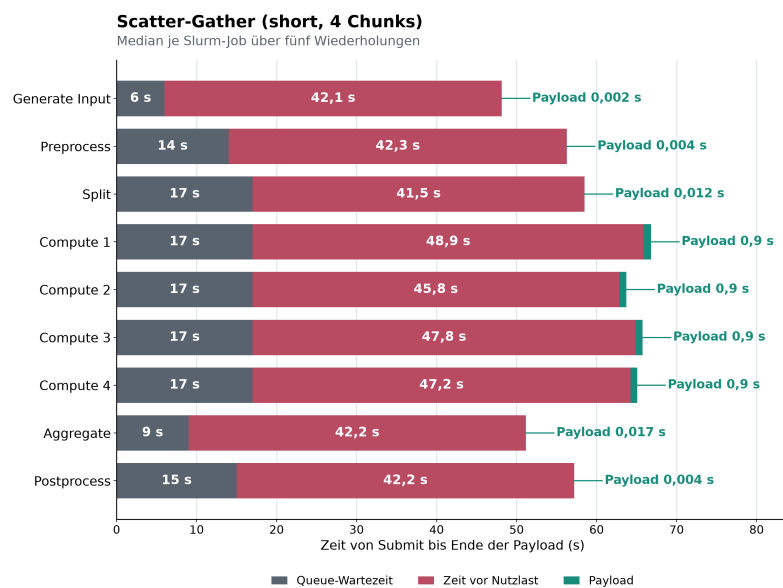
Aus diesen Ausführungsmodellen ergibt sich die Frage, wie sich die Laufzeit eines Workflows im regulären Clusterbetrieb zusammensetzt. Dazu wurden Beobachtungsläufe durchgeführt, zunächst mit Nextflow, bei dem beide Workflow-Muster in fünf Wiederholungen ausgeführt wurden. Für jeden Slurm-Job wurde die Zeit vom Einreichen bis zum Ende der Nutzlast in drei Anteile zerlegt: Die Wartezeit in der Warteschlange, die Zeit zwischen dem Start des Jobs und dem ersten Zeitstempel der Nutzlast, sowie die Laufzeit der Nutzlast selbst. Die ersten beiden Anteile wurden aus den Zeitstempeln des Slurm-Kommandos `sacct` bestimmt, die Laufzeit der Nutzlast aus den Zeitmessungen der Benchmark-Skripte.

Die Abbildungen 5.1a und 5.1b zeigen die Zerlegung für beide Muster. Die Laufzeit der Nutzlast liegt bei den meisten Schritten im Millisekundenbereich und ist in den Balken daher kaum sichtbar. Lediglich die Compute-Schritte erreichen mit 3,6 beziehungsweise 0,9 Sekunden nennenswerte Werte. Die Wartezeit in der Warteschlange schwankt zwischen 6 und 17 Sekunden. Der mittlere Anteil liegt dagegen durchgehend bei über 40 Sekunden je Job und stellt damit den mit Abstand größten Zeitanteil dar. Auffällig ist zudem, dass dieser Anteil auch zwischen gleichartigen Jobs schwankt: Die vier Compute-Jobs des Scatter-Gather-Musters wurden gleichzeitig eingereicht und warteten gleich lange, ihr mittlerer Anteil reicht dennoch von 45,8 bis 48,9 Sekunden.

Dieser mittlere Anteil wird hier bewusst nicht als Prolog bezeichnet, denn er umfasst zwei Dinge: die Arbeiten, die der Cluster vor dem Start des Jobskripts ausführt, und die Schritte, die das Workflow-Management-System innerhalb des Jobs erledigt, bevor die Nutzlast beginnt. Aus der Zerlegung allein ist nicht ersichtlich, wie sich der Anteil auf den Cluster und das Workflow-Management-System verteilt.



(a) Pipeline-Muster



(b) Scatter-Gather-Muster mit vier Chunks

Abb. 5.1.: Zerlegung der Zeit vom Einreichen bis zum Ende der Nutzlast je Slurm-Job, Median über fünf Wiederholungen mit Nextflow.

Um diese Aufteilung näher zu untersuchen, wurde der zuvor bestimmte mittlere Zeitanteil anschließend für alle vier Systeme gesondert untersucht. Dazu wurde je System analysiert, welche Schritte nach dem Start des Slurm-Jobs und vor dem ersten Zeitstempel der Nutzlast ausgeführt werden. Da sich Aufbau und Ablauf der Jobskripte zwischen den Systemen unterscheiden, erfolgte diese Untersuchung systemspezifisch.

Bei Nextflow und StreamFlow führt das erzeugte Jobskript vor der Nutzlast noch systemeigene Vorbereitungsschritte aus. Der Zeitraum zwischen der jeweils frühesten im Job verfügbaren Marke und dem ersten Zeitstempel der Nutzlast beträgt dort 55 beziehungsweise 102 Millisekunden. Bei AiiDA folgt im erzeugten Jobskript unmittelbar der Aufruf der Nutzlast. Bei Merlin fällt die Vorbereitung innerhalb des Slurm-Jobs nicht für jeden Workflow-Schritt einzeln an, sondern beim Start der Worker-Allokation.

Die Zerlegung zeigt damit, dass die innerhalb der Jobs vor der Nutzlast verbleibenden systemeigenen Vorbereitungsschritte in einem Zeitfenster im Millisekundenbereich liegen und gegenüber dem zuvor beobachteten Zeitanteil von über 40 Sekunden je Job klein ausfallen. Ihre Bestimmung erfordert zudem systemspezifische Untersuchungen der jeweiligen Jobabläufe und stellt daher keine einheitliche Vergleichsgröße dar. Aus diesem Grund wurde der Einfluss des Clusterbetriebs in der folgenden Messreihe durch eine dedizierte Ausführung experimentell ausgeschlossen.

5.5 Aufbau der dedizierten Messung

Die Messreihen wurden jeweils vollständig innerhalb einer einzelnen Slurm-Allokation ausgeführt. Dazu wurde ein Rechenknoten der Partition `ki-smallcpu` angefordert, auf dem die Systeme ihre Workflow-Schritte als lokale Prozesse starten, in derselben Ausführungsform wie in der lokalen Untersuchung in Kapitel 4. Warteschlange und Prolog fallen nur beim Start der jeweiligen Allokation an und liegen vollständig außerhalb der eigentlichen Messung. Die Ausführung von Benchmark-Messungen auf dedizierten Rechenknoten entspricht dem Vorgehen vergleichbarer Untersuchungen [7, S. 103].

Dieser Aufbau folgt unmittelbar aus dem Befund des vorangegangenen Abschnitts. Im regulären Clusterbetrieb entsteht der überwiegende Teil der Laufzeit durch Warteschlange und die Zeit vor der Nutzlast, während die

innerhalb der Jobs verbleibenden Schritte im Millisekundenbereich liegen. Ein Vergleich unter diesen Bedingungen würde deshalb überwiegend den Zustand des Clusters abbilden. Innerhalb einer bereits bestehenden Allokation entfallen diese Anteile für die einzelnen Workflow-Schritte, sodass Unterschiede zwischen den Systemen nicht mehr durch die je Schritt anfallenden Warteschlangen- und Prologzeiten überlagert werden.

Workflows, Benchmark-Skripte, Instrumentierung und Messgrößen sind dieselben wie in Kapitel 3 beschrieben. Auch die Referenzmessung ohne Workflow-Management-System wurde unverändert übernommen.

Die Allokation wurde zusätzlich exklusiv angefordert, sodass während der Messung keine Aufträge anderer Nutzer auf demselben Knoten liefen. Um den Einfluss dieser Einstellung zu prüfen, wurde dieselbe Messreihe auch ohne exklusive Reservierung ausgeführt. Abbildung 5.2 stellt die Differenzen beider Messreihen gegenüber. Die Boxen fassen die zwölf Konfigurationen je System zusammen, die eingezeichneten Punkte zeigen die einzelnen Konfigurationen. Die Unterschiede zwischen beiden Messreihen sind gering und nicht einheitlich gerichtet, bei einzelnen Systemen fielen die Werte auf dem geteilten Knoten sogar niedriger aus. Die exklusive Messreihe bildet daher die Grundlage der folgenden Auswertung. Die Messung auf dem geteilten Knoten dient ausschließlich dazu, den Einfluss einer geteilten Knotennutzung zu untersuchen.

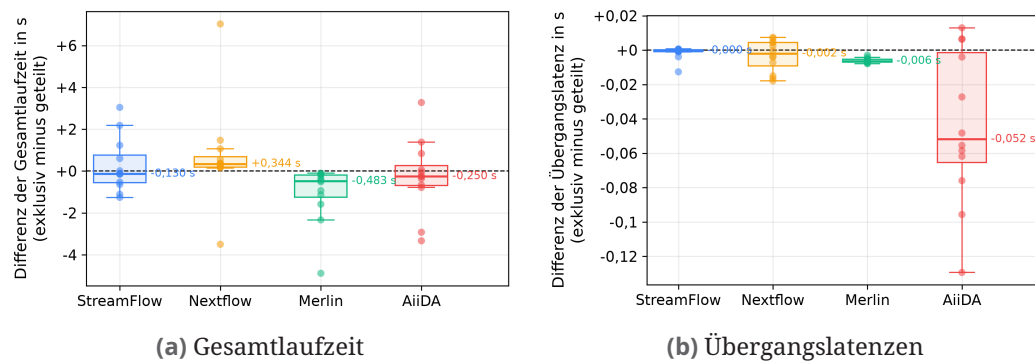


Abb. 5.2.: Differenzen der Mediane zwischen exklusivem und geteiltem Knoten (exklusiv minus geteilt). Werte nahe null kennzeichnen geringe Unterschiede zwischen beiden Messreihen.

5.6 Ergebnisse der dedizierten Messung

Berichtet werden dieselben vier Messgrößen wie auf der lokalen Ebene: die Gesamtlaufzeit, der Overhead gegenüber dem Referenzlauf sowie die beiden Koordinationsgrößen Übergangslatenz und Startspreizung. Pegasus ist aus den in Abschnitt 5.3.4 genannten Gründen nicht enthalten, sodass die folgenden Ergebnisse vier Systeme mit jeweils zwölf Konfigurationen umfassen.

Abbildung 5.3 zeigt die Gesamtlaufzeiten des Pipeline-Musters je Rechenlast. StreamFlow, Nextflow und Merlin liegen dabei in einem engen Band, bei der kurzen Rechenlast zwischen 4,9 und 6,7 Sekunden, bei der mittleren zwischen 44,2 und 45,1 Sekunden und bei der langen zwischen 188,4 und 192,1 Sekunden. AiiDA liegt bei allen drei Rechenlasten darüber, mit 12,3, 56,6 und 229,7 Sekunden. Mit wachsender Rechenlast werden die relativen Abstände zwischen den Systemen kleiner, da der Overhead gegenüber der zunehmenden Laufzeit der Rechenschritte an Gewicht verliert.

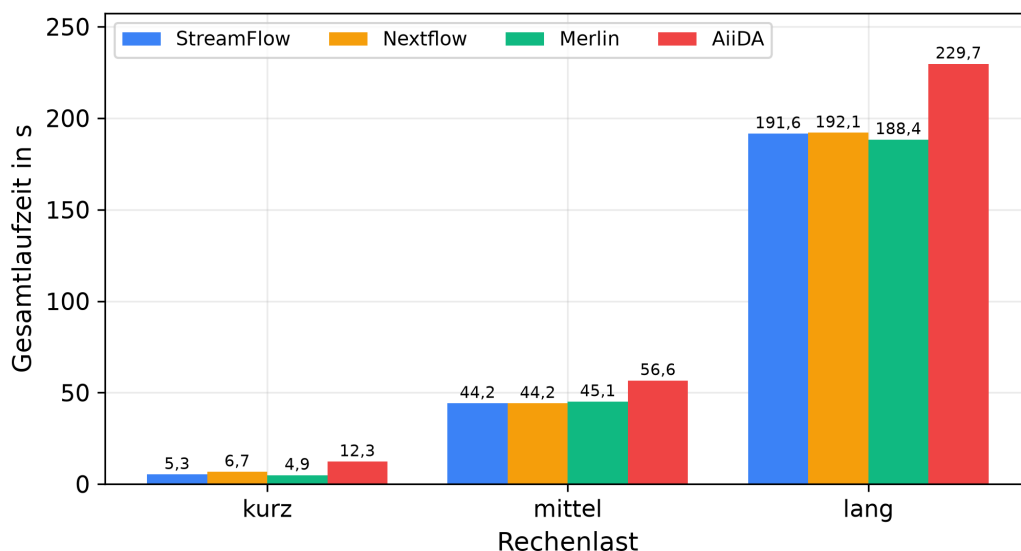


Abb. 5.3.: Median der Gesamtlaufzeit des Pipeline-Musters je Rechenlast auf dem HPC-System, lineare Skala

Abbildung 5.4 zeigt die Gesamtlaufzeiten des Scatter-Gather-Musters über die Zahl der Teilstücke. Bei der mittleren und der langen Rechenlast sinken die Laufzeiten mit jeder Verdopplung der Teilstückzahl deutlich, bei der langen Rechenlast etwa von 202,2 auf 53,6 Sekunden in StreamFlow und von 223,4 auf 70,4 Sekunden in AiiDA. Bei der kurzen Rechenlast fällt der Rückgang geringer aus. In StreamFlow, Nextflow und Merlin liegen die Laufzeiten bei einem

Teilstück zwischen 5,3 und 7,1 Sekunden und bei vier Teilstücken zwischen 3,0 und 4,8 Sekunden. AiiDA bleibt dort mit 16,4, 15,4 und 16,1 Sekunden über alle Teilstückzahlen nahezu unverändert.

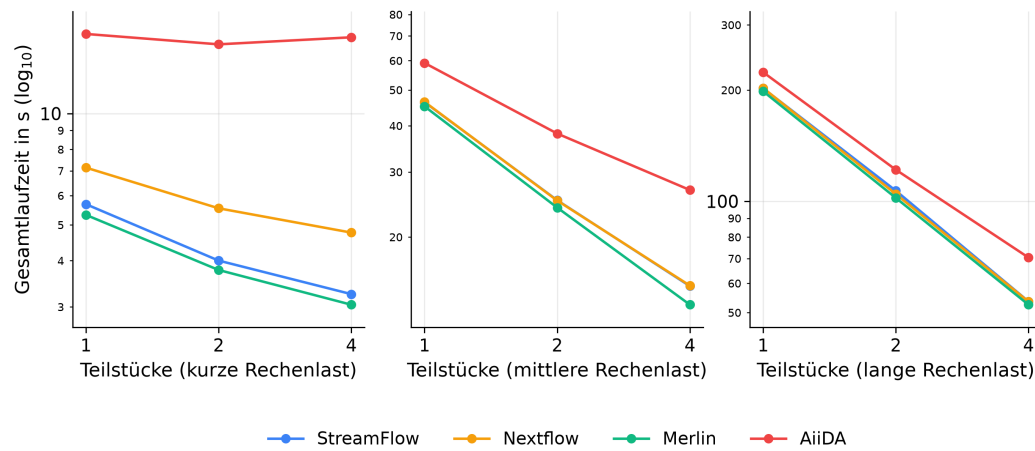


Abb. 5.4.: Median der Gesamtlaufzeit des Scatter-Gather-Musters über die Zahl der Teilstücke je Rechenlast auf dem HPC-System

Abbildung 5.5 stellt den Overhead aller zwölf Konfigurationen gegenüber. Es bilden sich zwei Gruppen. StreamFlow, Nextflow und Merlin liegen in zehn der zwölf Konfigurationen zwischen 1,6 und 4,1 Sekunden und wechseln untereinander die Reihenfolge. Höhere Werte treten bei den Scatter-Gather-Läufen der langen Rechenlast mit einem und zwei Teilstücken auf, dort liegen StreamFlow bei 5,4 und 4,6 Sekunden und Nextflow bei 5,0 und 6,9 Sekunden. AiiDA liegt mit 7,0 bis 14,9 Sekunden in allen zwölf Konfigurationen über den drei übrigen Systemen.

Ein systemübergreifend einheitlicher Zusammenhang zwischen Rechenlast und Overhead ist nicht erkennbar, obwohl sich die Laufzeit der Rechenschritte zwischen der kurzen und der langen Rechenlast um mehr als das Fünzigfache unterscheidet. Bei StreamFlow nimmt der Overhead im Scatter-Gather-Muster zwar mit der Rechenlast zu, bei den übrigen Systemen zeigt sich diese Entwicklung jedoch nicht durchgängig.

Bei AiiDA zeigt sich ein Zusammenhang mit dem Umfang des Workflows. Bei gleicher Rechenlast liegt der Overhead im Scatter-Gather-Muster durchgängig über dem des Pipeline-Musters. Zudem steigt er mit der Zahl der Teilstücke. Bei der kurzen Rechenlast nimmt er von 12,6 Sekunden bei einem Teilstück auf 14,9 Sekunden bei vier Teilstücken zu. Diese Tendenz war bereits auf der

lokalen Ebene zu beobachten, dort stieg der Overhead in derselben Konfiguration von 7,4 auf 9,4 Sekunden.

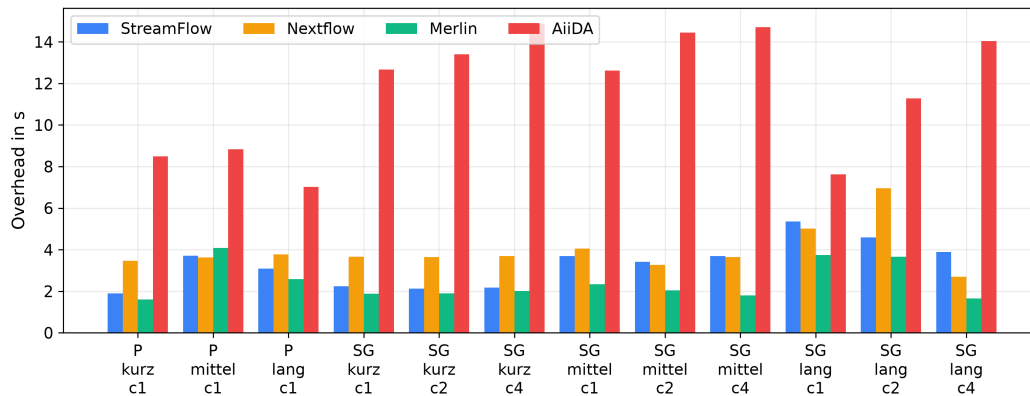


Abb. 5.5.: Overhead gegenüber dem Referenzlauf für alle zwölf Konfigurationen auf dem HPC-System

Abbildung 5.6 zeigt die Übergangslatenzen zwischen aufeinanderfolgenden Schritten. Der Median je Übergang liegt in StreamFlow bei 0,06 Sekunden, in Nextflow bei 0,11, in Merlin bei 0,14 und in AiiDA bei 1,95 Sekunden, AiiDA liegt damit etwa um den Faktor 35 über StreamFlow. Innerhalb eines Systems sind die Werte über die einzelnen Übergänge weitgehend gleich. Höhere Werte treten insbesondere an den Übergängen in die und aus der parallelen Verarbeitungsphase auf, am deutlichsten beim Übergang vom Verarbeiten zum Zusammenführen. Dieser liegt in StreamFlow bei 0,14 gegenüber sonst rund 0,06 Sekunden und in AiiDA bei 2,18 gegenüber rund 1,95 Sekunden. An diesen Übergängen werden mehrere parallele Instanzen gestartet beziehungsweise ihre Ergebnisse zusammengeführt. Die Übergangslatenz zeigt sich damit auch auf dem HPC-System als eine weitgehend systemspezifische Größe.

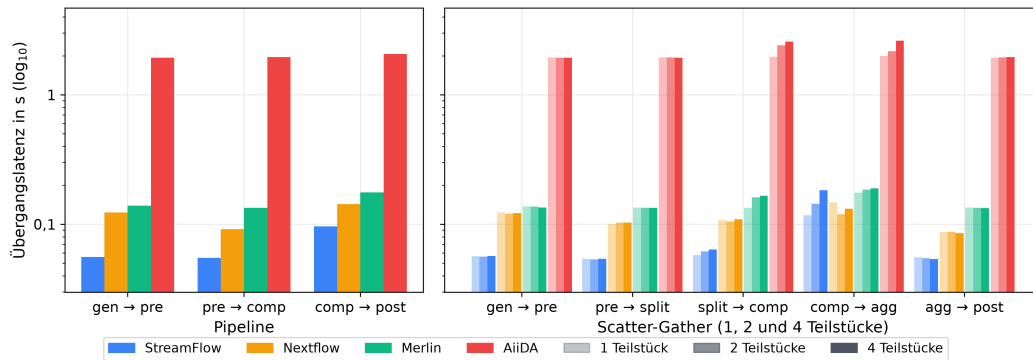


Abb. 5.6.: Median der Übergangslatenzen zwischen aufeinanderfolgenden Schritten auf dem HPC-System, logarithmische Skala

Abbildung 5.7 zeigt die Startspreizung der parallelen Verarbeitungsinstanzen, also den Abstand zwischen dem ersten und dem letzten Instanzstart. Bei zwei Teilstücken liegt sie je nach Rechenlast zwischen 3 und 4 Millisekunden in StreamFlow, zwischen 6 und 7 in Nextflow, zwischen 47 und 50 in Merlin und zwischen 24 und 32 Millisekunden in AiiDA. Bei vier Teilstücken liegen die entsprechenden Bereiche bei 18 bis 19, 21 bis 23, 99 bis 114 und 961 bis 1011 Millisekunden. Die Startspreizung unterscheidet sich zwischen den Rechenlasten damit nur geringfügig, zwischen den Teilstückzahlen dagegen deutlich. In allen vier Systemen wächst sie mit der Zahl der Teilstücke, am stärksten in AiiDA, wo sie sich von zwei auf vier Teilstücke mehr als verdreißigfach.

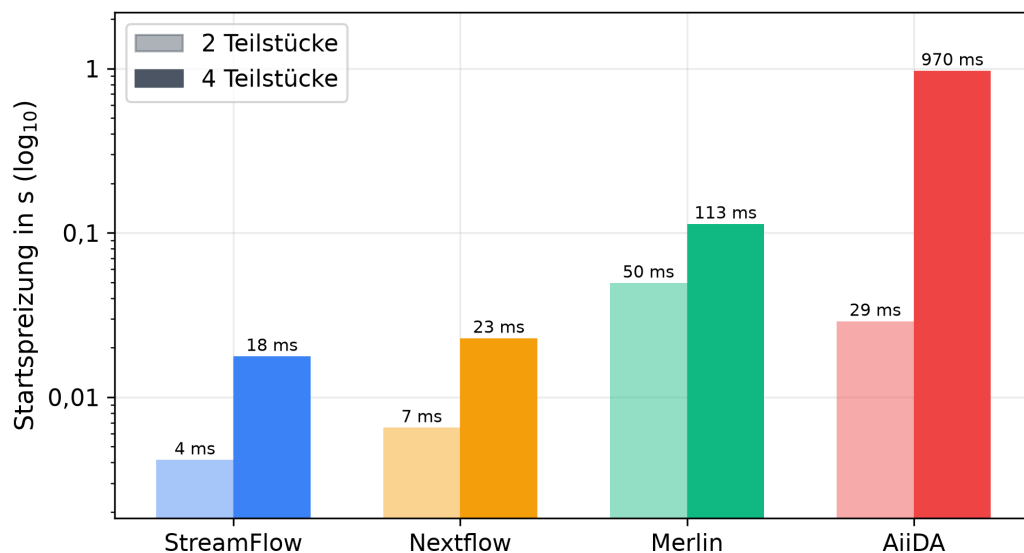


Abb. 5.7.: Median der Startspreizung der parallelen Verarbeitungsinstanzen auf dem HPC-System, logarithmische Skala

AiiDA weist beim Overhead und bei den Übergangslatenzen die höchsten Werte auf, bei der Startspreizung mit vier Teilstücken ebenfalls, bei zwei Teilstücken liegt dort Merlin darüber. Bereits im Pipeline-Muster summieren sich die drei Übergangslatenzen von jeweils rund zwei Sekunden auf etwa sechs Sekunden und erklären damit einen wesentlichen Teil des gemessenen Overheads von 7,0 bis 8,8 Sekunden. Im Scatter-Gather-Muster treten mit zunehmender Zahl der Teilstücke zusätzlich höhere Latenzen beim Start und beim Zusammenführen der parallelen Verarbeitungsinstanzen auf, besonders deutlich in der Startspreizung, die bei vier Teilstücken in AiiDA etwa eine Sekunde erreicht. Innerhalb der Gruppe aus StreamFlow, Nextflow und Merlin folgt die Höhe des Overheads dagegen nicht unmittelbar aus den Koordinationsgrößen. Merlin weist innerhalb dieser Gruppe die höchsten Übergangslatenzen und Startspreizungen auf, liegt beim Overhead aber in elf der zwölf Konfigurationen am niedrigsten.

5.7 Einfluss der AiiDA-Konfiguration

Der Abstand von AiiDA zu den übrigen Systemen ist in der dedizierten Messung größer als in der lokalen Untersuchung. Diesem Unterschied wird im Folgenden nachgegangen. Zugleich ist AiiDA das einzige der vier Systeme, das während der Ausführung auf eine Datenbank und ein Repository zugreift, und beide lagen im bisherigen Aufbau auf dem gemeinsamen Dateisystem /fshpc. Ergänzend wurde deshalb untersucht, welchen Einfluss der Ablageort dieser Datenhaltung auf die gemessenen Zeiten hat. Dazu wurde dieselbe SQLite-Konfiguration einmal auf /fshpc und einmal auf dem knotenlokalen Speicher des Rechenknotens ausgeführt. Die PostgreSQL-Konfiguration dient zusätzlich der Einordnung.

Abbildung 5.8 zeigt den Overhead der drei Konfigurationen über alle zwölf Messkonfigurationen. Die Verlagerung der Datenhaltung von /fshpc auf den knotenlokalen Speicher senkte den Overhead der SQLite-Konfiguration in allen zwölf Fällen von 12,5 bis 29,9 Sekunden auf 3,5 bis 11,6 Sekunden. Je nach Messkonfiguration entspricht dies einer Verringerung um den Faktor 2,2 bis 3,7. Die PostgreSQL-Konfiguration liegt mit 8,4 bis 15,4 Sekunden bei gleichem Ablageort /fshpc zwischen den beiden SQLite-Reihen.

Auch die Koordinationsgrößen zeigen dieses Muster. Bei vier Teilstücken beträgt die Startspreizung der SQLite-Konfiguration 2,99 Sekunden auf /fshpc

und 0,50 Sekunden auf dem knotenlokalen Speicher. Die Übergangslatenz vom Verarbeiten zum Zusammenführen sinkt entsprechend von 4,96 auf 1,44 Sekunden.

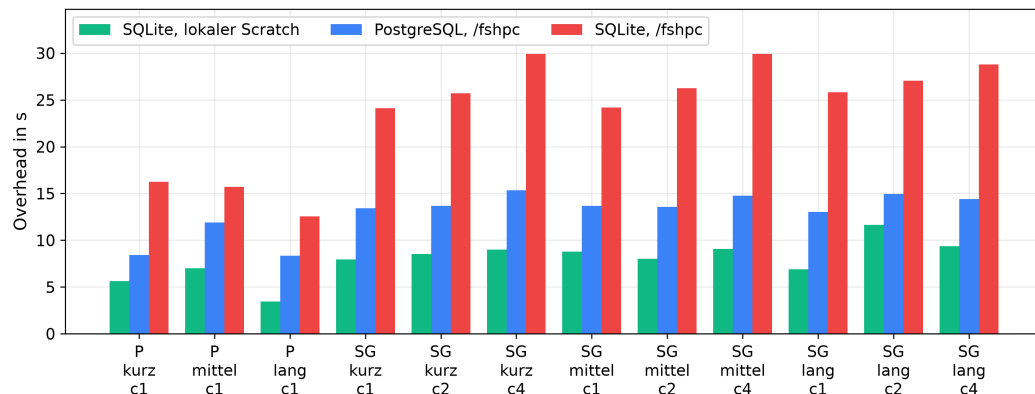


Abb. 5.8.: Overhead der drei AiiDA-Konfigurationen für alle zwölf Konfigurationen auf einem geteilten Knoten

Für die untersuchte SQLite-Konfiguration hat der Ablageort der Datenhaltung damit erheblichen Einfluss auf die gemessenen Zeiten. Da zwischen den beiden Läufen ausschließlich der Ablageort verändert wurde, ist der Unterschied auf die Verlagerung der Datenhaltung zurückzuführen. Zugleich bleibt der Overhead von AiiDA auch in der schnellsten der drei untersuchten Konfigurationen mit 3,5 bis 11,6 Sekunden in nahezu allen Messkonfigurationen über dem der drei übrigen Systeme. Der in Abschnitt 5.6 beobachtete Abstand bleibt damit auch bei veränderter AiiDA-Konfiguration im Wesentlichen bestehen, während die absolute Höhe des Overheads deutlich von der Konfiguration beeinflusst wird.

5.8 Zwischenfazit

Vier der fünf untersuchten Systeme ließen sich auf MOGON unter den Rechten eines gewöhnlichen Nutzerkontos bereitstellen und betreiben, Pegasus konnte nicht in die Untersuchung einbezogen werden. Der Aufwand für Bereitstellung und Betrieb unterschied sich dabei. Nextflow stand als Modul zur Verfügung und benötigte keine zusätzlichen Dienste. Bei StreamFlow, Merlin und AiiDA waren dagegen eigene Installationen und weitere system-spezifische Einrichtungsschritte erforderlich. Die Workflow-Beschreibungen von Nextflow, StreamFlow und Merlin konnten unverändert übernommen

werden, angepasst wurden dort nur die getrennten Angaben zur Ausführungsumgebung. Bei AiiDA wurden die Ressourcenangaben in den WorkChains ergänzt. Unterschiede zeigten sich zudem in der Einbindung von Slurm: Nextflow, StreamFlow und AiiDA reichen die Workflow-Schritte einzeln ein, während Merlin die Schritte innerhalb einer gemeinsamen Worker-Allokation ausführt.

Die Untersuchung des regulären Slurm-Betriebs zeigte, dass die Wartezeit in der Warteschlange und die Zeit vor Beginn der Nutzlast die Laufzeit der einzelnen Jobs deutlich stärker bestimmen als die Anteile der Workflow-Management-Systeme. Für den Vergleich wurden diese Einflüsse deshalb durch die Ausführung innerhalb einer bereits bestehenden Allokation ausgeschlossen. In der dedizierten Messung liegen StreamFlow, Nextflow und Merlin beim Overhead erneut überwiegend in einem engen Bereich, während AiiDA höhere Overhead-Werte und Übergangslatenzen aufweist. Der Abstand von AiiDA steht dabei wie auf der lokalen Ebene vor allem mit den Übergängen zwischen den Workflow-Schritten und dem Start paralleler Verarbeitungsinstanzen in Zusammenhang. Die ergänzende Untersuchung von AiiDA zeigt darüber hinaus, dass die Höhe dieser Werte deutlich von der gewählten AiiDA-Konfiguration und insbesondere vom Ablageort der Datenhaltung beeinflusst werden kann.

Die HPC-Untersuchung zeigt damit einerseits, dass mehrere der auf der lokalen Ebene beobachteten Unterschiede auch unter den Bedingungen des HPC-Systems bestehen bleiben, andererseits treten zusätzliche Aspekte hervor, die auf der lokalen Ebene keine Rolle spielten. Dazu gehören die Bereitstellung unter Nutzerrechten, die Einbindung des Ressourcenverwalters und die clusterbedingten Zeitanteile. Diese Befunde bilden zusammen mit den Ergebnissen der lokalen Untersuchung die Grundlage für die Diskussion, in der die Forschungsfragen beantwortet werden.

Diskussion

Dieses Kapitel ordnet die Ergebnisse beider Untersuchungsebenen ein. Zunächst werden die vier Forschungsfragen beantwortet, anschließend werden die Grenzen der Arbeit dargelegt.

6.1 Beantwortung der Forschungsfragen

F1: Modellierung der beiden Workflow-Muster. Sowohl das Pipeline- als auch das Scatter-Gather-Muster ließen sich in allen fünf Systemen mit denselben Benchmark-Skripten umsetzen. Für die beiden untersuchten Muster stellt die grundsätzliche Abbildbarkeit damit kein wesentliches Unterscheidungsmerkmal der Systeme dar. Unterschiede zeigen sich vielmehr in der Art der Workflow-Beschreibung und darin, wie die Beziehungen zwischen den einzelnen Schritten ausgedrückt werden. Die beobachteten Beschreibungsformen reichen von systemeigenen Formaten über den Standard CWL bis zu Programmierschnittstellen und lassen sich damit in die von Suter et al. unterschiedenen Kategorien einordnen [39, S. 8].

Beim Scatter-Gather-Muster treten diese Unterschiede deutlicher hervor. Neben den Abhängigkeiten müssen dort die Aufteilung in parallele Verarbeitungsinstanzen und deren anschließende Zusammenführung ausgedrückt werden. Auch eine Variation der Teilstückzahl erfordert je nach System unterschiedliche Änderungen an der Workflow-Beschreibung. Die Umsetzung beider Muster zeigt damit, dass sich die betrachteten Systeme bei diesen beiden Mustern weniger durch ihre Ausdrucksfähigkeit als durch die Art der Workflow-Beschreibung und den Aufwand bei Änderungen unterscheiden.

F2: Laufzeitverhalten der Workflow-Management-Systeme. Auf beiden Untersuchungsebenen zeigt sich ein ähnliches Bild beim Overhead. StreamFlow, Nextflow und Merlin liegen in einem gemeinsamen Bereich, AiiDA darüber, und Pegasus in der lokalen Untersuchung mit deutlichem Abstand an der Spitze. Dass diese Anordnung auf einem einzelnen Rechner und auf einem

dedizierten Rechenknoten gleichermaßen auftritt, spricht dagegen, sie allein der jeweiligen Ausführungsumgebung zuzuschreiben. Ein proportionaler Zusammenhang zwischen Rechenlast und Overhead ist in keinem der Systeme erkennbar, weshalb der Overhead mit wachsender Rechenlast gegenüber der Rechenzeit an Gewicht verliert. Dies gehört zu den Gründen, aus denen die Aufteilung auf mehrere Teilstücke beim Scatter-Gather-Muster erst ab der mittleren Rechenlast spürbare Gewinne bringt, während bei der kurzen Rechenlast der Overhead einen größeren Teil der Gesamtlaufzeit ausmacht.

Die beiden Koordinationsgrößen machen sichtbar, an welchen Stellen zusätzliche Zeit anfällt. Besonders deutlich zeigt sich dies bei Pegasus und AiiDA. Bei Pegasus fällt die mediane Übergangslatenz von rund 20 Sekunden mit dem höchsten gemessenen Overhead zusammen. Bei AiiDA fallen die Übergangslatenzen mit ein bis zwei Sekunden geringer aus, liegen aber auf beiden Ebenen deutlich über denen der drei übrigen Systeme und machen bereits im Pipeline-Muster einen wesentlichen Teil des gemessenen Overheads aus. Die Ergebnisse sprechen damit dafür, dass die höheren Overhead-Werte von Pegasus und AiiDA zu einem erheblichen Teil mit der Koordination und Übergabe zwischen aufeinanderfolgenden Workflow-Schritten zusammenhängen.

Die Startspreizung ist davon getrennt zu betrachten. Sie beschreibt, wie schnell ein System die parallelen Instanzen startet. Merlin zeigt das deutlich: Lokal liegt seine Startspreizung bei vier Teilstücken mit rund 0,73 Sekunden in derselben Größenordnung wie bei AiiDA, während sein Overhead deutlich niedriger bleibt. Eine größere Startspreizung führt damit nicht zwangsläufig zu einem höheren Overhead.

F3: Betrieb auf dem HPC-System. Vier der fünf untersuchten Systeme ließen sich auf MOGON unter den Rechten eines gewöhnlichen Nutzerkontos bereitstellen und betreiben, Pegasus konnte nicht einbezogen werden. Dabei unterscheiden sich nicht nur die erforderlichen Einrichtungsschritte, sondern auch die Betriebsmodelle unter Slurm. Nextflow, StreamFlow und AiiDA reichen die Workflow-Schritte einzeln als Slurm-Jobs ein. Die Steuerung erfolgt bei Nextflow und StreamFlow durch einen Vordergrundprozess und bei AiiDA durch einen dauerhaft laufenden Daemon. Merlin verwendet dagegen eine gemeinsame Worker-Allokation, innerhalb derer die einzelnen Workflow-Schritte abgearbeitet werden. Damit fallen Warteschlange und Startphase eines Slurm-Jobs bei den ersten drei Systemen für jeden eingereichten Schritt erneut an, bei Merlin dagegen nur einmal beim Start der Allokation.

Für die Laufzeitzusammensetzung zeigt der reguläre Slurm-Betrieb, dass die Ausführungsumgebung einen wesentlich größeren Einfluss haben kann als das Workflow-Management-System selbst. In den Beobachtungsläufen schwankte die Wartezeit in der Slurm-Warteschlange zwischen 6 und 17 Sekunden je Job. Hinzu kamen vor Beginn der eigentlichen Nutzlast durchgehend mehr als 40 Sekunden. Die anschließende systemspezifische Untersuchung zeigte dagegen, dass die innerhalb dieses Zeitraums von den Workflow-Management-Systemen mit einzelnen Slurm-Jobs ausgeführten Vorbereitungsschritte im Millisekundenbereich liegen. Eine Messung im regulären Clusterbetrieb würde damit überwiegend Wartezeit und die Zeit vor Beginn der Nutzlast erfassen und die Unterschiede zwischen den Systemen überlagern. Für den eigentlichen Systemvergleich wurde deshalb eine bereits bestehende Slurm-Allokation verwendet, innerhalb derer Warteschlange und Jobstart nicht erneut für jeden Workflow-Schritt anfielen.

Für den HPC-Betrieb ist damit auch die Einbindung in die Ausführungsumgebung von Bedeutung. Die Art der Slurm-Einbindung beeinflusst, wie häufig die vom Cluster verursachten Zeitanteile innerhalb eines Workflows anfallen. Bei AiiDA kam mit dem Ablageort von Datenbank und Repository ein weiterer Einflussfaktor hinzu, dessen Veränderung den gemessenen Overhead deutlich beeinflusste. Die gemessenen Laufzeiten hängen auf dem HPC-System daher nicht nur vom Workflow-Management-System selbst, sondern auch von den Bedingungen der Ausführungsumgebung ab.

F4: Eignung für Workflow-Arten und Anwendungsfälle. Aus den Ergebnissen ergibt sich keine allgemeine Rangfolge der Systeme. Welches System geeignet erscheint, hängt vielmehr davon ab, welche Eigenschaften für den jeweiligen Anwendungsfall im Vordergrund stehen.

Nextflow beschreibt Workflows datenflussorientiert und stellt die schnelle Entwicklung, parallele Ausführung und Reproduzierbarkeit in den Mittelpunkt [11]. Beide untersuchten Muster ließen sich kompakt über Prozesse und Channels abbilden, wobei auch Änderungen der Teilstückzahl keine Anpassung der Workflow-Struktur erforderten. Zusammen mit dem geringen Overhead, den niedrigen Koordinationszeiten und der direkten Slurm-Anbindung ergibt sich damit ein günstiges Profil für datenflussorientierte Workflows mit sequenziellen und parallelen Verarbeitungsschritten. Im regulären Clusterbetrieb ist allerdings zu berücksichtigen, dass jeder Prozess als

eigener Slurm-Job eingereicht wird und damit die Warte- und Startzeiten des Ressourcenverwalters je Schritt erneut entstehen können.

StreamFlow ist auf Workflows in hybriden Ausführungsumgebungen ausgerichtet und trennt die Workflow-Beschreibung von der Konfiguration der Ausführungsumgebung [8]. Diese Trennung zeigte sich beim Wechsel auf MOGON: Die in CWL formulierte Workflow-Beschreibung blieb unverändert, während die Ausführungskonfiguration angepasst wurde. Zusammen mit dem geringen Overhead und den niedrigen gemessenen Koordinationszeiten bietet sich StreamFlow damit insbesondere an, wenn eine standardisierte Workflow-Beschreibung und die Trennung von Workflow und Ausführungsumgebung im Vordergrund stehen. Einschränkend war die Bereitstellung der untersuchten Vorabversion aufwendiger und erforderte Anpassungen für die Slurm-Ausführung.

Merlin ist für große Ensembles von Rechenaufgaben auf HPC-Systemen ausgelegt und verteilt Aufgaben über ein Erzeuger-Verbraucher-Modell an Worker [28]. Dieses Modell ließ sich beim Scatter-Gather-Muster für die parallelen Verarbeitungsinstanzen nutzen. Auf MOGON wurden die Workflow-Schritte innerhalb einer gemeinsamen Worker-Allokation ausgeführt. Dadurch fielen die Warte- und Startzeiten des Ressourcenverwalters nicht für jeden Schritt erneut an. Merlin bietet sich damit insbesondere an, wenn viele Rechenaufgaben innerhalb einer einmal angeforderten Allokation abgearbeitet werden sollen, ohne dass jede Aufgabe erneut die Warteschlange durchläuft. Die von Merlin angestrebte Skalierung auf sehr große Aufgabenzahlen wurde in dieser Arbeit jedoch nicht untersucht, da das Scatter-Gather-Muster höchstens vier parallele Verarbeitungsinstanzen umfasste.

AiiDA stellt die dauerhafte Nachvollziehbarkeit wissenschaftlicher Abläufe in den Mittelpunkt und hält Prozesse und Daten während der Ausführung in einem Provenance-Graphen fest [15]. Dieser Schwerpunkt geht mit einem umfangreicheren Betriebsmodell sowie in der Untersuchung mit höheren Koordinationszeiten und höherem Overhead als bei Nextflow, StreamFlow und Merlin einher. Zudem zeigte die Konfigurationsstudie einen deutlichen Einfluss des Ablageorts der Datenhaltung auf die gemessenen Zeiten. Damit stellt AiiDA insbesondere dann eine relevante Option dar, wenn die dauerhafte Nachvollziehbarkeit von Prozessen und Daten für den Anwendungsfall stärker gewichtet wird als der in dieser Untersuchung beobachtete höhere Overhead und der zusätzliche Koordinations- und Betriebsaufwand, wobei

die absolute Höhe der gemessenen Zeiten von der Konfiguration der Datenhaltung abhängt.

Pegasus richtet sich auf großskalige, mehrstufige Simulations- und Analyse-Abläufe auf verteilten Rechenressourcen [10, S. 17] und übersetzt eine abstrakte Workflow-Beschreibung in einen ausführbaren Workflow [10, S. 18]. In der lokalen Untersuchung wies Pegasus allerdings den mit Abstand höchsten Overhead auf, der bei kurzen Rechenschritten besonders stark ins Gewicht fiel. Die Dokumentation beschreibt den Aufwand beim Einplanen kurzer Jobs selbst als Problemfall und sieht dafür das Bündeln mehrerer Aufgaben vor [27]. Pegasus erscheint damit für die hier untersuchten feingranularen Workflows weniger günstig, wenn ein geringer Laufzeit-Overhead im Vordergrund steht. Die Eignung für das untersuchte HPC-System konnte nicht bewertet werden, da sich Pegasus dort nicht bereitstellen ließ.

Systemübergreifend zeigt sich, dass die Bedeutung des Laufzeit-Overheads von der Größe der einzelnen Rechenaufgaben abhängt. Bei kurzen Rechenschritten kann die zusätzlich durch das Workflow-Management-System beanspruchte Zeit einen erheblichen Anteil der Gesamtlaufzeit ausmachen. Mit zunehmender Rechenlast verliert dieser Anteil dagegen an Gewicht. Für die Systemwahl können dann andere Eigenschaften stärker in den Vordergrund rücken, etwa die Art der Workflow-Beschreibung oder die Nachvollziehbarkeit von Prozessen und Daten. Auf einem HPC-System kommt hinzu, wie das jeweilige System den Ressourcenverwalter einbindet. Im regulären Slurm-Betrieb fielen bei einzeln eingereichten Jobs Warte- und Startzeiten für jeden Workflow-Schritt erneut an, während Merlin die Schritte innerhalb einer gemeinsamen Worker-Allokation ausführte. Welche dieser Eigenschaften für die Eignung eines Systems entscheidend sind, hängt damit von den Anforderungen des jeweiligen Anwendungsfalls ab. Die Grenzen dieser Aussagen betrachtet Abschnitt 6.2.

6.2 Grenzen der Arbeit

Die Untersuchung betrachtet mit dem Pipeline- und dem Scatter-Gather-Muster zwei grundlegende Workflow-Strukturen. Die Ergebnisse zeigen, wie die fünf Systeme diese Muster modellieren und ausführen. Komplexere Strukturen wie größere Abhängigkeitsgraphen oder verschachtelte Workflows

wurden nicht untersucht. Aussagen zur Modellierung und Eignung gelten daher für die beiden betrachteten Muster.

Die Rechenschritte selbst bestehen aus bewusst einfach gehaltenen Python-Skripten, damit alle Systeme dieselbe kontrollierte Rechenlogik ausführen. Die verwendeten Rechenaufgaben bilden jedoch keine reale wissenschaftliche Anwendung mit großen Datenmengen oder intensiven Dateizugriffen ab. Inwieweit sich die beobachteten Unterschiede unter solchen Anwendungslasten verändern, wurde daher nicht untersucht.

Das Scatter-Gather-Muster wurde mit einem, zwei und vier Teilstücken ausgeführt. Die Ergebnisse beschreiben damit das Verhalten bei wenigen parallelen Verarbeitungsinstanzen. Ob sich die beobachteten Unterschiede bei deutlich größeren Aufgabenzahlen in gleicher Weise fortsetzen, lässt sich daraus nicht ableiten. Dies betrifft insbesondere Merlin, dessen Entwurf ausdrücklich auf große Ensembles zielt.

Die Messungen fanden auf einem einzelnen Rechner und auf einem einzelnen HPC-System statt. Die absoluten Laufzeiten hängen damit von diesen beiden Umgebungen ab und lassen sich nicht unmittelbar auf andere Rechner oder HPC-Umgebungen übertragen. Die innerhalb einer Untersuchungsebene beobachteten Unterschiede beziehen sich somit auf die jeweils verwendete Ausführungsumgebung. Dies gilt auch für die systemspezifischen Befunde auf MOGON, beispielsweise die bei StreamFlow erforderliche Anpassung oder den bei AiiDA beobachteten Einfluss des Ablageorts der Datenhaltung. Ob diese Befunde unter anderen HPC-Umgebungen in gleicher Weise auftreten, wurde nicht untersucht.

Eine weitere Eingrenzung ergibt sich aus der Definition des Overheads. Er wurde als zusätzlicher Zeitbedarf gegenüber einer Referenzausführung derselben Rechenlogik ohne Workflow-Management-System bestimmt. Die Messgröße gibt somit an, wie viel Zeit zusätzlich anfällt, nicht, an welcher Stelle im System sie entsteht. Die Übergangslatenzen und die Startspreizung grenzen einzelne Anteile davon näher ein, erfassen jedoch nicht sämtliche internen Vorgänge eines Systems.

Jede Konfiguration wurde bei kurzer und mittlerer Rechenlast fünfmal, bei langer Rechenlast dreimal ausgeführt und über den Median ausgewertet. Die Verwendung des Medians reduziert den Einfluss einzelner abweichender Läufe. Mit mehr Wiederholungen ließe sich zusätzlich angeben, wie stark die Werte zwischen den Läufen einer Konfiguration streuen.

Pegasus konnte auf MOGON nicht ausgeführt werden, da es sich nicht bereitstellen ließ. Für dieses System liegen daher keine Laufzeitdaten auf der HPC-Ebene vor, die quantitativen Aussagen zu Pegasus stützen sich allein auf die lokale Untersuchung. Die fehlende Bereitstellbarkeit von Pegasus unter den gegebenen Bedingungen stellt dabei selbst ein Ergebnis der Untersuchung dar.

Fazit und Ausblick

7.1 Fazit

Diese Arbeit hat fünf Workflow-Management-Systeme anhand zweier Workflow-Muster und derselben Rechenlogik verglichen, zunächst unter kontrollierten Bedingungen auf einem einzelnen Rechner und anschließend auf dem HPC-System MOGON. Untersucht wurden die Modellierung, das Laufzeitverhalten anhand von Makespan, Overhead und Koordinationsgrößen sowie der Betrieb unter Slurm.

Beide Muster ließen sich in allen fünf Systemen umsetzen. Die wesentlichen Unterschiede liegen daher nicht in der Abbildbarkeit, sondern in der Art der Workflow-Beschreibung, im Koordinationsverhalten und im Betriebsmodell. Beim Overhead bildeten StreamFlow, Nextflow und Merlin auf beiden Ebenen eine Gruppe mit niedrigen Werten, AiiDA lag darüber und Pegasus in der lokalen Untersuchung mit großem Abstand an der Spitze. Mit zunehmender Rechenlast verliert der Overhead gegenüber der eigentlichen Rechenzeit an Bedeutung.

Auf dem HPC-System zeigte sich ein starker Einfluss der Ausführungsumgebung auf die gemessenen Laufzeiten. Die vom Cluster verursachten Zeitanteile können die Unterschiede zwischen den Systemen überlagern, und die Systeme binden den Ressourcenverwalter unterschiedlich ein, von einzelnen Jobs je Schritt bis zur gemeinsamen Worker-Allokation. Für den Systemvergleich in dieser Arbeit wurde deshalb innerhalb einer bestehenden Allokation gemessen.

Für die Eignungsfrage folgt daraus, dass sie sich nicht an der Abbildbarkeit der Muster festmachen lässt, denn diese war in allen fünf Systemen gegeben. Unterschiede zeigten sich dagegen im Laufzeitverhalten und im Betrieb. Mit abnehmender Bedeutung des Overheads können für die Wahl daher andere, aus der Ausrichtung der Systeme hervorgehende Eigenschaften stärker ins Gewicht fallen. Dazu zählen beispielsweise die Unterstützung hybrider Ausführungsumgebungen bei StreamFlow, die Ausrichtung von Merlin auf große

Ensembles von Rechenaufgaben oder die dauerhafte Nachvollziehbarkeit bei AiiDA. Eine allgemeine Rangfolge der Systeme ergibt sich daraus nicht. Welches System geeignet erscheint, hängt damit auch davon ab, welche dieser Eigenschaften für den jeweiligen Anwendungsfall im Vordergrund stehen.

7.2 Ausblick

An den Grenzen der Arbeit setzen mögliche weiterführende Arbeiten an. Der Vergleich ließe sich zunächst auf komplexere Workflow-Strukturen ausdehnen und mit Rechenlasten wiederholen, die realen wissenschaftlichen Anwendungen näherkommen, etwa mit großen Datenmengen und intensiven Dateizugriffen. So ließe sich prüfen, ob die hier beobachteten Unterschiede zwischen den Systemen auch unter solchen Bedingungen fortbestehen.

Ein zweiter Ansatzpunkt ist die Skalierung. Die Untersuchung umfasste höchstens vier parallele Verarbeitungsinstanzen. Mit deutlich mehr Aufgaben ließe sich prüfen, wie sich die Systeme bei großen Aufgabenzahlen verhalten, insbesondere Merlin, dessen Entwurf genau darauf zielt. Messungen auf weiteren HPC-Systemen würden zudem zeigen, welche der auf MOGON beobachteten Betriebs- und Laufzeiteigenschaften an die konkrete Ausführungsumgebung gebunden sind.

Offen bleibt schließlich der HPC-Vergleich für Pegasus. Lässt sich Pegasus mit der benötigten HTCondor-Umgebung auf einem Cluster betreiben, kann die quantitative Untersuchung auf alle fünf Systeme ausgeweitet werden.

Literatur

- [3]Rosa M. Badia, Eduard Ayguade und Jesus Labarta. “Workflows for Science: a Challenge when Facing the Convergence of HPC and Big Data”. In: *Supercomputing Frontiers and Innovations* 4.1 (2017), S. 27–47 (zitiert auf den Seiten 1, 5, 6).
- [4]Shishir Bharathi, Ann Chervenak, Ewa Deelman et al. “Characterization of Scientific Workflows”. In: *2008 Third Workshop on Workflows in Support of Large-Scale Science*. 2008, S. 1–10 (zitiert auf den Seiten 6, 19).
- [5]Peter Buneman, Sanjeev Khanna und Wang-Chiew Tan. “Why and Where: A Characterization of Data Provenance”. In: *Database Theory — ICDT 2001*. 2001, S. 316–330 (zitiert auf Seite 10).
- [6]Weiwei Chen und Ewa Deelman. “Workflow overhead analysis and optimizations”. In: *Proceedings of the 6th Workshop on Workflows in Support of Large-Scale Science*. 2011, S. 11–20 (zitiert auf den Seiten 18, 66).
- [7]Tainã Coleman, Henri Casanova, Loïc Pottier et al. “WfBench: Automated Generation of Scientific Workflow Benchmarks”. In: *2022 IEEE/ACM International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS)*. 2022, S. 100–111 (zitiert auf den Seiten 20, 68).
- [8]Iacopo Colonnelli, Barbara Cantalupo, Ivan Merelli und Marco Aldinucci. “StreamFlow: cross-breeding cloud with HPC”. In: *IEEE Transactions on Emerging Topics in Computing* 9.4 (2021), S. 1723–1737 (zitiert auf den Seiten 9, 14, 16, 34, 80).
- [9]Susan B. Davidson und Juliana Freire. “Provenance and Scientific Workflows: Challenges and Opportunities”. In: *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*. 2008, S. 1345–1350 (zitiert auf Seite 16).
- [10]Ewa Deelman, Karan Vahi, Gideon Juve et al. “Pegasus, a workflow management system for science automation”. In: *Future Generation Computer Systems* 46 (2015), S. 17–35 (zitiert auf den Seiten 10, 14, 16, 36, 37, 81).
- [11]Paolo Di Tommaso, Maria Chatzou, Evan W. Floden et al. “Nextflow enables reproducible computational workflows”. In: *Nature Biotechnology* 35.4 (2017), S. 316–319 (zitiert auf den Seiten 9, 14, 31, 79).
- [12]Philipp Diercks, Dennis Gläser, Ontje Lünsdorf et al. “Evaluation of tools for describing, reproducing and reusing scientific workflows”. In: *ing.grid* 1.1 (2023) (zitiert auf Seite 11).

- [15]Sebastiaan P. Huber, Spyros Zoupanos, Martin Uhrin et al. “AiiDA 1.0, a scalable computational infrastructure for automated reproducible workflows and data provenance”. In: *Scientific Data* 7, 300 (2020) (zitiert auf den Seiten 10, 11, 14, 38, 48, 80).
- [16]Elise Larssonneur, Jonathan Mercier, Nicolas Wiart et al. “Evaluating Workflow Management Systems: A Bioinformatics Use Case”. In: *2018 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. 2018, S. 2773–2775 (zitiert auf Seite 11).
- [28]J. Luc Peterson, Ben Bay, Joe Koning et al. “Enabling machine learning-ready HPC ensembles with Merlin”. In: *Future Generation Computer Systems* 131 (2022), S. 255–268 (zitiert auf den Seiten 9, 14, 35, 80).
- [29]Nick Russell, Arthur H. M. ter Hofstede, Wil M. P. van der Aalst und Nataliya Mulyar. *Workflow Control-Flow Patterns: A Revised View*. Techn. Ber. BPM-06-22. BPM Center, 2006 (zitiert auf Seite 6).
- [30]Ludovica Sacco, Carolina Sopranzetti und Sandro Fiore. “Enabling Provenance Tracking in Workflow Management Systems”. In: *2024 IEEE International Conference on Big Data (BigData)*. 2024, S. 4402–4409 (zitiert auf Seite 16).
- [31]Rina Singh, Jeffrey A. Graves, Valentine Anantharaj und Sreenivas R. Sukumar. “Evaluating Scientific Workflow Engines for Data and Compute Intensive Discoveries”. In: *2019 IEEE International Conference on Big Data (Big Data)*. 2019, S. 4553–4560 (zitiert auf Seite 11).
- [39]Frédéric Suter, Tainã Coleman, İlkay Altıntaş et al. “A terminology for scientific workflow systems”. In: *Future Generation Computer Systems* 174, 107974 (2026) (zitiert auf den Seiten 1, 5–7, 14, 15, 18, 77).
- [42]Zhangbing Zhou, Sami Bhiri und Manfred Hauswirth. “Control and data dependencies in business processes based on semantic business activities”. In: *Proceedings of the 10th International Conference on Information Integration and Web-based Applications & Services*. 2008, S. 257–263 (zitiert auf Seite 6).

Webpages

- [@1]AiiDA-Dokumentation 2.8, *Installationsanleitung*. AiiDA Team. URL: https://aiida.readthedocs.io/projects/aiida-core/en/stable/installation/guide_complete.html (besucht am 1. Aug. 2026) (zitiert auf den Seiten 30, 64).
- [@2]AiiDA-Dokumentation, *Visualisierung des Provenance-Graphen*. AiiDA Team. URL: https://aiida.readthedocs.io/projects/aiida-core/en/latest/howto/visualising_graphs.html (besucht am 1. Aug. 2026) (zitiert auf Seite 39).

- [@13]*High Performance Computing*. Zentrum für Datenverarbeitung, Johannes Gutenberg-Universität Mainz. URL: <https://hpc.uni-mainz.de/> (besucht am 15. Aug. 2026) (zitiert auf Seite 7).
- [@14]*Homebrew Formulae: nextflow*. Homebrew. URL: <https://formulae.brew.sh/formula/nextflow> (besucht am 1. Aug. 2026) (zitiert auf Seite 29).
- [@17]*Merlin-Dokumentation, Celery*. Lawrence Livermore National Laboratory. URL: https://merlin.readthedocs.io/en/latest/user_guide/celery/ (besucht am 1. Aug. 2026) (zitiert auf den Seiten 30, 35).
- [@18]*Merlin-Dokumentation, Kommandozeilenschnittstelle*. Lawrence Livermore National Laboratory. URL: https://merlin.readthedocs.io/en/stable/user_guide/command_line/ (besucht am 1. Aug. 2026) (zitiert auf Seite 35).
- [@19]*Merlin-Dokumentation, Konfiguration*. Lawrence Livermore National Laboratory. URL: https://merlin.readthedocs.io/en/latest/user_guide/configuration/ (besucht am 16. Aug. 2026) (zitiert auf Seite 9).
- [@20]*MOGON Dokumentation*. Zentrum für Datenverarbeitung, Johannes Gutenberg-Universität Mainz. URL: <https://docs.hpc.uni-mainz.de/> (besucht am 8. Aug. 2026) (zitiert auf Seite 58).
- [@21]*MOGON Dokumentation, What is MOGON?* Zentrum für Datenverarbeitung, Johannes Gutenberg-Universität Mainz. URL: <https://docs.hpc.uni-mainz.de/docs/introduction/what-is-mogon/> (besucht am 19. Aug. 2026) (zitiert auf Seite 61).
- [@22]*Neuer Hochleistungsrechner eingeweiht: MOGON NHR Süd-West*. Zentrum für Datenverarbeitung, Johannes Gutenberg-Universität Mainz. 2023. URL: <https://hpc.uni-mainz.de/2023/03/14/neuer-hochleistungsrechner-eingeweiht-mogon-nhr-sued-west/> (besucht am 8. Aug. 2026) (zitiert auf Seite 58).
- [@23]*Nextflow-Dokumentation, Berichte und Ausführungsprotokolle*. Seqera Labs. URL: <https://www.nextflow.io/docs/latest/reports.html> (besucht am 1. Aug. 2026) (zitiert auf Seite 32).
- [@24]*Nextflow-Dokumentation, Installation*. Seqera Labs. URL: <https://www.nextflow.io/docs/latest/install.html> (besucht am 1. Aug. 2026) (zitiert auf Seite 29).
- [@25]*Pegasus-Dokumentation 5.1.2*. University of Southern California, Information Sciences Institute. URL: <https://pegasus.isi.edu/documentation/index.html> (besucht am 1. Aug. 2026) (zitiert auf Seite 63).
- [@26]*Pegasus-Dokumentation 5.1.2, Installation*. University of Southern California, Information Sciences Institute. URL: <https://pegasus.isi.edu/documentation/user-guide/installation.html> (besucht am 1. Aug. 2026) (zitiert auf Seite 30).

- [@27]*Pegasus-Dokumentation 5.1.2, Optimizing Workflows for Efficiency and Scalability*. University of Southern California, Information Sciences Institute. URL: <https://pegasus.isi.edu/documentation/user-guide/optimization.html> (besucht am 11. Aug. 2026) (zitiert auf Seite 81).
- [@32]*Slurm-Dokumentation 24.11, Prolog and Epilog Guide*. SchedMD LLC. URL: https://slurm.schedmd.com/archive/slurm-24.11.6/prolog_epilog.html (besucht am 15. Aug. 2026) (zitiert auf Seite 8).
- [@33]*Slurm-Dokumentation 24.11, Quick Start Administrator Guide*. SchedMD LLC. URL: https://slurm.schedmd.com/archive/slurm-24.11.6/quickstart_admin.html (besucht am 15. Aug. 2026) (zitiert auf Seite 7).
- [@34]*Slurm-Dokumentation 24.11, Quick Start User Guide*. SchedMD LLC. URL: <https://slurm.schedmd.com/archive/slurm-24.11.6/quickstart.html> (besucht am 15. Aug. 2026) (zitiert auf den Seiten 7, 8).
- [@35]*Slurm-Dokumentation 24.11, sbatch*. SchedMD LLC. URL: <https://slurm.schedmd.com/archive/slurm-24.11.6/sbatch.html> (besucht am 15. Aug. 2026) (zitiert auf den Seiten 7, 8).
- [@36]*StreamFlow 0.2.0, Installation*. Alpha Group, University of Torino. URL: <https://streamflow.di.unito.it/documentation/0.2/guide/install.html> (besucht am 31. Juli 2026) (zitiert auf Seite 30).
- [@37]*StreamFlow-Dokumentation 0.2, inspect*. Alpha Group, University of Torino. URL: <https://streamflow.di.unito.it/documentation/0.2/guide/inspect.html> (besucht am 31. Juli 2026) (zitiert auf Seite 34).
- [@38]*StreamFlow, Quellcode-Repository*. Alpha Group, University of Torino. URL: <https://github.com/alpha-unito/streamflow> (besucht am 31. Juli 2026) (zitiert auf Seite 30).
- [@40]Anas Tahiri. *Repository zur Bachelorarbeit*. 2026. URL: https://gitlab.rlp.net/hpca/abschlussarbeiten/BA/2026_tahiri_workflows (besucht am 24. Aug. 2026) (zitiert auf den Seiten 31, 41).
- [@41]*Workflow Systems*. Workflows Community Initiative. URL: <https://workflows.community/systems> (besucht am 6. Aug. 2026) (zitiert auf den Seiten 14, 15).

Abbildungsverzeichnis

2.1.	Ablauf von der Einreichung eines Rechenauftrags bis zu seiner Ausführung auf einem Slurm-verwalteten HPC-System.	8
3.1.	Aufbau der beiden Benchmark-Workflows. Hervorgehoben ist der compute-Schritt als einziger Träger der Rechenlast, beim Scatter-Gather-Muster in n parallelen Instanzen.	19
3.2.	Ablauf des Mess- und Auswertungsverfahrens für eine Konfiguration. Jede Wiederholung umfasst einen Referenzlauf und den zugehörigen Systemlauf, beide werden nach demselben Verfahren von außen gemessen und erzeugen Timing-Dateien. Aus den Wiederholungen werden anschließend die berichteten Kennwerte gebildet.	22
4.1.	Median der Gesamtlaufzeit des Pipeline-Musters je Rechenlast auf der lokalen Ebene, logarithmische Skala	51
4.2.	Median der Gesamtlaufzeit des Scatter-Gather-Musters über die Zahl der Teilstücke je Rechenlast auf der lokalen Ebene, logarithmische Skala	52
4.3.	Overhead gegenüber dem Referenzlauf für alle zwölf Konfigurationen auf der lokalen Ebene, logarithmische Skala	53
4.4.	Median der Übergangslatenzen zwischen aufeinanderfolgenden Schritten auf der lokalen Ebene, logarithmische Skala	53
4.5.	Median der Startspreizung der parallelen Verarbeitungsinstanzen auf der lokalen Ebene, logarithmische Skala	54
5.1.	Zerlegung der Zeit vom Einreichen bis zum Ende der Nutzlast je Slurm-Job, Median über fünf Wiederholungen mit Nextflow.	67
5.2.	Differenzen der Mediane zwischen exklusivem und geteiltem Knoten (exklusiv minus geteilt). Werte nahe null kennzeichnen geringe Unterschiede zwischen beiden Messreihen.	69
5.3.	Median der Gesamtlaufzeit des Pipeline-Musters je Rechenlast auf dem HPC-System, lineare Skala	70

5.4.	Median der Gesamtlaufzeit des Scatter-Gather-Musters über die Zahl der Teilstücke je Rechenlast auf dem HPC-System	71
5.5.	Overhead gegenüber dem Referenzlauf für alle zwölf Konfigurationen auf dem HPC-System	72
5.6.	Median der Übergangslatenzen zwischen aufeinanderfolgenden Schritten auf dem HPC-System, logarithmische Skala	73
5.7.	Median der Startspreizung der parallelen Verarbeitungsinstanzen auf dem HPC-System, logarithmische Skala	73
5.8.	Overhead der drei AiiDA-Konfigurationen für alle zwölf Konfigurationen auf einem geteilten Knoten	75
A.1.	Von Nextflow erzeugter Workflow-Graph des Pipeline-Workflows	97
A.2.	Zeitstrahl eines Pipeline-Laufs mit der kurzen Rechenlast	97
A.3.	Von Nextflow erzeugter Zeitstrahl eines Pipeline-Laufs mit der mittleren Rechenlast	98
A.4.	Von Nextflow erzeugter Workflow-Graph des Scatter-Gather-Workflows	98
A.5.	Von Nextflow erzeugter Zeitstrahl eines Scatter-Gather-Laufs mit vier Teilstücken und kurzer Rechenlast	98
A.6.	Von StreamFlow erzeugter Zeitstrahl eines Pipeline-Laufs mit der kurzen Rechenlast	99
A.7.	Von StreamFlow erzeugter Zeitstrahl eines Scatter-Gather-Laufs .	99
A.8.	Von Pegasus erzeugte abstrakte Darstellung des Pipeline-Workflows	100
A.9.	Von Pegasus erzeugte Darstellung des Pipeline-Workflows mit den beteiligten Dateien	100
A.10.	Von Pegasus erzeugte Darstellung des geplanten Graphen desselben Workflows	100
A.11.	Von Pegasus erzeugte abstrakte Darstellung des Scatter-Gather-Workflows mit vier Teilstücken	101
A.12.	Von Pegasus erzeugte Darstellung des Scatter-Gather-Workflows mit den beteiligten Dateien	101
A.13.	Von Pegasus erzeugte Darstellung des geplanten Graphen desselben Workflows	101
A.14.	Aus der AiiDA-Datenbank erzeugter Provenance-Graph des Pipeline-Laufs	102
A.15.	Aus der AiiDA-Datenbank erzeugter Provenance-Graph eines Scatter-Gather-Laufs mit vier Teilstücken	102

Tabellenverzeichnis

3.1. Start der Läufe und Erkennung des Laufendes in der äußeren Messung	24
4.1. Lokale Bereitstellung je System: zusätzliche Komponenten, einmalige Konfiguration und vor der Ausführung erforderliche Schritte	31
4.2. Umsetzung des Pipeline-Musters im Überblick	39
4.3. Umsetzung des Scatter-Gather-Musters im Überblick	49

Programmausdrucksver- zeichnis

3.1.	Aufbau eines instrumentierten Benchmark-Schritts	25
4.1.	workflow-Block aus benchmark.nf (Nextflow, Pipeline)	32
4.2.	Verweis auf die Ausgabe des Vorgängers in workflow.cwl (Stream- Flow, Pipeline)	33
4.3.	Abhängigkeit und Datenweitergabe eines Schritts (Merlin, Pipeline)	34
4.4.	Ein- und Ausgaben eines Jobs (Pegasus, Pipeline)	36
4.5.	Reihenfolge in spec.outline() (AiiDA, Pipeline)	37
4.6.	Fan-out und Fan-in im workflow-Block (Nextflow, Scatter-Gather) .	41
4.7.	Fan-out über scatter am compute-Schritt (StreamFlow, Scatter- Gather)	43
4.8.	Parametrisierter Verarbeitungsschritt und Fan-in (Merlin, Scatter- Gather)	44
4.9.	Erzeugung der compute-Jobs je Teilstück (Pegasus, Scatter-Gather)	46
4.10.	Fan-out in submit_compute (AiiDA, Scatter-Gather)	47
5.1.	Konfiguration von Nextflow für die Ausführung auf MOGON	59
5.2.	Auszug aus der Umgebungsbeschreibung von StreamFlow für die Ausführung auf MOGON	60
5.3.	Jobskript zum Start der Merlin-Worker im regulären Slurm-Betrieb auf MOGON	62
5.4.	Rechner-Eintrag und Ressourcenangaben für die Ausführung über Slurm	64

Ergänzende Grafiken

A.1 Laufartefakte in Nextflow

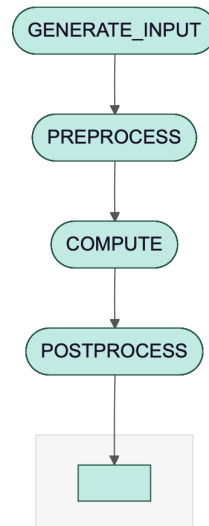


Abb. A.1.: Von Nextflow erzeugter Workflow-Graph des Pipeline-Workflows

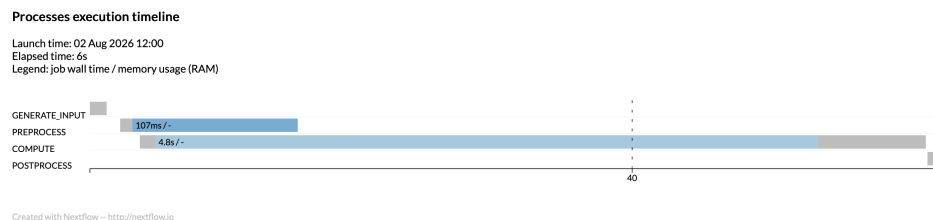


Abb. A.2.: Zeitstrahl eines Pipeline-Laufs mit der kurzen Rechenlast. Der Balken von PREPROCESS wurde auf eine Mindestbreite von circa einer Sekunde gezogen. Der Schritt PREPROCESS dauerte laut Bericht 102 ms, sein Balken erscheint dadurch rund zehnmal zu lang und ragt über den Beginn von COMPUTE hinaus. Die Ausführungszeiten überschneiden sich tatsächlich nicht.

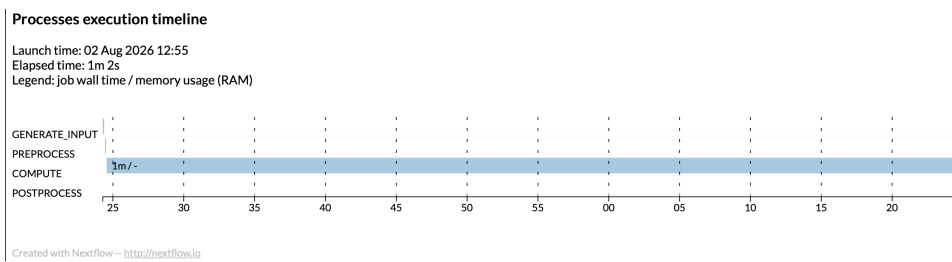


Abb. A.3.: Von Nextflow erzeugter Zeitstrahl eines Pipeline-Laufs mit der mittleren Rechenlast

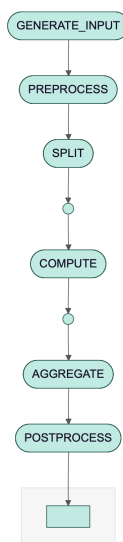


Abb. A.4.: Von Nextflow erzeugter Workflow-Graph des Scatter-Gather-Workflows

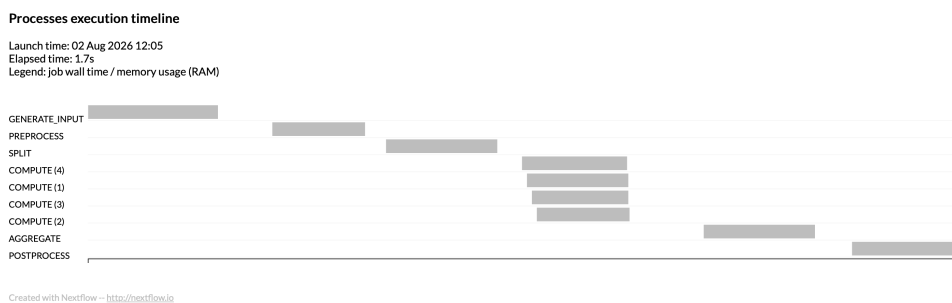


Abb. A.5.: Von Nextflow erzeugter Zeitstrahl eines Scatter-Gather-Laufs mit vier Teilstücken und kurzer Rechenlast

A.2 Laufartefakte in StreamFlow

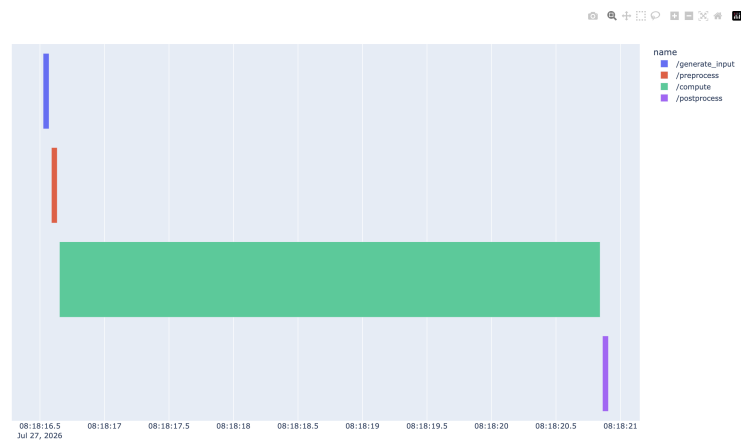


Abb. A.6.: Von StreamFlow erzeugter Zeitstrahl eines Pipeline-Laufs mit der kurzen Rechenlast

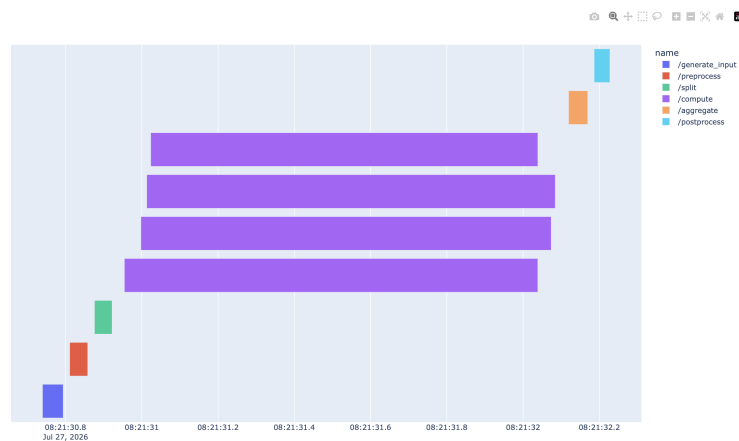


Abb. A.7.: Von StreamFlow erzeugter Zeitstrahl eines Scatter-Gather-Laufs

A.3 Laufartefakte in Pegasus

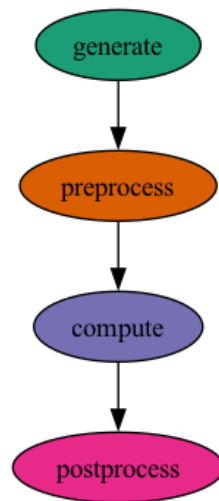


Abb. A.8.: Von Pegasus erzeugte abstrakte Darstellung des Pipeline-Workflows

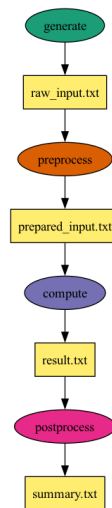


Abb. A.9.: Von Pegasus erzeugte Darstellung des Pipeline-Workflows mit den beteiligten Dateien

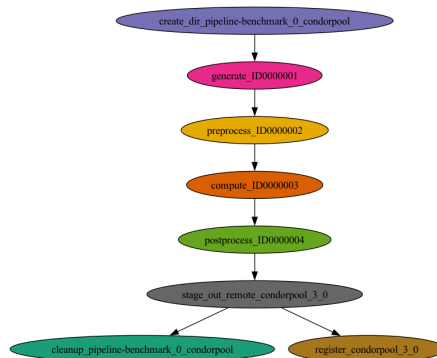


Abb. A.10.: Von Pegasus erzeugte Darstellung des geplanten Graphen desselben Workflows

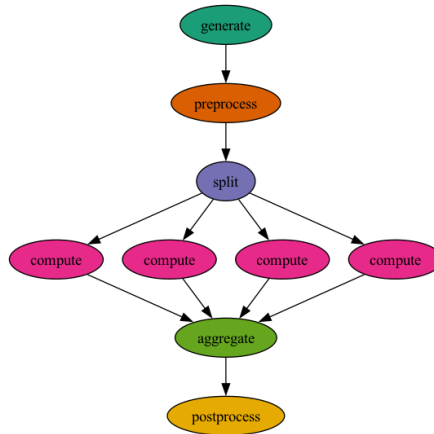


Abb. A.11.: Von Pegasus erzeugte abstrakte Darstellung des Scatter-Gather-Workflows mit vier Teilstücken

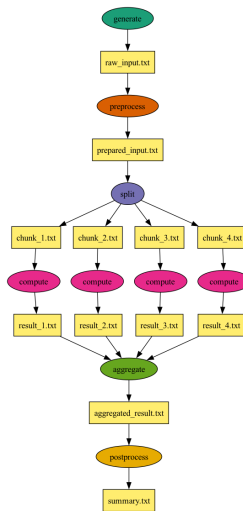


Abb. A.12.: Von Pegasus erzeugte Darstellung des Scatter-Gather-Workflows mit den beteiligten Dateien

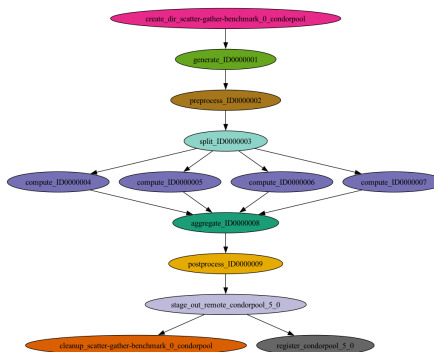


Abb. A.13.: Von Pegasus erzeugte Darstellung des geplanten Graphen desselben Workflows

A.4 Laufartefakte in AiiDA

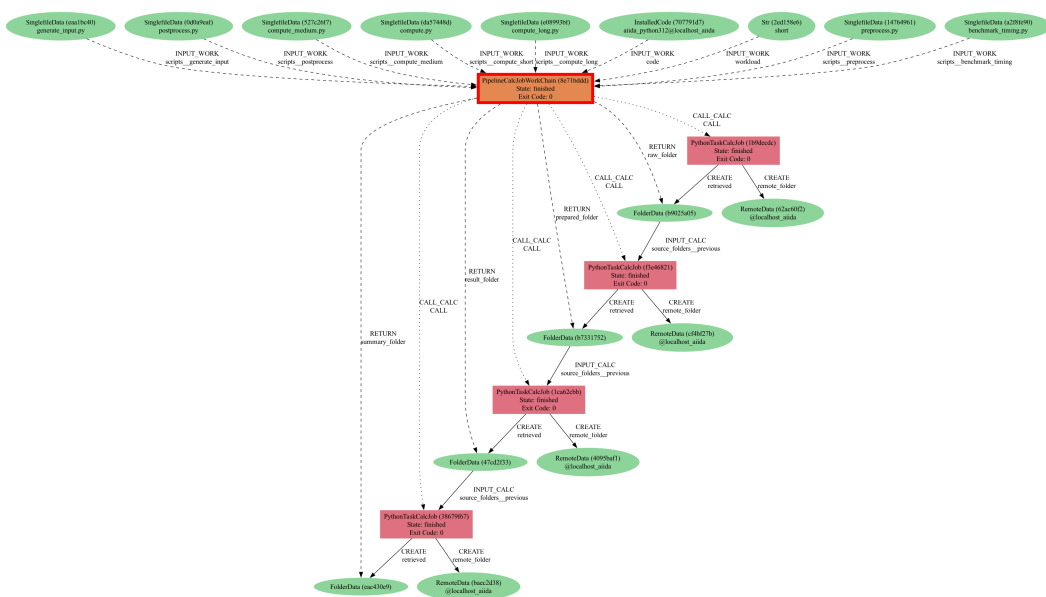


Abb. A.14.: Aus der AiiDA-Datenbank erzeugter Provenance-Graph des Pipeline-Laufs

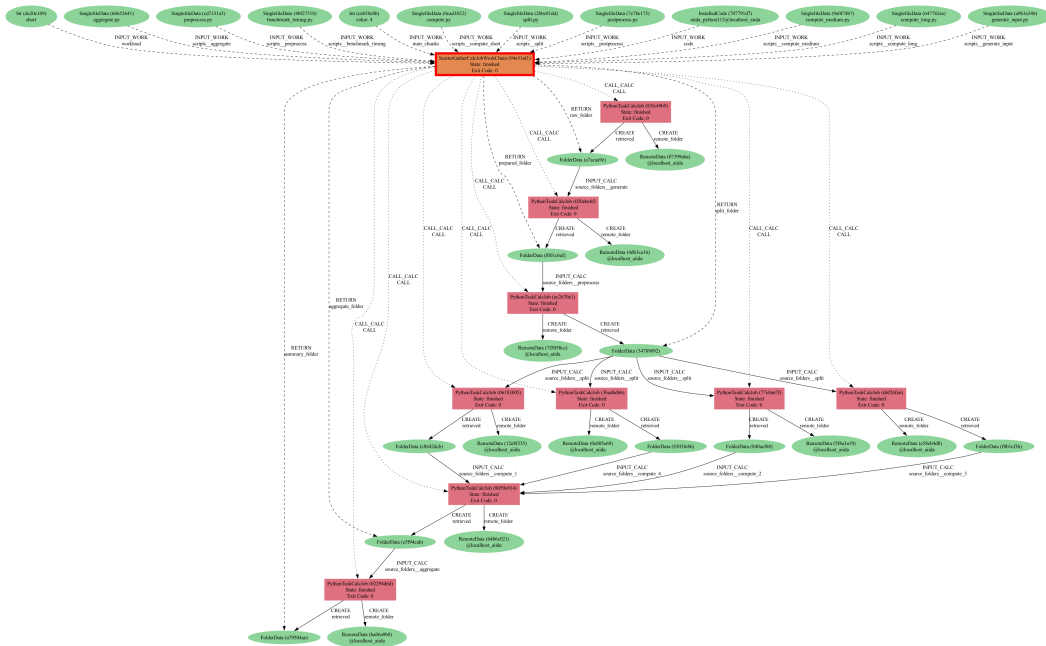


Abb. A.15.: Aus der AiiDA-Datenbank erzeugter Provenance-Graph eines Scatter-Gather-Laufs mit vier Teilstücken

Kolophon

Diese Arbeit wurde mit $\text{\LaTeX}2_{\epsilon}$ gesetzt. Sie verwendet den von Ricardo Langer entwickelten Stil *Clean Thesis*. Das Design des Stils *Clean Thesis* ist von Benutzerhandbüchern von Apple Inc. inspiriert.

Laden Sie den Stil *Clean Thesis* unter <http://cleanthesis.der-ric.de/> herunter.

Anpassungen an den Stil des Instituts für Informatik sind unter <https://gitlab.rlp.net/institut-fur-informatik/cleanthesis-jgu> zu finden.

Eigenständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe (dazu zählen auch „KI-Werkzeuge“). Sämtliche wörtlichen und sinngemäßen Übernahmen und Zitate sind kenntlich gemacht und nachgewiesen. Auch beim Einsatz von KI-Werkzeugen¹ stellt die Arbeit klar dar, in welchem Umfang diese genutzt wurden und welche Inhalte dadurch erzeugt oder beeinflusst wurden². Ich versichere, dass ich keine Hilfsmittel verwendet habe, deren Nutzung die Prüferinnen und Prüfer explizit ausgeschlossen haben.

Mit Abgabe der vorliegenden Leistung übernehme ich die Verantwortung für das eingereichte Gesamtprodukt. Ich verantworte damit auch alle KI-generierten Inhalte, die ich in meine Arbeit übernommen habe. Die Richtigkeit übernommener (KI-generierter) Aussagen und Inhalte habe ich nach bestem Wissen und Gewissen geprüft.

Ich habe die Arbeit nicht zum Erwerb eines anderen Leistungsnachweises in gleicher oder ähnlicher Form eingereicht. Von der Ordnung zur Sicherung guter wissenschaftlicher Praxis und zum Umgang mit wissenschaftlichem Fehlverhalten habe ich Kenntnis genommen.

Mir ist bekannt, dass ein Verstoß gegen die genannten Punkte prüfungsrechtliche Konsequenzen hat und insbesondere dazu führen kann, dass die Studien- und Prüfungsleistung als mit „nicht bestanden“ bewertet wird. Die Einschreibung kann für bis zu zwei Jahre widerrufen werden, wenn Studierende

¹Mit „KI-Werkzeugen“ werden computergestützte Systeme bezeichnet, die auf Basis maschinellen Lernens neue Inhalte generieren können (z. B. ChatGPT, Gemini, Claude, LLaMA, Midjourney, Stable Diffusion o. ä.).

²Die Nutzung von KI-Werkzeugen muss dann kenntlich gemacht werden, wenn Sie den Kern der Aufgabenstellung in von den Prüfern nicht anzunehmender Weise berührt: Sollte nichts anderes explizit mit den Prüferinnen und Prüfern vereinbart worden sein, so ist dies der Fall, sobald wesentliche Teile der eingereichten Arbeit (z. B. ganze Sätze des Textes, ganze Abbildungen, nicht-offensichtliche Teile von Rechnungen oder Beweisen, mehrere Zeilen von Programmcode am Stück) von solchen Werkzeugen generiert wurden und wörtlich oder in abgewandelter Form in die Arbeit übernommen wurden oder wesentliche Konzepte oder Ideen (z. B. eine Gliederung der Literatur oder ein Lösungsansatz für ein Problem) von KI-Werkzeugen generiert wurden.

zweimal oder häufiger bei Prüfungsleistungen täuschen (§ 69 Abs. 4 und 5 HochSchG).

Mainz, 25.08.2026



Anas Tahiri

Hinweis: Es wird im Falle von abweichenden Vereinbarungen mit den Prüferinnen und Prüfern empfohlen, jene im Vorfeld schriftlich festzuhalten und zusätzlich auch noch einmal in der eingereichten Arbeit zu benennen.

Nutzung von KI-Tools.

KI Tool	Genutzt für	Grund	Wann
ChatGPT, Claude	Erzeugung und Anpassung von Programmcode und Konfigurationen, technische Umsetzung von Abbildungen, Unterstützung bei Fehlersuche sowie Verarbeitung und Visualisierung von Messdaten	Unterstützung bei der technischen Umsetzung und Verarbeitung der Messergebnisse	Im praktischen Teil der Arbeit
ChatGPT, Claude	Beantwortung von Verständnisfragen zu Themen der Arbeit	Unterstützung beim Verständnis	Über die gesamte Arbeit hinweg
ChatGPT, Claude	Formulierungsvorschläge sowie Neuformulierung und sprachliche Überarbeitung meiner Textentwürfe	Verbesserung der Lesbarkeit und Verständlichkeit	Über die gesamte Arbeit hinweg
ChatGPT, Claude	Unterstützung bei der Strukturierung einzelner Abschnitte	Verbesserung des Aufbaus	Während der Ausarbeitung
ChatGPT	Gegenlesen der Arbeit	Erkennen möglicher Fehler	Nach Fertigstellung der Arbeit
Elicit	Literaturrecherche	Suche nach relevanter wissenschaftlicher Literatur	Während der Literaturrecherche

